

作者：Jason，在读研究生，熟悉硬件、STM32单片机、嵌入式Linux。已收获小米、联发科、浙江大华等大厂offer。在公众号分享嵌入式学习规划、嵌入式笔试面试题目、嵌入式求职规划、嵌入式Linux技术文章等。关注公众号回复【电子书】即可获得嵌入式全套500本电子书。

大号：《嵌入式Linux系统开发》，主做技术分享，关注博主免费修改简历。



小号：《程序员杰仔》，主要分享博主日常学习笔记。



我创建了知识星球，在知识星球中有问必答，解答读者各种问题，长期更新嵌入式精选资料。



本系列按类别对题目进行分类整理，这样有利于大家对嵌入式的笔试面试考察框架有一个完整的理解。

在其他平台转载发布请联系博主，翻版必究。以下所有内容已在微信公众号平台标明原创。

第一章、进程与线程

1、什么是进程、线程，有什么区别？

进程是资源（CPU、内存等）分配的基本单位，线程是CPU调度和分配的基本单位（程序执行的最小单位）。同一时间，如果CPU是单核，只有一个进程在执行，所谓的并发执行，也是顺序执行，只不过由于切换速度太快，你以为这些进程在同步执行而已。多核CPU可以同一时间点有多个进程在执行。

2、多进程、多线程的优缺点

说明：一个进程由进程控制块、数据段、代码段组成，进程本身不可以运行程序，而是像一个容器一样，先创建一个主线程，分配给主线程一定的系统资源，这时候就可以在主线程开始实现各种功能。当我们需要实现更复杂的功能时，可以在主线程里创建多个子线程，多个线程在同一个进程里，利用这个进程所拥有的系统资源合作完成某些功能。

优缺点：1）一个进程死了不影响其他进程，一个线程崩溃很可能影响到它本身所处的整个进程。2）创建多进程的系统开销大于创建多线程。3）多进程通讯因为需要跨越进程边界，不适合大量数据的传送，适合小数据或者密集数据的传送。多线程无需跨越进程边界，适合各线程间大量数据的传送。并且多线程可以共享同一进程里的共享内存和变量。

3、什么时候用进程，什么时候用线程

1）创建和销毁较频繁使用线程，因为创建进程开销大。2）需要大量数据传送使用线程，因为多线程切换速度快，不需要跨越进程边界。3）安全稳定选进程；快速频繁选线程；

4、多进程、多线程同步（通讯）的方法

进程间通讯：

（1）有名管道/无名管道（2）信号（3）共享内存（4）消息队列（5）信号量（6）socket

线程通讯（锁）：

（1）信号量（2）读写锁（3）条件变量（4）互斥锁（5）自旋锁

5、进程线程的状态转换图

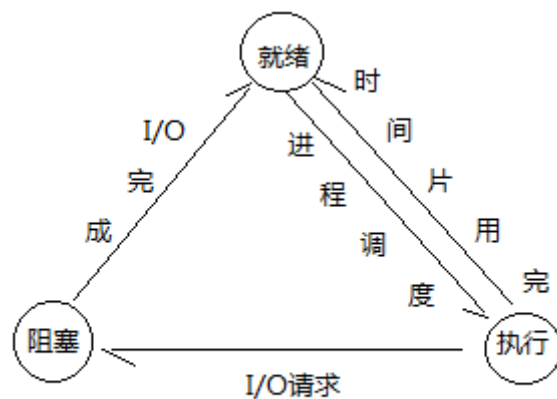
（1）就绪状态：进程已获得除CPU外的所有必要资源，只等待CPU时的状态。一个系统会将多个处于就绪状态的进程排成一个就绪队列。

（2）执行状态：进程已获CPU，正在执行。单处理机系统中，处于执行状态的进程只有一个；多处理机系统中，有多个处于执行状态的进程。

（3）阻塞状态：正在执行的进程由于某种原因而暂时无法继续执行，便放弃处理机而处于暂停状态，即进程执行受阻。（这种状态又称等待状态或封锁状态）

通常导致进程阻塞的典型事件有：请求I/O，申请缓冲空间等。

一般，将处于阻塞状态的进程排成一个队列，有的系统还根据阻塞原因不同把这些阻塞集成排成多个队列。



进程的三种基本状态及转换

(1) 就绪→执行

处于就绪状态的进程，当进程调度程序为之分配了处理机后，该进程便由就绪状态转变成执行状态。

(2) 执行→就绪

处于执行状态的进程在其执行过程中，因分配给它的一个时间片已用完而不得不让出处理机，于是进程从执行状态转变成就绪状态。

(3) 执行→阻塞

正在执行的进程因等待某种事件发生而无法继续执行时，便从执行状态变成阻塞状态。

(4) 阻塞→就绪

处于阻塞状态的进程，若其等待的事件已经发生，于是进程由阻塞状态转变为就绪状态。

6、父进程、子进程

父进程调用fork()以后，克隆出一个子进程，子进程和父进程拥有相同内容的代码段、数据段和用户堆栈。父进程和子进程谁先执行不一定，看CPU。所以我们一般会设置父进程等待子进程执行完毕。

7、说明什么是上下文切换？

你可以有很多角度，有进程上下文，有中断上下文。

进程上下文：一个进程在执行的时候，CPU的所有寄存器中的值、进程的状态以及堆栈中的内容，当内核需要切换到另一个进程时，它需要保存当前进程的所有状态，即保存当前进程的进程上下文，以便再次执行该进程时，能够恢复切换时的状态，继续执行。

中断上下文：由于触发信号，导致CPU中断当前进程，转而去执行另外的程序。那么当前进程的所有资源要保存，比如堆栈和指针。保存过后转而去执行中断处理程序，快速执行完毕返回，返回后恢复上一个进程的资源，继续执行。这就是中断的上下文。

第二章、C/C++题目

1、new和malloc

做嵌入式，对于内存是十分在意的，因为可用内存有限，所以嵌入式笔试面试题目，内存的题目高频。

1) malloc和free是c++/c语言的库函数，需要头文件支持stdlib.h；new和delete是C++的关键字，不需要头文件，需要编译器支持；

2) 使用new操作符申请内存分配时，无需指定内存块的大小，编译器会根据类型信息自行计算。而malloc则需要显式地支持所需内存的大小。

3) new操作符内存分配成功时，返回的是对象类型的指针，类型严格与对象匹配，无需进行类型转换，故new是符合类型安全性的操作符。而malloc内存分配成功则是返回void，需要通过强制类型转换将void指针转换成我们需要的类型。

4) new内存分配失败时，会抛出bad_alloc异常。malloc分配内存失败时返回NULL。

2、在1G内存的计算机中能否malloc(1.2G)？为什么？（2021浙江大华二面问题）

答：是有可能申请1.2G的内存的。

解析：回答这个问题前需要知道malloc的作用和原理，应用程序通过malloc函数可以向程序的虚拟空间申请一块虚拟地址空间，与物理内存没有直接关系，得到的是在虚拟地址空间中的地址，之后程序运行所提供的物理内存是由操作系统完成的。

3、extern“C”的作用

我们可以在C++中使用C的已编译好的函数模块，这时候就需要用到extern“C”。也就是extern“C”都是在c++文件里添加的。

extern在链接阶段起作用（四大阶段：预处理--编译--汇编--链接）。

4、strcat、strncat、strcmp、strcpy哪些函数会导致内存溢出？如何改进？（2021浙江大华二面问题）

strcpy函数会导致内存溢出。

strcpy拷贝函数不安全，他不做任何的检查措施，也不判断拷贝大小，不判断目的地址内存是否够用。

```
1 char *strcpy(char *strDest, const char *strSrc)
```

strncpy拷贝函数，虽然计算了复制的大小，但是也不安全，没有检查目标的边界。

```
1 strncpy(dest, src, sizeof(dest));
```

strncpy_s是安全的

strcmp(str1,str2)，是比较函数，若str1=str2，则返回零；若str1<str2，则返回负数；若str1>str2，则返回正数。（比较字符串）

strncat()主要功能是在字符串的结尾追加n个字符。

```
1 char * strncat(char *dest, const char *src, size_t n);
```

strcat()函数主要用来将两个char类型连接。例如：

```
1 char d[20]="Golden";
2 char s[20]="view";
3 strcat(d,s);
4 //打印d
5 printf("%s",d);
```

输出 d 为 GoldenView（中间无空格）

延伸：

memcpy拷贝函数，它与strcpy的区别就是memcpy可以拷贝任意类型的数据，strcpy只能拷贝字符串类型。

memcpy 函数用于把资源内存（src所指向的内存区域）拷贝到目标内存（dest所指向的内存区域）；
有一个size变量控制拷贝的字节数；

函数原型：

```
1 void *memcpy(void *dest, void *src, unsigned int count);
```

5、static的用法（定义和用途）（必考）

1) 用static修饰局部变量：使其变为静态存储方式(静态数据区)，那么这个局部变量在函数执行完成之后不会被释放，而是继续保留在内存中。

2) 用static修饰全局变量：使其只在本文件内部有效，而其他文件不可连接或引用该变量。

3) 用static修饰函数：对函数的连接方式产生影响，使得函数只在本文件内部有效，对其他文件是不可见的（这一点在大工程中很重要很重要，避免很多麻烦，很常见）。这样的函数又叫作静态函数。使用静态函数的好处是，不用担心与其他文件的同名函数产生干扰，另外也是对函数本身的一种保护机制。

6、const的用法（定义和用途）（必考）

const主要用来修饰变量、函数形参和类成员函数：

1) 用const修饰常量：定义时就初始化，以后不能更改。

2) 用const修饰形参：func(const int a){};该形参在函数里不能改变

3) 用const修饰类成员函数：该函数对成员变量只能进行只读操作，就是const类成员函数是不能修改成员变量的数值的。

被const修饰的东西都受到强制保护，可以预防意外的变动，能提高程序的健壮性。

参考一个大佬的回答：

我只要一听到被面试者说：“const意味着常数”，我就知道我正在和一个业余者打交道。去年Dan Saks已经在他的文章里完全概括了const的所有用法，因此ESP(译者：Embedded Systems Programming)的每一位读者应该非常熟悉const能做什么和不能做什么.如果你从没有读到那篇文章，只要能说出const意味着“只读”就可以了。尽管这个答案不是完全的答案，但我接受它作为一个正确的答案。如果应试者能正确回答这个问题，我将问他一个附加的问题：下面的声明都是什么意思？

```
1 const int a;  
2 int const a;  
3 const int *a;  
4 int * const a;  
5 int const * a const;
```

前两个的作用是一样，a是一个常整型数。

第三个意味着a是一个指向常整型数的指针（也就是，整型数是不可修改的，但指针可以）。

第四个意思a是一个指向整型数的常指针（也就是说，指针指向的整型数是可以修改的，但指针是不可修改的）。

最后一个意味着a是一个指向常整型数的常指针（也就是说，指针指向的整型数是不可修改的，同时指针也是不可修改的）。

7、volatile作用和用法

一个定义为volatile的变量是说这变量可能会被意想不到地改变，这样，编译器就不会去假设这个变量的值了。精确地说就是，优化器在用到这个变量时必须每次都小心地重新读取这个变量在内存中的值，而不是使用保存在寄存器里的备份（虽然读写寄存器比读写内存快）。

回答不出这个问题的人是不会被雇佣的。这是区分C程序员和嵌入式系统程序员的最基本的问题。搞嵌入式的小伙伴们经常同硬件、中断、RTOS等等打交道，所有这些都要求用到volatile变量。不懂得volatile的内容将会带来灾难。

以下几种情况都会用到volatile：

- 1、并行设备的硬件寄存器（如：状态寄存器）
- 2、一个中断服务子程序中会访问到的非自动变量
- 3、多线程应用中被几个任务共享的变量

8、const常量和#define的区别（编译阶段、安全性、内存占用等）

用#define max 100；定义的常量是没有类型的（不进行类型安全检查，可能会产生意想不到的错误），所给出的是一个立即数，编译器只是把所定义的常量值与所定义的常量的名字联系起来，define所定义的宏变量在预处理阶段的时候进行替换，在程序中使用到该常量的地方都要进行拷贝替换；

用const int max = 255；定义的常量有类型（编译时会进行类型检查）名字，存放在内存的静态区域中，在编译时确定其值。在程序运行过程中const变量只有一个拷贝，而#define所定义的宏变量却有多多个拷贝，所以宏定义在程序运行过程中所消耗的内存要比const变量的大得多

9、变量的作用域（全局变量和局部变量）

全局变量：在所有函数体的外部定义的，程序的所在部分（甚至其它文件中的代码）都可以使用。全局变量不受作用域的影响（也就是说，全局变量的生命期一直到程序的结束）。

局部变量：出现在一个作用域内，它们是局限于一个函数的。局部变量经常被称为自动变量，因为它们进入作用域时自动生成，离开作用域时自动消失。关键字auto可以显式地说明这个问题，但是局部变量默认为auto，所以没有必要声明为auto。

局部变量可以和全局变量重名，在局部变量作用域范围内，全局变量失效，采用的是局部变量的值。

10、sizeof 与strlen（字符串，数组）

1.如果是数组

```
1  #include<stdio.h>
2  int main()
3  {
4      int a[5]={1,2,3,4,5};
5      printf("sizeof 数组名=%d\n",sizeof(a));
6      printf("sizeof *数组名=%d\n",sizeof(*a));
7  }
```

运行结果

```
1  sizeof 数组名=20
2  sizeof *数组名=4
```

2.如果是指针，sizeof只会检测到是指针的类型，指针都是占用4个字节的空间（32位机）。

sizeof是什么？是一个操作符,也是关键字，就不是一个函数，这和strlen()不同，strlen()是一个函数。

那么sizeof的作用是什么？返回一个对象或者类型所占的内存字节数。我们会对sizeof()中的数据或者指针做运算吗？基本不会。例如sizeof(1+2.0),直接检测到其中类型是double,即是sizeof(double) = 8。如果是指针，sizeof只会检测到是指针的类型，指针都是占用4个字节的空间（32位机）。

```
1 char *p = "sadasdasd";
2 sizeof(p):4
3 sizeof(*p):1//指向一个char类型的
```

除非使用strlen()，仅对字符串有效，直到'\0'为止了，计数结果不包括\0。

要是非要使用sizeof来得到指向内容的大小，就得使用数组名才行，如

```
1 char a[10];
2 sizeof(a):10 //检测到a是一个数组的类型。
```

数据类型	32位机 (字节)	64位机 (字节)	备注
char	1	1	
short	2	2	
int	4	4	
long	4	8	32位与64位不同
float	4	4	
char *	4	8	其他指针类型如long *, int * 也是如此
long long	8	8	
double	8	8	
long double	10/12	10/16	有效位10字节。32位为了对齐实际分配12字节；64位分配16字节

关于strlen()，它是一个函数，考察的比较简单：

```
1 strlen "\n\t\ttag\AA tang"
```

答案：11

11、经典的sizeof(struct)和sizeof(union)内存对齐

内存对齐作用：

- 1.平台原因(移植原因)：不是所有的硬件平台都能访问任意地址上的任意数据的；某些硬件平台只能在某些地址处取某些特定类型的数据，否则抛出硬件异常。
- 2.性能原因：数据结构(尤其是栈)应该尽可能地在自然边界上对齐。原因在于，为了访问未对齐的内存，处理器需要作两次内存访问；而对齐的内存访问仅需要一次访问。

结构体struct内存对齐的3大规则:

- 1.对于结构体的各个成员，第一个成员的偏移量是0，排列在后面的成员其当前偏移量必须是当前成员类型的整数倍；
- 2.结构体内所有数据成员各自内存对齐后，结构体本身还要进行一次内存对齐，保证整个结构体占用内存大小是结构体内最大数据成员的最小整数倍；
- 3.如程序中有#pragma pack(n)预编译指令，则所有成员对齐以n字节为准(即偏移量是n的整数倍)，不再考虑当前类型以及最大结构体内类型。

```
1  #pragma pack(1)
2
3  struct fun{
4      int i;
5      double d;
6      char c;
7  };
```

sizeof(fun) = 13

```
1  struct CAT_s
2  {
3      int Id;
4      char Color;
5      unsigned short Age;
6      char *Name;
7      void(*Jump)(void);
8  }Garfield;
```

- 1.使用32位编译，int占4，char占1，unsigned short占2，char*占4，函数指针占4个，由于是32位编译是4字节对齐，所以该结构体占16个字节。（说明：按几字节对齐，是根据结构体的最长类型决定的，这里是int是最长的字节，所以按4字节对齐）；
- 2.使用64位编译，int占4，char占1，unsigned short占2，char*占8，函数指针占8个，由于是64位编译是8字节对齐（说明：按几字节对齐，是根据结构体的最长类型决定的，这里是函数指针是最长的字节，所以按8字节对齐）所以该结构体占24个字节。

```

1 //64位
2 struct C
3 {
4     double t;    //8    1111 1111
5     char b;    //1    1
6     int a;    //4    0001111
7     short c;    //2    11000000
8 };
9 sizeof(C) = 24; //注意: 1 4 2 不能拼在一起

```

char是1，然后在int之前，地址偏移量得是4的倍数，所以char后面补三个字节，也就是char占了4个字节，然后int四个字节，最后是short，只占两个字节，但是总的偏移量得是double的倍数，也就是8的倍数，所以short后面补六个字节

联合体union内存对齐的2大规则:

- 1.找到占用字节最多的成员；
- 2.union的字节数必须是占用字节最多的成员的字节的倍数，而且需要能够容纳其他的成员

```

1 //x64
2 typedef union {
3     long i;
4     int k[5];
5     char c;
6 }D

```

要计算union的大小,首先要找到占用字节最多的成员,本例中是long,占用8个字节,int k[5]中都是int类型,仍然是占用4个字节的，然后union的字节数必须是占用字节最多的成员的字节的倍数,而且需要能够容纳其他的成员,为了要容纳k(20个字节),就必须要保证是8的倍数的同时还要大于20个字节,所以是24个字节。

引申：位域（大疆笔试题）

C语言允许在一个结构体中以位为单位来指定其成员所占内存长度，这种以位为单位的成员称为“位段”或称“位域”(bit field)。利用位段能够用较少的位数存储数据。一个位段必须存储在同一存储单元中，不能跨两个单元。如果第一个单元空间不能容纳下一个位段，则该空间不用，而从下一个单元起存放该位段。

- 1.位段声明和结构体类似
- 2.位段的成员必须是int、unsigned int、signed int
- 3.位段的成员名后边有一个冒号和一个数字

```

1  typedef struct_data{
2      char m:3;
3      char n:5;
4      short s;
5
6      union{
7          int a;
8          char b;
9      };
10
11     int h;
12 }__attribute__((packed)) data_t;

```

答案12

m和n一起，刚好占用一个字节内存，因为后面是short类型变量，所以在short s之前，应该补一个字节。所以m和n其实是占了两个字节的，然后是short两个字节，加起来就4个字节，然后联合体占了四个字节，总共8个字节了，最后int h占了四个字节，就是12个字节了

`attribute((packed))` 取消对齐

GNU C的一大特色就是**attribute**机制。**attribute**可以设置函数属性（Function Attribute）、变量属性（Variable Attribute）和类型属性（Type Attribute）。

attribute书写特征是：**attribute**前后都有两个下划线，并且后面会紧跟一对括弧，括弧里面是相应的**attribute**参数。

跨平台通信时用到。不同平台内存对齐方式不同。如果使用结构体进行平台间的通信，会有问题。例如，发送消息的平台上，结构体为24字节，接受消息的平台上，此结构体为32字节（只是随便举个例子），那么每个变量对应的值就不对了。

不同框架的处理器对齐方式会有不同，这个时候不指定对齐的话，会产生错误结果

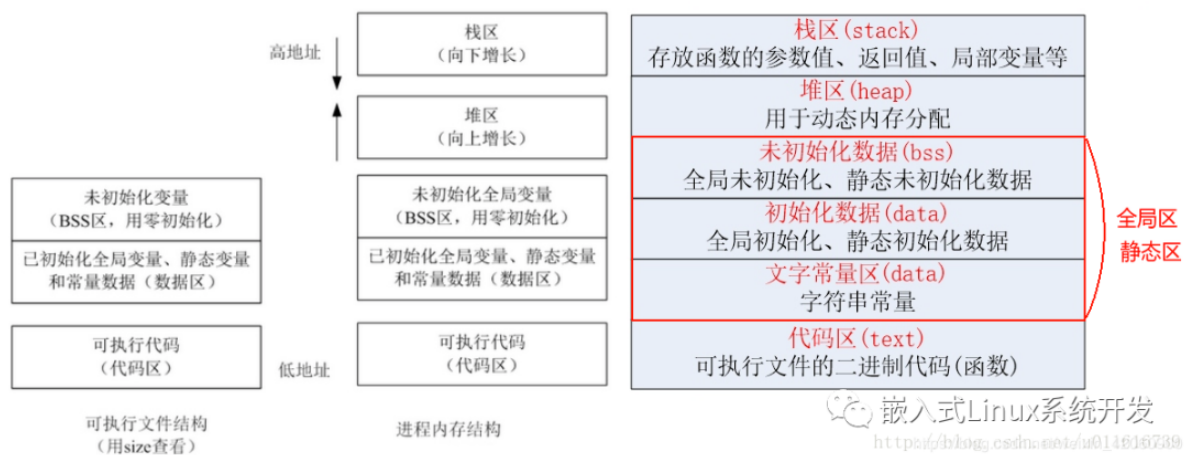
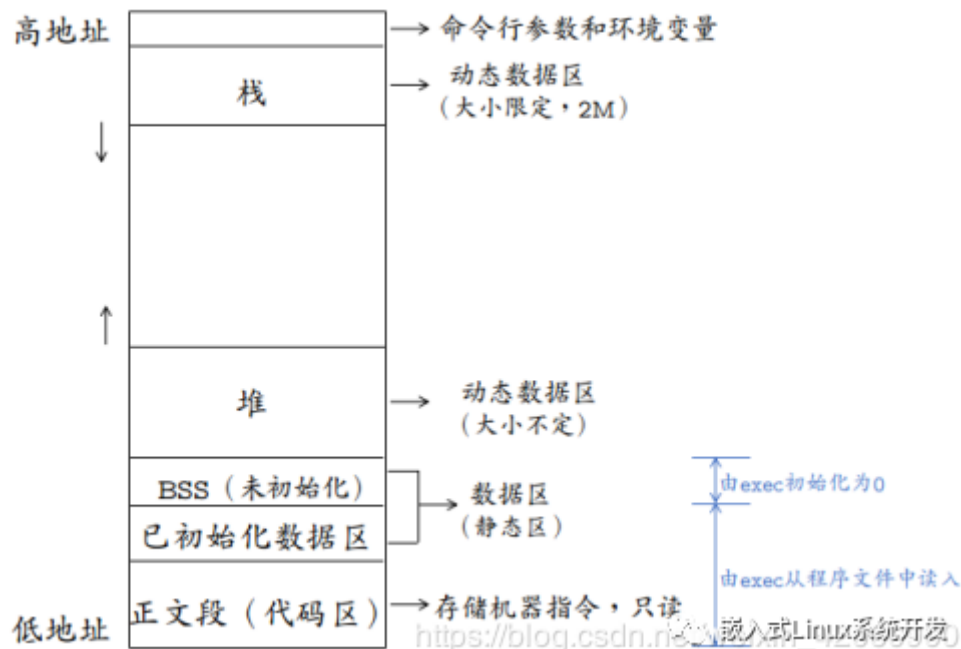
12、inline函数

在C语言中，如果一些函数被频繁调用，不断地有函数入栈，即函数栈，会造成栈空间或栈内存的大量消耗。为了解决这个问题，特别的引入了inline修饰符，表示为内联函数。

大多数的机器上，调用函数都要做很多工作：调用前要先保存寄存器，并在返回时恢复，复制实参，程序还必须转向一个新位置执行C++中支持内联函数，其目的是为了提高函数的执行效率，用关键字 inline 放在函数定义(注意是定义而非声明)的前面即可将函数指定为内联函数，内联函数通常就是将它程序中的每个调用点上“内联地”展开。

内联是以代码膨胀（复制）为代价，仅仅省去了函数调用的开销，从而提高函数的执行效率。

13、内存四区，什么变量分别存储在什么区域，堆上还是栈上。



文字常量区, 叫.rodata, 不可以改变, 改变会导致段错误

```

1  int a0=1;
2  static int a1;
3  const static a2=0;
4  extern int a3;
5
6  void fun(void)
7  {
8      int a4;
9      volatile int a5;
10     return;
11 }

```

a0 : 全局初始化变量 ; 生命周期为整个程序运行期间 ; 作用域为所有文件 ; 存储位置为data段。

a1 : 全局静态未初始化变量 ; 生命周期为整个程序运行期间 ; 作用域为当前文件 ; 储存位置为BSS段。

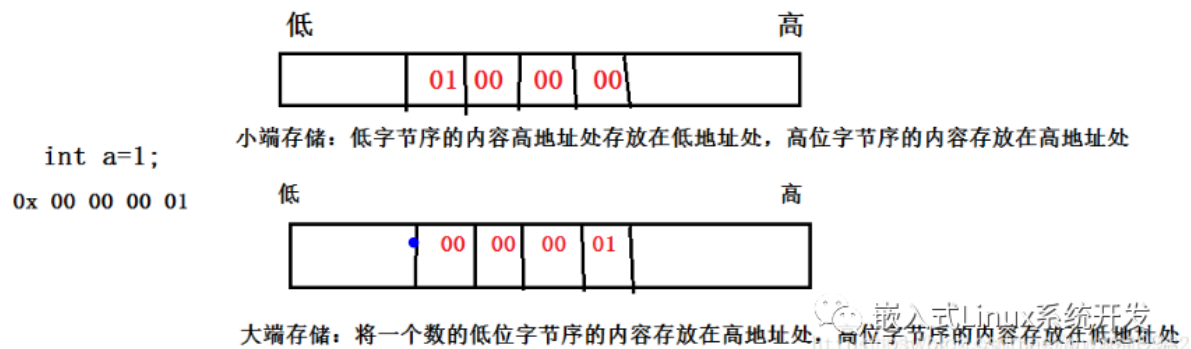
a2 : 全局静态变量

a3 : 全局初始化变量 ; 其他同a0。

a4 : 局部变量 ; 生命周期为fun函数运行期间 ; 作用域为fun函数内部 ; 储存位置为栈。

a5 : 局部易变变量 ;

14、使用32位编译情况下，给出判断所使用机器大小端的方法。



联合体方法判断方法：利用union结构体的从低地址开始存，且同一时间内只有一个成员占有内存的特性。大端存储符合阅读习惯。联合体占用内存是最大的那个，和结构体不一样。

a和c公用同一块内存区域，所以更改c，必然会影响a的数据

```
1 #include<stdio.h>
2
3 int main(){
4     union w
5     {
6         int a;
7         char b;
8     };
9     c.a = 1;
10    if(c.b == 1)
11        printf("小端存储\n");
12    else
13        printf("大端存储\n");
14    return 0;
15 }
```

指针方法

通过将int强制类型转换成char单字节，p指向a的起始字节（低字节）

```
1 #include <stdio.h>
2 int main ()
3 {
4     int a = 1;
5     char *p = (char *)&a;
6     if(*p == 1)
7     {
8         printf("小端存储\n");
9     }
10    else
11    {
12        printf("大端存储\n");
13    }
14    return 0;
15 }
```

15、用变量a给出下面的定义

```
1  a) 一个整型数；
2  b) 一个指向整型数的指针；
3  c) 一个指向指针的指针，它指向的指针是指向一个整型数；
4  d) 一个有10个整型的数组；
5  e) 一个有10个指针的数组，该指针是指向一个整型数；
6  f) 一个指向有10个整型数数组的指针；
7  g) 一个指向函数的指针，该函数有一个整型参数并返回一个整型数；
8  h) 一个有10个指针的数组，该指针指向一个函数，该函数有一个整型参数并返回一个整型数
9  答案：
10 a) int a
11 b) int *a;
12 c) int **a;
13 d) int a[10];
14 e) int *a [10];
15 f) int a[10], *p=a;
16 g) int (*a)(int)
17 h) int( *a[10])(int)
```

16、与或非，异或。运算符优先级

sum=a&b<<c+a^c;

其中a=3,b=5,c=4 (先加再移位再&再异或) 答案4

优先级	运算符	结合性
1	() [] .	从左到右
2	! + (正) - (负) ~ ++ --	从右向左
3	* / %	从左向右
4	+ (加) - (减)	从左向右
5	<< >> >>>	从左向右
6	< <= > >= instanceof	从左向右
7	== !=	从左向右
8	& (按位与)	从左向右
9	^	从左向右
10		从左向右
11	&&	从左向右
12		从左向右
13	?:	从右向左
14	= += -= *= /= % = &= = ^= ~< <= >> >>=	从右向左

第三章、网络编程

1、TCP、UDP的区别

TCP---传输控制协议,提供的是面向连接、可靠的字节流服务。当客户和服务器彼此交换数据前,必须先在双方之间建立一个TCP连接,之后才能传输数据。

UDP---用户数据报协议,是一个简单的面向数据报的运输层协议。UDP不提供可靠性,它只是把应用程序传给IP层的数据报发送出去,但是并不能保证它们能到达目的地。

- 1) TCP是面向连接的，UDP是面向无连接的
- 2) UDP程序结构较简单
- 3) TCP是面向字节流的，UDP是基于数据报的
- 4) TCP保证数据正确性，UDP可能丢包
- 5) TCP保证数据顺序到达，UDP不保证

2、TCP、UDP的优缺点

TCP优点：可靠稳定

TCP的可靠体现在TCP在传输数据之前，会有三次握手来建立连接，而且在数据传递时，有确认、窗口、重传、拥塞控制机制，在数据传完之后，还会断开连接用来节约系统资源。

TCP缺点：慢，效率低，占用系统资源高，易被攻击

在传递数据之前要先建立连接，这会消耗时间，而且在数据传递时，确认机制、重传机制、拥塞机制等都会消耗大量时间，而且要在每台设备上维护所有的传输连接。然而，每个连接都会占用系统的CPU，内存等硬件资源。因为TCP有确认机制、三次握手机制，这些也导致TCP容易被利用，实现DOS、DDOS、CC等攻击。

UDP优点：快，比TCP稍安全

UDP没有TCP拥有的各种机制，是一种无状态的传输协议，所以传输数据非常快，没有TCP的这些机制，被攻击利用的机会就少一些，但是也无法避免被攻击。

UDP缺点：不可靠，不稳定

因为没有TCP的这些机制，UDP在传输数据时，如果网络质量不好，就会很容易丢包，造成数据的缺失。

3、TCP UDP适用场景

TCP：传输一些对信号完整性，信号质量有要求的信息。

UDP：对网络通讯质量要求不高时，要求网络通讯速度要快的场景。

4、TCP为什么是可靠连接？

因为tcp传输的数据满足3大条件，不丢失，不重复，按顺序到达。

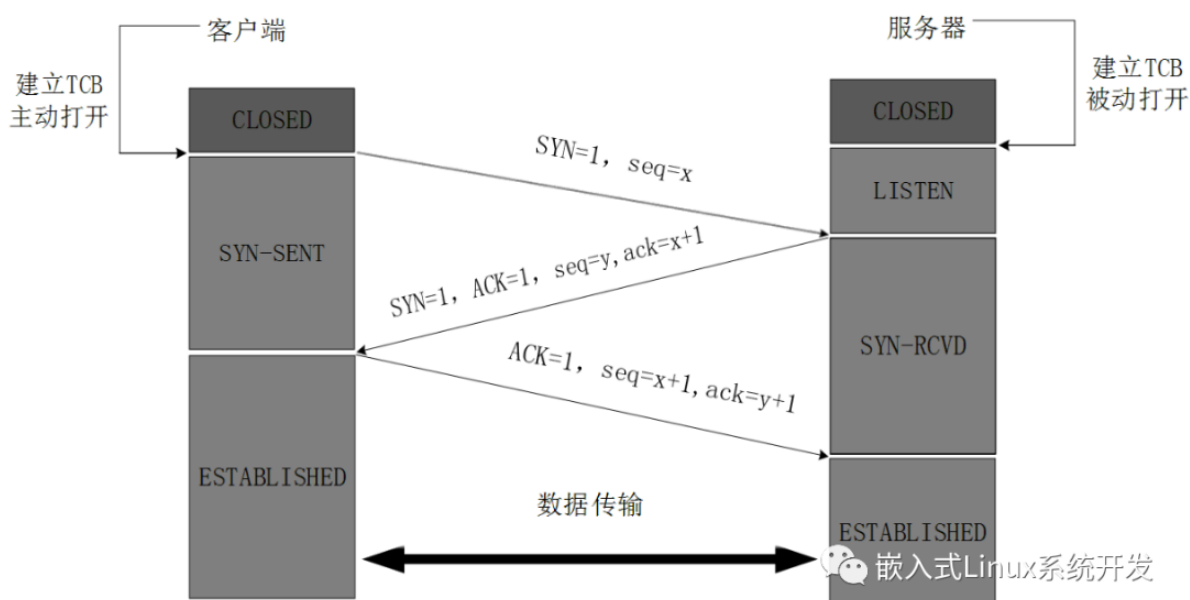
5、OSI典型网络模型，简单说说有哪些

ISO/OSI 模型			TCP/IP 协议				TCP/IP 模型
应用层	文件传输 协议 (FTP)	远程登录 协议 (Telnet)	电子邮件 协议 (SMTP)	网络文件服 务协议 (NFS)	网络管理 协议 (SNMP)	应用层	
表示层							
会话层							
传输层	TCP UDP					传输层	
网络层	IP	ICMP	ARP RARP			网际层	
数据链路层	Ethernet IEEE 802.3	FDDI	Token-Ring/ IEEE 802.5	ARCnet	PPP/SLIP	网络接口层	
物理层						硬件层	

TCP/IP 模型与 OSI 模型的对比

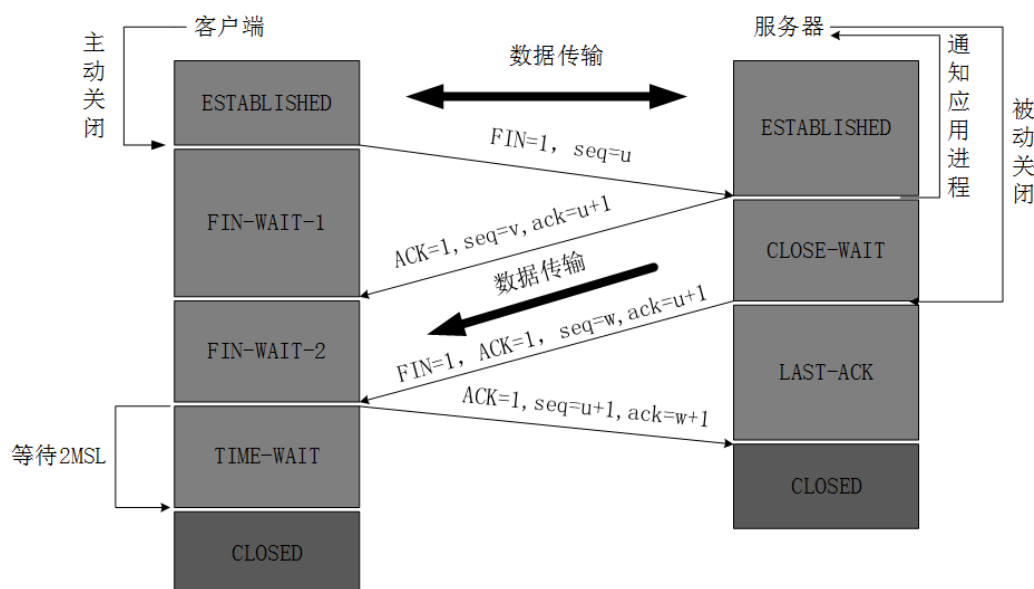
6、三次握手、四次挥手

三次握手：



- 1、TCP服务器进程先创建传输控制块TCB，时刻准备接受客户进程的连接请求，此时服务器就进入了LISTEN（监听）状态；
- 2、TCP客户进程也是先创建传输控制块TCB，然后向服务器发出连接请求报文，这是报文首部中的同部位 $SYN=1$ ，同时选择一个初始序列号 $seq=x$ ，此时，TCP客户端进程进入了SYN-SENT（同步已发送状态）状态。TCP规定，SYN报文段（ $SYN=1$ 的报文段）不能携带数据，但需要消耗掉一个序号。
- 3、TCP服务器收到请求报文后，如果同意连接，则发出确认报文。确认报文中应该 $ACK=1$ ， $SYN=1$ ，确认号是 $ack=x+1$ ，同时也要为自己初始化一个序列号 $seq=y$ ，此时，TCP服务器进程进入了SYN-RCVD（同步收到）状态。这个报文也不能携带数据，但是同样要消耗一个序号。
- 4、TCP客户进程收到确认后，还要向服务器给出确认。确认报文的 $ACK=1$ ， $ack=y+1$ ，自己的序列号 $seq=x+1$ ，此时，TCP连接建立，客户端进入ESTABLISHED（已建立连接）状态。TCP规定，ACK报文段可以携带数据，但是如果不携带数据则不消耗序号。
- 5、当服务器收到客户端的确认后也进入ESTABLISHED状态，此后双方就可以开始通信了。

四次挥手：



嵌入式Linux系统开发

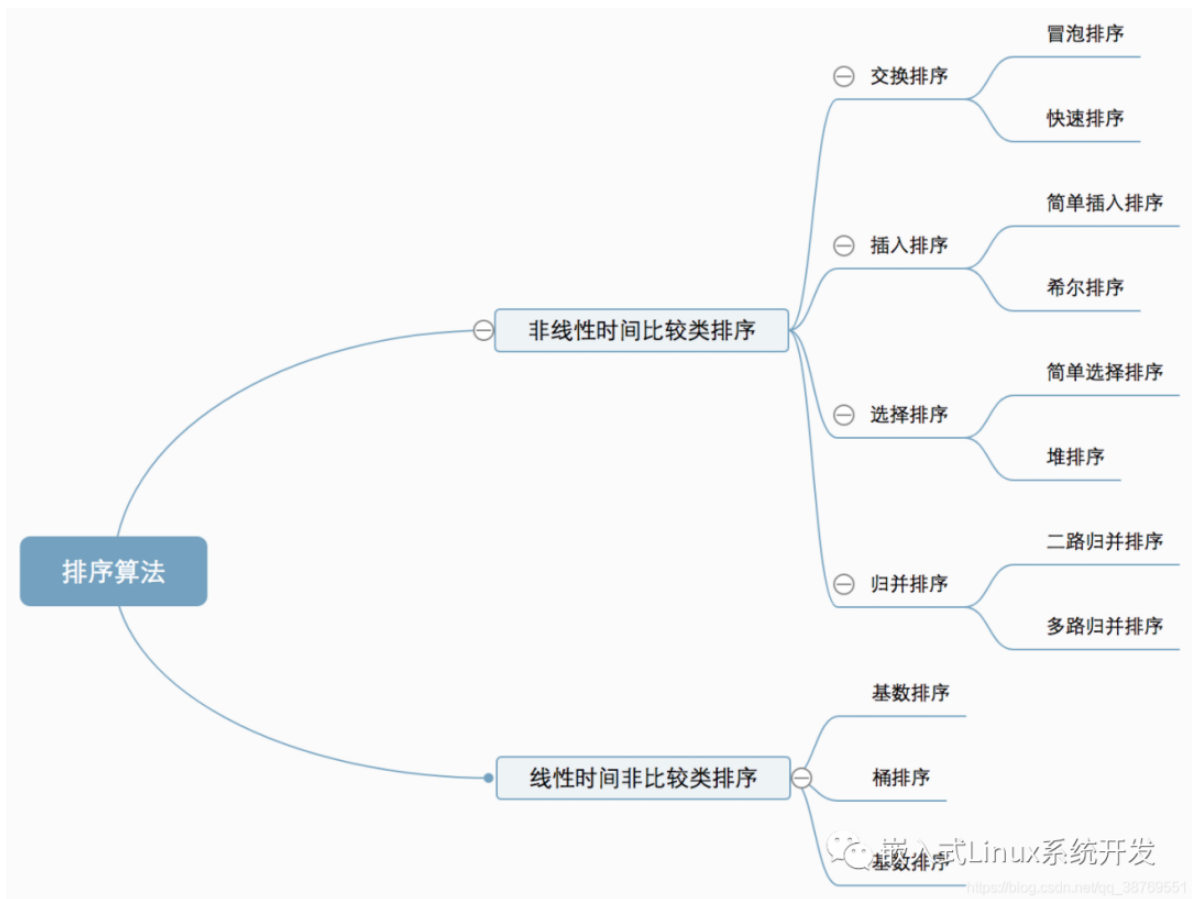
- 1、客户端进程发出连接释放报文，并且停止发送数据。释放数据报文首部，`FIN=1`，其序列号为`seq=u`（等于前面已经传送过来的数据的最后一个字节的序号加1），此时，客户端进入`FIN-WAIT-1`（终止等待1）状态。TCP规定，`FIN`报文段即使不携带数据，也要消耗一个序号。
- 2、服务器收到连接释放报文，发出确认报文，`ACK=1`，`ack=u+1`，并且带上自己的序列号`seq=v`，此时，服务端就进入了`CLOSE-WAIT`（关闭等待）状态。TCP服务器通知高层的应用进程，客户端向服务器的方向就释放了，这时候处于半关闭状态，即客户端已经没有数据要发送了，但是服务器若发送数据，客户端依然要接受。这个状态还要持续一段时间，也就是整个`CLOSE-WAIT`状态持续的时间。
- 3、客户端收到服务器的确认请求后，此时，客户端就进入`FIN-WAIT-2`（终止等待2）状态，等待服务器发送连接释放报文（在这之前还需要接受服务器发送的最后的的数据）。
- 4、服务器将最后的数据发送完毕后，就向客户端发送连接释放报文，`FIN=1`，`ack=u+1`，由于在半关闭状态，服务器很可能又发送了一些数据，假定此时的序列号为`seq=w`，此时，服务器就进入了`LAST-ACK`（最后确认）状态，等待客户端的确认。
- 5、客户端收到服务器的连接释放报文后，必须发出确认，`ACK=1`，`ack=w+1`，而自己的序列号是`seq=u+1`，此时，客户端就进入了`TIME-WAIT`（时间等待）状态。注意此时TCP连接还没有释放，必须经过 $2 * MSL$ （最长报文段寿命）的时间后，当客户端撤销相应的TCB后，才进入`CLOSED`状态。
- 6、服务器只要收到了客户端发出的确认，立即进入`CLOSED`状态。同样，撤销TCB后，就结束了这次的TCP连接。可以看到，服务器结束TCP连接的时间要比客户端早一些。

第四章、常见算法

十种常见排序算法可以分为两大类：

非线性时间比较类排序：通过比较来决定元素间的相对次序，由于其时间复杂度不能突破 $O(n\log n)$ ，因此称为非线性时间比较类排序。

线性时间非比较类排序：不通过比较来决定元素间的相对次序，它可以突破基于比较排序的时间下界，以线性时间运行，因此称为线性时间非比较类排序。



算法优劣评价术语

稳定性:

稳定: 如果 a 原本在 b 前面, 而 $a = b$, 排序之后 a 仍然在 b 的前面;

不稳定: 如果 a 原本在 b 的前面, 而 $a = b$, 排序之后 a 可能会出现在 b 的后面;

排序方式:

内排序: 所有排序操作都在内存中完成, 占用常数内存, 不占用额外内存。

外排序: 由于数据太大, 因此把数据放在磁盘中, 而排序通过磁盘和内存的数据传输才能进行, 占用额外内存。

复杂度:

时间复杂度: 一个算法执行所耗费的时间。

空间复杂度: 运行完一个程序所需内存的大小。

排序方法	最好时间	平均时间	最坏时间	辅助空间	稳定性
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
二分插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希 尔		$O(n^{1.25})$		$O(1)$	不稳定
冒 泡	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快 速	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$	$O(\lg n)$	不稳定
直接选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$		不稳定
归 并	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	$O(n)$	稳定
基 数	$O(d(rd+n))$	$O(d(rd+n))$	$O(d(rd+n))$	$O(rd+n)$	稳定

排序算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	排序方式	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	内排序	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	内排序	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	内排序	稳定
希尔排序	$O(n \log n)$	$O(n(\log_2^2 n))$	$O(n(\log_2^2 n))$	$O(1)$	内排序	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	外排序	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	内排序	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	内排序	不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	外排序	稳定
桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	外排序	稳定
基数排序	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	内排序	稳定

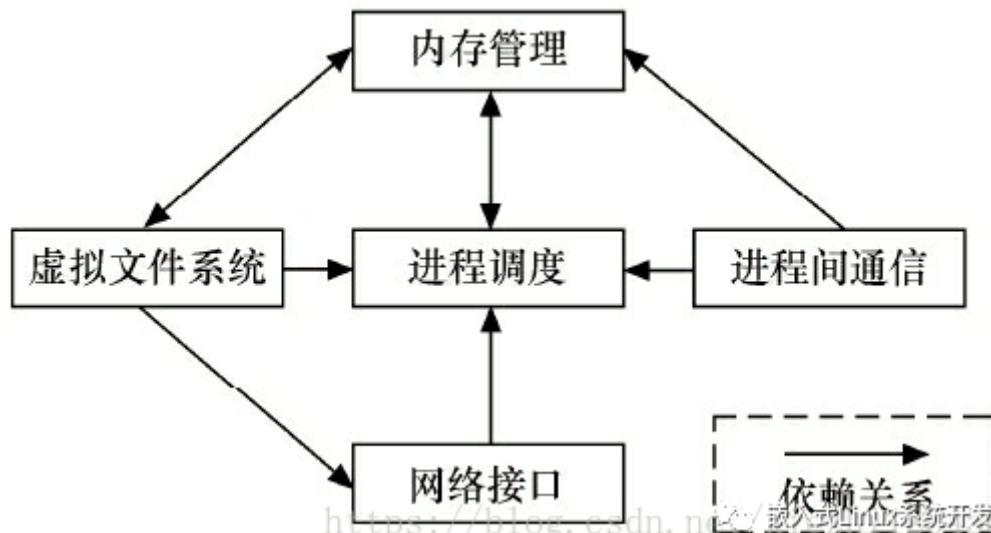
至于各种算法的原理以及代码实现，由于太多并且比较复杂，不在本文列出。但推荐两本入门的书：《啊哈！算法》、《大话数据结构》。电子版我会发在交流群里。

排序算法很多，嵌入式要求的不会太多，你会冒泡排序、快速排序、插入排序就可以解决很多问题。难的比如动态规划问题，图的路径问题，嵌入式考的比较少，纯软才会考这些。（大公司和独角兽公司考的会相对难一些）

第五章、Linux操作系统题目

1、Linux内核的组成部分

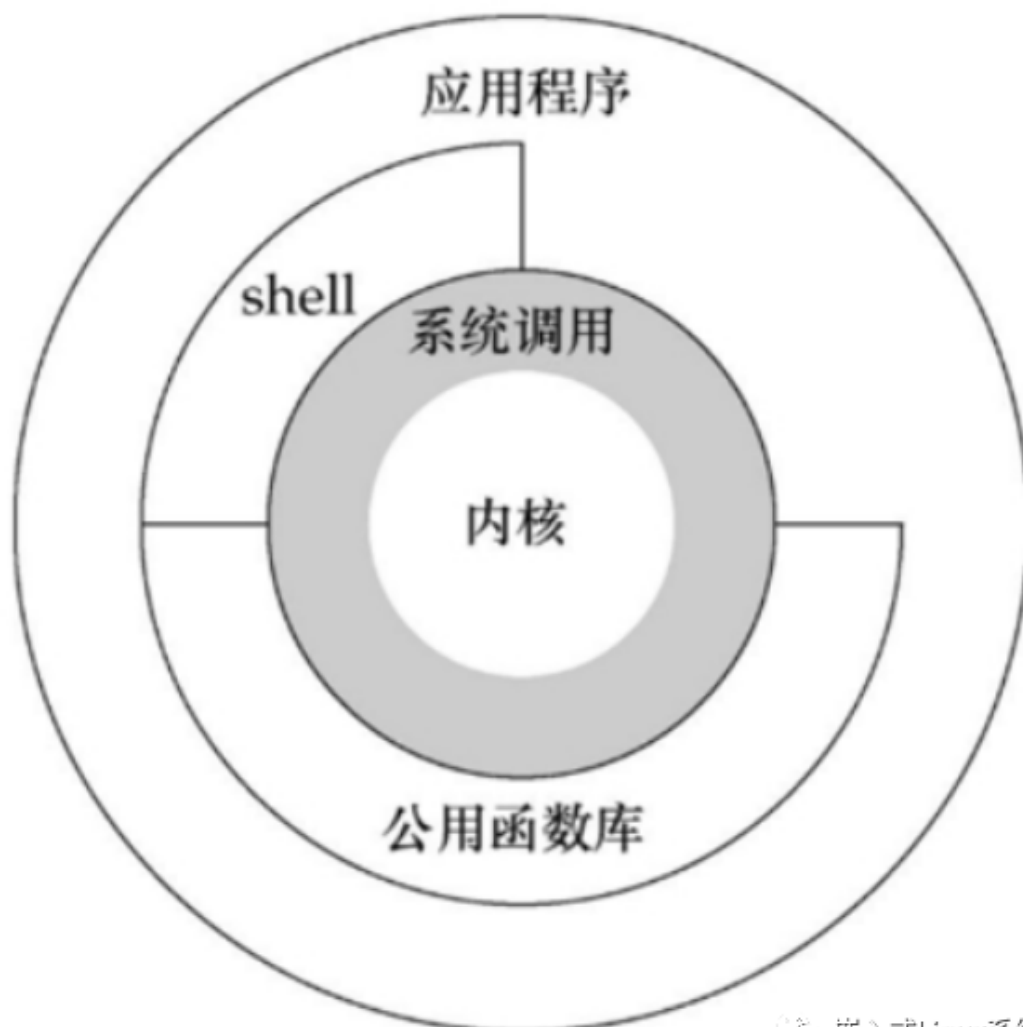
Linux内核主要由五个子系统组成：进程调度，内存管理，虚拟文件系统，网络接口，进程间通信。

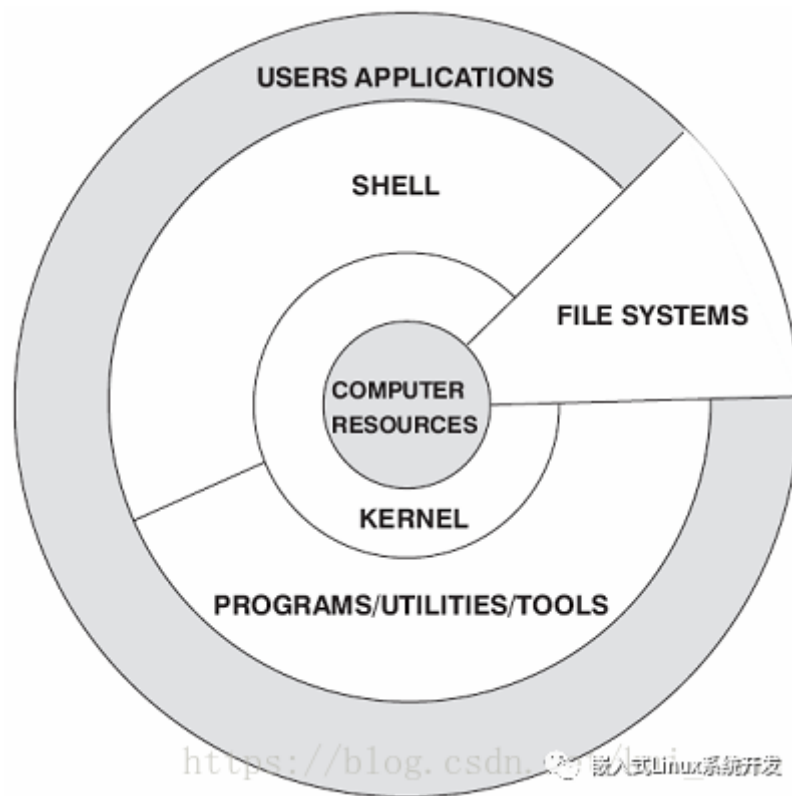


2、Linux系统的组成部分

Linux系统一般有4个主要部分：

内核、shell、文件系统和应用程序。





3、用户空间与内核通信方式有哪些？

- 1)系统调用。用户空间进程通过系统调用进入内核空间，访问指定的内核空间数据；
- 2)驱动程序。用户空间进程可以使用封装后的系统调用接口访问驱动设备节点，以和运行在内核空间的驱动程序通信；
- 3)共享内存mmap。在代码中调用接口，实现内核空间与用户空间的地址映射，在实时性要求很高的项目中为首选，省去拷贝数据的时间等资源，但缺点是不好控制；
- 4)copy_to_user()、copy_from_user()，是在驱动程序中调用接口，实现用户空间与内核空间的数据拷贝操作，应用于实时性要求不高的项目中。

以及：

```
1 | procfs(/proc)
2 | sysctl (/proc/sys)
3 | sysfs(/sys)
4 | netlink 套接口
```

4、系统调用与普通函数调用的区别

系统调用：

- 1.使用INT和IRET指令，内核和应用程序使用的是不同的堆栈，因此存在堆栈的切换，从用户态切换到内核态，从而可以使用特权指令操控设备
- 2.依赖于内核，不保证移植性
- 3.在用户空间和内核上下文环境间切换，开销较大
- 4.是操作系统的一个入口点

普通函数调用：

- 1.使用CALL和RET指令，调用时没有堆栈切换

2.平台移植性好

3.属于过程调用，调用开销较小

4.一个普通功能函数的调用

5、内核态，用户态的区别

内核态，操作系统在内核态运行——运行操作系统程序

用户态，应用程序只能在用户态运行——运行用户程序

当一个进程在执行用户自己的代码时处于用户运行态（用户态），此时特权级最低，为3级，是普通的用户进程运行的特权级，大部分用户直接面对的程序都是运行在用户态。Ring3状态不能访问Ring0的地址空间，包括代码和数据；当一个进程因为系统调用陷入内核代码中执行时处于内核运行态（内核态），此时特权级最高，为0级。执行的内核代码会使用当前进程的内核栈，每个进程都有自己的内核栈。

6、bootloader、内核、根文件的关系

启动顺序：bootloader->linux kernel->rootfile->app

Bootloader全名为启动引导程序，是第一段代码，它主要用来初始化处理器及外设，然后调用Linux内核。Linux内核在完成系统的初始化之后需要挂载某个文件系统作为根文件系统（RootFilesystem），然后加载必要的内核模块，启动应用程序。（一个嵌入式Linux系统从软件角度看可以分为四个部分：引导加载程序（Bootloader），Linux内核，文件系统，应用程序。）

7、Bootloader启动的两个阶段：

Stage1:汇编语言

- 1) 基本的硬件初始化（关闭看门狗和中断，MMU（带操作系统），CACHE。配置系统工作时钟）
- 2) 为加载stage2准备RAM空间
- 3) 拷贝内核映像和文件系统映像到RAM中
- 4) 设置堆栈指针sp
- 5) 跳到stage2的入口点

Stage2:c语言

- 1) 初始化本阶段要使用到的硬件设备（led uart等）
- 2) 检测系统的内存映射
- 3) 加载内核映像和文件系统映像
- 4) 设置内核的启动参数

嵌入式系统中广泛采用的非易失性存储器通常是Flash，而Bootloader就位于该存储器的最前端，所以系统上电或复位后执行的第一段程序便是Bootloader。

8、linux下检查内存状态的命令

```
1 1) 查看进程: top
2 2) 查看内存: free
3 3) cat /proc/meminfo
4 4) vmstat
```

假如一个公司服务器有很多用户，你使用top命令，可以看到哪个同事在使用什么命令，做什么事情，占用了多少CPU。

9、一个程序从开始运行到结束的完整过程（四个过程）

预处理（Pre-Processing）、编译（Compiling）、汇编（Assembling）、链接（Linking）

10、什么是堆，栈，内存泄漏和内存溢出？

栈由系统操作，程序员不可以操作。

所以内存泄漏是指堆内存的泄漏。堆内存是指程序从堆中分配的，大小任意的（内存块的大小可以在程序运行期决定），使用完后必须显式释放的内存。应用程序一般使用malloc，new等函数从堆中分配到一块内存，使用完后，程序必须负责相应的调用free或delete释放该内存块，否则，这块内存就不能被再次使用。

内存溢出：你要求分配的内存超出了系统能给你的，系统不能满足需求，于是产生溢出。

内存越界：向系统申请了一块内存，而在使用内存时，超出了申请的范围（常见的有使用特定大小数组时发生内存越界）

内存溢出问题是C语言或者C++语言所固有的缺陷，它们既不检查数组边界，又不检查类型可靠性（type-safety）。众所周知，用C/C++语言开发的程序由于目标代码非常接近机器内核，因而能够直接访问内存和寄存器，这种特性大大提升了C/C++语言代码的性能。只要合理编码，C/C++应用程序在执行效率上必然优于其它高级语言。然而，C/C++语言导致内存溢出问题的可能性也要大许多。

11、死锁的原因、条件

产生死锁的原因主要是：

- （1）因为系统资源不足。
- （2）进程运行推进的顺序不合适。
- （3）资源分配不当等。

如果系统资源充足，进程的资源请求都能够得到满足，死锁出现的可能性就很低，否则就会因争夺有限的资源而陷入死锁。其次，进程运行推进顺序与速度不同，也可能产生死锁

这四个条件是死锁的必要条件，只要系统发生死锁，这些条件必然成立，而只要上述条件之一不满足，就不会发生死锁。

- （1）互斥条件：一个资源每次只能被一个进程使用。
- （2）请求与保持条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放。
- （3）不剥夺条件：进程已获得的资源，在未使用完之前，不能强行剥夺。
- （4）循环等待条件：若干进程之间形成一种头尾相接的循环等待资源关系。

12、硬链接与软链接

链接操作实际上是给系统中已有的某个文件指定另外一个可用于访问它的名称。对于这个新的文件名，我们可以为之指定不同的访问权限，以控制对信息的共享和安全性的问题。如果链接指向目录，用户就可以利用该链接直接进入被链接的目录而不用打一大堆的路径名。而且，即使我们删除这个链接，也不会破坏原来的目录。

1>硬链接

硬链接只能引用同一文件系统中的文件。它引用的是文件在文件系统中的物理索引(也称为inode)。当您移动或删除原始文件时，硬链接不会被破坏，因为它所引用的是文件的物理数据而不是文件在文件结构中的位置。硬链接的文件不需要用户有访问原始文件的权限，也不会显示原始文件的位置，这样有助于文件的安全。如果您删除的文件有相应的硬链接，那么这个文件依然会保留，直到所有对它的引用都被删除。

2>软链接（符号链接）

软连接，其实就是新建立一个文件，这个文件就是专门用来指向别的文件的（那就和windows下的快捷方式的那个文件有很接近的意味）。软连接产生的是一个新的文件，但这个文件的作用就是专门指向某个文件的，删了这个软连接文件，那就等于不需要这个连接，和原来的存在的实体原文件没有任何关系，但删除原来的文件，则相应的软连接不可用。

13、计算机中，32bit与64bit有什么区别

64bit计算主要有两大优点：可以进行更大范围的整数运算；可以支持更大的内存。

64位操作系统下的虚拟内存空间大小：地址空间大小不是 2^{32} ，也不是 2^{64} ，而一般是 2^{48} 。因为并不需要 2^{64} 那么大的寻址空间，过大的空间只会造成资源的浪费。所以64位Linux一般使用48位表示虚拟空间地址，40位标识物理地址。

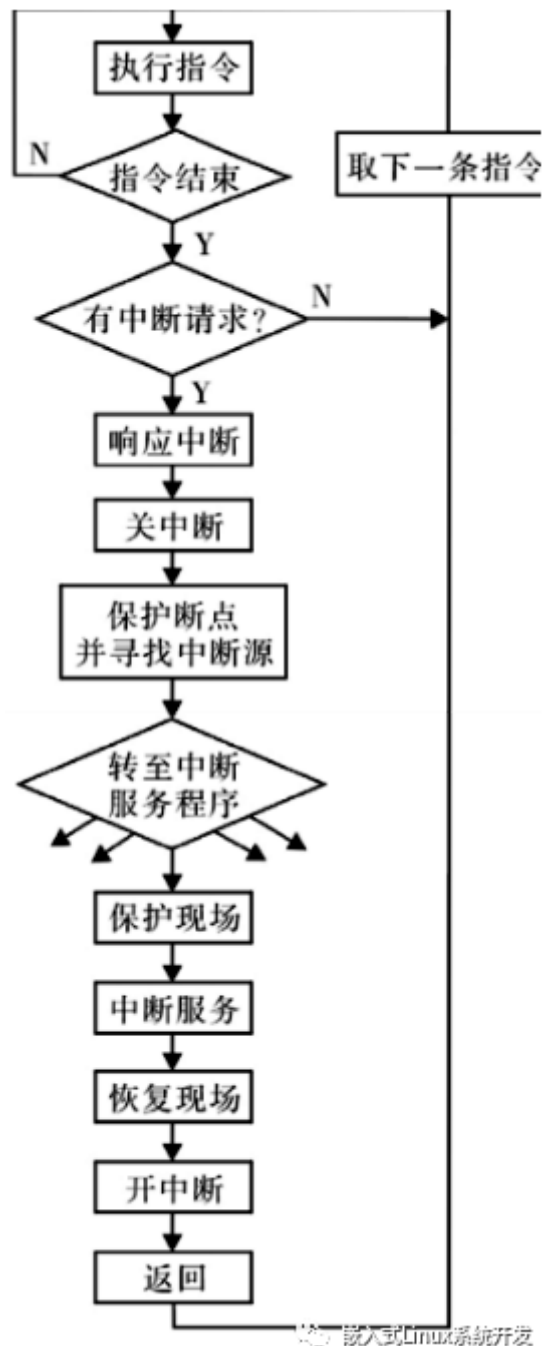
14、中断和异常的区别

内中断：同步中断（异常）是由cpu内部的电信号产生的中断，其特点为当前执行的指令结束后才转而产生中断，由于有cpu主动产生，其执行点必然是可控的。

外中断：异步中断是由cpu的外设产生的电信号引起的中断，其发生的时间点不可预期。

15、中断怎么发生，中断处理流程

请求中断→响应中断→关闭中断→保留断点→中断源识别→保护现场→中断服务子程序→恢复现场→中断返回。



16、Linux 操作系统挂起、休眠、关机相关命令

关机命令有 `halt` , `init 0` , `poweroff` , `shutdown -h 时间` , 其中 `shutdown` 是最安全的

重启命令有 `reboot` , `init 6` , `shutdown -r 时间`

在linux命令中 `reboot` 是重新启动, `shutdown -r now` 是立即停止然后重新启动

具体可用参数可以百度。

17、说一个linux下编译优化选项：

加：`-o`

18、在有数据cache情况下，DMA数据链路为：

外设-DMA-DDR-cache-CPU

19、linux命令

- 1、改变文件属性的命令：chmod (chmod 777 /etc/squid 运行命令后，squid文件夹（目录）的权限就被修改为777（可读可写可执行））
- 2、查找文件中匹配字符串的命令：grep
- 3、查找当前目录：pwd
- 4、删除目录：rm -rf 目录名
- 5、删除文件：rm 文件名
- 6、创建目录（文件夹）：mkdir
- 7、创建文件：touch
- 8、vi和vim 文件名也可以创建
- 9、解压：tar -xzvf 压缩包
打包：tar -cvzf 目录（文件夹）
- 10、查看进程对应的端口号

```
1 1、先查看进程pid
2 ps -ef | grep 进程名
3
4 2、通过pid查看占用端口
5 netstat -nap | grep 进程pid
```

20、硬实时系统和软实时系统

软实时系统：

Windows、Linux系统通常为软实时，当然有补丁可以将内核做成硬实时的系统，不过商用没有这么做的。

硬实时系统：

对时间要求很高，限定时间内不管做没做完必须返回。

VxWorks，uCOS，FreeRTOS，WinCE，RT-thread等实时系统；

21、MMU基础

现代操作系统普遍采用虚拟内存管理（Virtual Memory Management）机制，这需要MMU（Memory Management Unit，内存管理单元）的支持。有些嵌入式处理器没有MMU，则不能运行依赖于虚拟内存管理的操作系统。

也就是说：操作系统可以分成两类，用MMU的、不用MMU的。

用MMU的是：Windows、MacOS、Linux、Android；不用MMU的是：FreeRTOS、VxWorks、UCOS.....

与此相对应的：CPU也可以分成两类，带MMU的、不带MMU的。

带MMU的是：Cortex-A系列、ARM9、ARM11系列；

不带MMU的是：Cortex-M系列.....（STM32是M系列，没有MMU，不能运行Linux，只能运行一些UCOS、FreeRTOS等等）。

MMU就是负责虚拟地址（virtual address）转化成物理地址（physical address），转换过程比较复杂，可以自行百度。

第六章、单片机常见面试题

1、ROM与RAM

这一点我另一篇文章讲解过，这里放链接：

[ROM与RAM的区别](#)

2、IO口工作方式（学过STM32的人应该很熟悉）

上拉输入、下拉输入、推挽输出、开漏输出。

3、请说明总线接口USRT、I2C、USB的异同点

（串/并、速度、全/半双工、总线拓扑等）

总线接口	串/并	同步/异步	速率	工作方式	用线	总线拓扑结构	信距离
UART	串	异步	慢 波特率设置	全双工	2线 Rx、Tx	RS485支持总线式、 星形、树形	远 最远1200m
I2C	串	同步	慢	半双工	2线 SDA、SCL	总线型（特殊的树形）	近
SPI	串	同步	快	全双工	3线或4线 SCLK、SIMO、 SOMI、SS(片选)	环形	远
USB	串	同步	快	半双工	4线 Vbus（5V）、GND、 D+、D-（3.3V）	 嵌入式Linux系统开发 https://blog.csdn.net/qq_38769551	

4、IIC协议时序图

必须会画出来，我面试被问到过，让我画，我画了个大概。

IIC协议有两根线，一根SCL时钟线，一根SDA数据线，如图可以看到开始信号和结束信号的电平状态。开始后，因为IIC总线可以挂在很多设备（不超过8个），所以先发送一个设备地址，选中这个设备，设备地址最后一位代表了是写还是读。选中设备后，再发送寄存器地址，代表选中某个寄存器，再开始传输数据。

八位设备地址=7位从机地址+读/写地址，

再给地址添加一个方向位用来表示接下来数据传输的方向，

0表示主设备向从设备(write)写数据，

1表示主设备向从设备(read)读数据

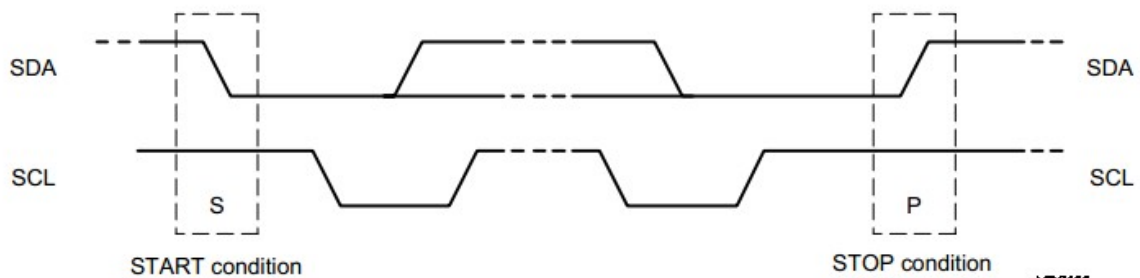


图 5 起始和停止条件

嵌入式Linux系统开发

开始信号：SCL 为高电平时，SDA 由高电平向低电平跳变，开始传送数据。

结束信号：SCL 为高电平时，SDA 由低电平向高电平跳变，结束传送数据。

应答信号：接收数据的 IC 在接收到 8bit 数据后，向发送数据的 IC 发出特定的低电平脉冲，表示已收到数据。CPU 向受控单元发出一个信号后，等待受控单元发出一个应答信号，CPU 接收到应答信号后，根据实际情况作出是否继续传递信号的判断。若未收到应答信号，由判断为受控单元出现故障。

IIC信号在数据传输过程中，当SCL=1高电平时，数据线SDA必须保持稳定状态，不允许有电平跳变，只有在时钟线上的信号为低电平期间，数据线上的高电平或低电平状态才允许变化。SCL=1时 数据线SDA的任何电平变换会看做是总线的起始信号或者停止信号。

IIC我也有一篇文章有讲解，请看链接：

[IIC总线最多可以挂多少个设备](#)

5、单片机的SP指针始终指向

栈顶

6、IIC总线在传送数据过程中共有三种类型信号：

它们分别是：开始信号、结束信号和应答信号。

7、FIQ中断向量入口地址：

FIQ和IRQ是两种不同类型的中断，ARM为了支持这两种不同的中断，提供了对应的叫做FIQ和IRQ处理器模式（ARM有7种处理模式）。

FIQ的中断向量地址在0x0000001C，而IRQ的在0x00000018。

8、SPI四种模式，简述其中一种模式，画出时序图

在芯片资料上极性和相位一般表示为CPOL（Clock POLarity）和CPHA(Clock PHase), 极性和相位组合成4种工作模式。

	CPOL	CPHA
MODE0	0	0
MODE1	0	1
MODE2	1	0
MODE3	1	1

嵌入式Linux系统开发

spi四种模式SPI的相位(CPHA)和极性(CPOL)分别可以为0或1，对应的4种组合构成了SPI的4种模式(mode)

Mode 0 CPOL=0, CPHA=0

Mode 1 CPOL=0, CPHA=1

Mode 2 CPOL=1, CPHA=0

Mode 3 CPOL=1, CPHA=1

时钟极性CPOL: 即SPI空闲时，时钟信号SCLK的电平（1:空闲时高电平; 0:空闲时低电平）时钟相位

CPHA: 即SPI在SCLK第几个边沿开始采样（0:第一个边沿开始; 1:第二个边沿开始）

sd卡的spi常用的是mode 0 和mode 3，这两种模式的相同的地方是都在时钟上升沿采样传输数据，区别这两种方式的简单方法就是看空闲时，时钟的电平状态，低电平为mode 0，高电平为mode 3。

具体的通信过程请自行百度，2021年秋招大疆笔试题考了这道题。

第七章、杂项面试题

1、讲一讲冯诺依曼和哈佛体系的区别

哈佛结构是一种将程序指令存储和数据存储分开的存储器结构。目前使用哈佛结构的中央处理器和微控制器有很多，ARM9、ARM10和ARM11，51单片机属于哈佛结构。

冯·诺伊曼结构也称普林斯顿结构，是一种将程序指令存储器和数据存储器合并在一起的存储器结构。

2、面向对象编程的三大特性

以及重载的意思。重载，是指允许存在多个同名函数，而这些函数的参数表不同（或许参数个数不同，或许参数类型不同，或许两者都不同）。

3、http默认端口号

80

4、linux中mysql数据库默认的端口是

3306

5、编程习惯小知识点

C语言编程中，单片机平台，一般有.c和.h文件，如果一个人在.h文件中定义了一个变量，会有什么后果。（讨论编程习惯的问题，我一般是只在.h文件中声明函数，不会做变量定义；另外，编程中每一个模块都会有对应的.c和.h文件，最终的总程序自己定义一个comm.c和comm.h去调用各个模块，这样的习惯我觉得还行）

if语句中如果是或运算（|），第一个条件满足时，第二个条件还会判断吗。或运算的话，当然不会，因为0|1=1，中断了。

第八章、各大公司面试题

1、阿里面试

阿里面试总结，岗位：阿里云网络产品研发实习生。岗位要求：BOSS直聘截图。



职位详情

嵌入式技术

网络协议

数据结构

多线程

Linux

软件工程师

后端开发

阿里云网络产品作为阿里云的核心技术团队，基于自研洛神系统，提供了虚拟化网络服务，例如虚拟专有网络（VPC）、软件定义网络（SDN）控制器和服务器负载均衡（SLB）等多项网络产品及技术，在历次双11中为上层应用保驾护航，为用户提供如丝般顺滑的网络体验。

岗位要求：

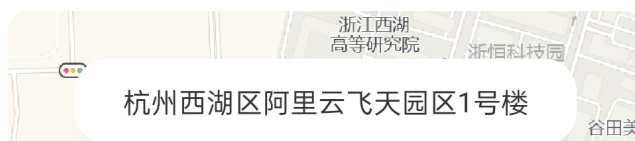
- 掌握C, python, go语言等开发语言，熟悉linux
- 成绩好，在校发表论文，研究项目，获奖
- 高速数据面开发经验，有网络路由交换基础
- 数据结构、算法功底扎实
- 移动端网络开发能力
- 嵌入式开发经验

满足以上几项即可



阿里云计算有限公司

已上市 · 10000人以上 · 互联网



杭州西湖区阿里云飞天园区1号楼

立即沟通

嵌入式Linux系统开发

大公司面试注重基础，小公司面试注重项目和实战。

上来先几分钟自我介绍，然后问你做过的一个项目的流程，你做了什么？技术面会问你他在简历中没看懂的地方，会给你讲解面试不上来的题目。问完项目以后问基础。

1、Linux系统方面

1) 进程和线程的区别（不能太书面化）（需要从内存角度，或者其他角度描述）

需要用自己理解的方式去回答这个问题，如果你回答了xxx是基本单位，这样子是不行的。技术面试会追问你，需要你用自己的话去理解进程和线程。线程和进程是否是独立的？线程和进程的之间内存空间是否是共享的？进程和进程之间是否是共享的？

2) 线程之间是如何调度的

比如说有两个线程：线程A和线程B，线程B要先于线程A运行，或者是线程B运行了多次之后在启动线程A，如何进行调度？

3) 非实时操作系统和实时操作系统的区别和选择，以及应用场景。

2、C语言方面

1) static的作用，函数中变量使用static的作用，它的生命周期会如何。

2) extern的作用

3) c语言中有哪些宏定义？

4) #define和#define这两个的区别

5) 代码编译的四个过程，预处理的阶段做了哪些事情（深入问条件编译里面做了哪些事情）四个过程都生成哪些文件？

3、数据结构

1) 何为数据结构？

2) 问了队列、栈、链表的概念和特征，队列和栈在项目中的应用场景。什么情况下用栈，什么情况下用队列和链表。

4、网络的概念

总结：

1、简历中有的项目，一定要会，不会的东西不写，写上去的东西保证自己每个细节都懂，也就是确实是自己做的项目，不是捏造的项目。

2、大厂面试注重基础，因为大公司会培养你，只要你基础扎实就行，所以基础很重要，整个面试一个多小时，大部分时间在linux系统、编程能力、数据结构、计算机网络基本概念上。

3、大厂面试喜欢问你对于某个常见的东西自己的理解，因为书上的名词大家都会说，但是你自己理解的肯定是不一样的。比如富士康的一个面试官问过：你认为OSI七层网络模型和生活中给的什么很像？答案：快递业务。

2、CVTE面试

岗位：嵌入式软件工程师。面试时间：20分钟。岗位职责：BOOS直聘截图

单片机开发

嵌入式技术

嵌入式系统软件工程师

C语言

嵌入式软件开发

【岗位职责】

- 1、负责小家电、智能门锁产品软件功能需求分析、设计以及开发工作
- 2、参与小家电、智能门锁产品软件架构设计方案选型
- 3、按照公司的相关制度要求撰写软件设计文档
- 4、对接客户并处理客户反馈的需求以及问题

【任职要求】

- 1、本科以上学历，3年以上小家电、智能门锁经验者优先；
- 2、熟悉C语言，使用STM32、C51等单片机做过开发；
- 3、能够读懂硬件原理图，独立的根据原理图写出硬件使用说明书，编写硬件测试程序；
- 4、熟悉RS232、RS485、RS422等多种串口通讯；
- 5、熟悉各种通讯总线，理解I2C、SPI、USB、CAN等各种通讯原理，并有相关编程经验；
- 6、熟悉多种存储器原理，包括各种FLASH、RAM及总线访问原理；
- 7、具有良好的沟通能力、团队合作精神，工作积极主动、责任心强，有良好的逻辑思维能力，能独立分析问题；

立即沟通

嵌入式Linux系统开发

面试

1、简历中写了做过海思音视频项目，所以问了海思项目是怎么学的？（考察自学能力）。

2、问：学海思项目，是因为兴趣，还是为了毕设？

回答：是因为兴趣，因为自己毕设是51单片机。

问：如果一个Linux和51单片机进行通讯，让我怎么设计通讯协议，然后问我怎么进行检验。

回答：可以设计串口，加奇偶校验，并且检查数据包的总字节数。

问：如果字节总个数是对的，但是有些位错了，那应该怎么去检查？

（不该说自己的毕设是51单片机的，因为自己本身不了解51单片机，所以这一块答得一般）后来面试官知道我对51单片机没怎么学过，是春招后才准备现学现用的，就没有继续问下去了。

3、二叉树学过吗？

回答：学过，但是后面没怎么用，几乎忘光了，比较熟悉和常用的是链表。

4、如何判断一个链表有环？

回答：快慢指针

5、线程和进程的区别？

6、线程和进程间的通讯方式有哪些？

7、访问临界资源时应该怎么办？

8、线程和进程的API，知道哪些？

回答：我把API名字和API的参数是什么都说了，面试官就没继续往下问了

9、工作的意向地点

10、对于加班的看法（据说CVTE加班很猛，另外两个是多益和三七）（广州）

11、有没有转管理层的意向？

12、有什么问题想问的？

回答：如果有幸入职贵公司，请问是否会有一些培训制度？

总结：除了一开始的那个怎么设计单片机和Linux通信协议没答好，其他的问题答得还可以，HR小哥哥挺有耐心挺温柔的，也会在面试时一步步引导。

从面试过程可以看出，很多公司，面试官一般都是根据你的简历和你的回答，来决定下一个问题。所以千万不要自己给自己挖坑，简历写的东西，必须要会，自己回答的东西，自己必须要会。所以面试官比较随和的情况下，可以通过自己的回答，把面试官引导到全是自己熟悉的领域。

3、富士康面试

岗位：嵌入式软件工程师。面试时间：30分钟。岗位职责：官网

软件开发工程师(嵌入式)

CNSBG

深圳市

面议

27天前更新

工作描述:

- 1.负责小基站嵌入式系统开发；
- 2.负责 Smallcell GW 的 UE 接入和 S1 信令处理模块设计开发工作；
- 3.配合硬件工程师调试和解决硬件问题，提升系统硬件性能；
- 4.Linux 系统性能分析优化，提出优化措施并实施；
- 5.配合高层软件工程师完成整机功能和性能调测；
- 6.配合产线生产小基站软件升级及相关测试。

职位要求:

1. 必须具备基本的C语言编程能力
2. 大学英语四级
3. 表达沟通能力强

嵌入式Linux系统开发

薪资：试用期基本工资7200，转正按情况加（0-1000），算月工资8000，加班有加班费，算上加班费大概9~10K，13薪。

所应聘部门是基于高通平台，做通信模块的。

- 1、自我介绍
- 2、如何理解交叉编译？
- 3、OSI七层网络和TCP/IP网络模型区别？以及每一层名称。
- 4、你觉得网络分层协议和生活中什么比较像？（快递业务）
- 5、什么是上下文切换（我从中断上下文方面讲。然后他问为什么响应中断要保护现场，中断处理流程是什么）
- 6、你如何理解异步（我从文件IO中的异步非阻塞回答）
- 7、你是XXXX专业，是不是和嵌入式不太匹配，都是自学的吗？
- 8、数据结构学的怎么样？
- 9、计算机操作系统学的怎么样？
- 10、你熟悉哪些数据结构？
- 11、如何理解数据结构和算法的关系？
- 12、你是XXXX专业，是如何学习嵌入式的？（我都是做项目，项目中学习）
- 13、C语言和其他编程语言有什么关系，处于一个什么地位？（更底层）
- 14、你如何理解编程语言和日常说话语言的关系？

15、你如何理解指针？（也是一种变量而已，只是存储的内容是地址，所以可以叫指针变量）

16、什么是系统调用？

17、用户程序运行在usr space还是kernel space？

18、普通调用和系统调用的区别？

19、系统调用用什么函数（ioctl等）？

20、数据结构学的怎么样，学过红黑树吗？讲一讲。

回答：红黑树，Red-Black Tree 「RBT」是一个自平衡(不是绝对的平衡)的二叉查找树(BST)，树上的每个节点都遵循下面的规则：

1) 每个节点都有红色或黑色

2) 树的根始终是黑色的(黑土地孕育黑树根)

3) 没有两个相邻的红色节点（红色节点不能有红色父节点或红色子节点，并没有说不能出现连续黑色节点）

4) 从节点（包括根）到其任何后代NULL节点(叶子结点下方挂的两个空节点，并且认为他们是黑色的)的每条路径都具有相同数量的黑色节点）

补充：二叉查找树，Binary Search Tree 「BST」，要了解二叉查找树，我们首先看下二叉查找树有哪些特性：

1) 某节点的左子树节点值仅包含小于该节点值

2) 某节点的右子树节点值仅包含大于该节点值

3) 左右子树每个也必须是二叉查找树

21、讲一讲冯诺依曼和哈佛体系的区别

总结：他特别喜欢问你是如何理解某个东西的，这样的问法比单问你知识点牛多了，就是看你到底有没有对底层原理有理解，然后用通俗的话表达出来。

4、小米面试

岗位：嵌入式软件工程师（相机驱动岗）。面试时间：40分钟。

以下问题面试者全部答出，已offer，薪资请去小程序offershow上查。

面试官上来先来了一段他自己的自我介绍

1、请进行一个简单的自我介绍（2分钟）

2、C语言全局变量可否定义在头文件中？

回答：不能，并且这不是一个好的习惯。

3、全局变量和局部变量是否可以重名？

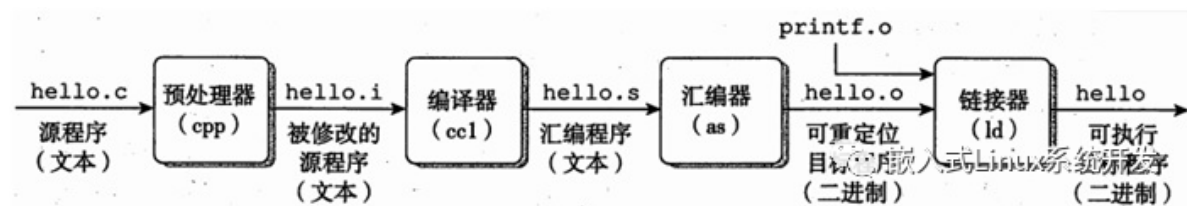
回答：可以重名。只是作用域不同，局部变量在局部生效。

4、extern C 的作用？

回答：extern C 的主要作用就是为了能够正确实现C++代码调用其他C语言代码，即在C++代码中嵌入式C语言代码。

5、从代码编译到可执行文件的流程？

回答：一个源程序到一个可执行程序的过程：预编译、编译、汇编、链接。



6、进程和线程的区别？

回答：进程是资源（CPU、内存等）分配的基本单位，线程是CPU调度和分配的基本单位（程序执行的最小单位）。同一时间，如果CPU是单核，只有一个进程在执行，所谓的并发执行，也是顺序执行，只不过由于切换速度太快，你以为这些进程在同步执行而已。多核CPU可以同一时间点有多个进程在执行。

7、手撕代码：写一个双向链表的插入。

8、问简历上面的项目。

9、IIC协议说一下（因为我简历里面写了IIC）

回答：I2C协议有两条信号线，SDA和SCL，分别是数据线和时钟线，同一总线可以挂载多个IIC设备，靠设备地址区分。起始信号和终止信号如下，更具体的请自行百度：

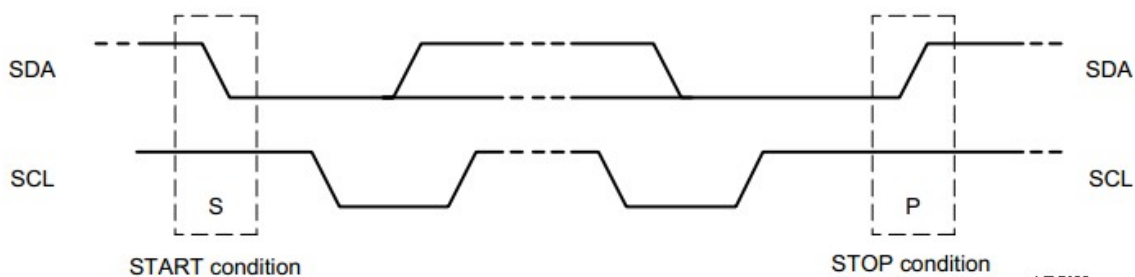


图 5 起始和停止条件

嵌入式Linux系统开发

10、C++有了解吗，用它做过什么项目没有。

回答：我主要用C语言，C++用的比较少。

11、设备树有了解吗？

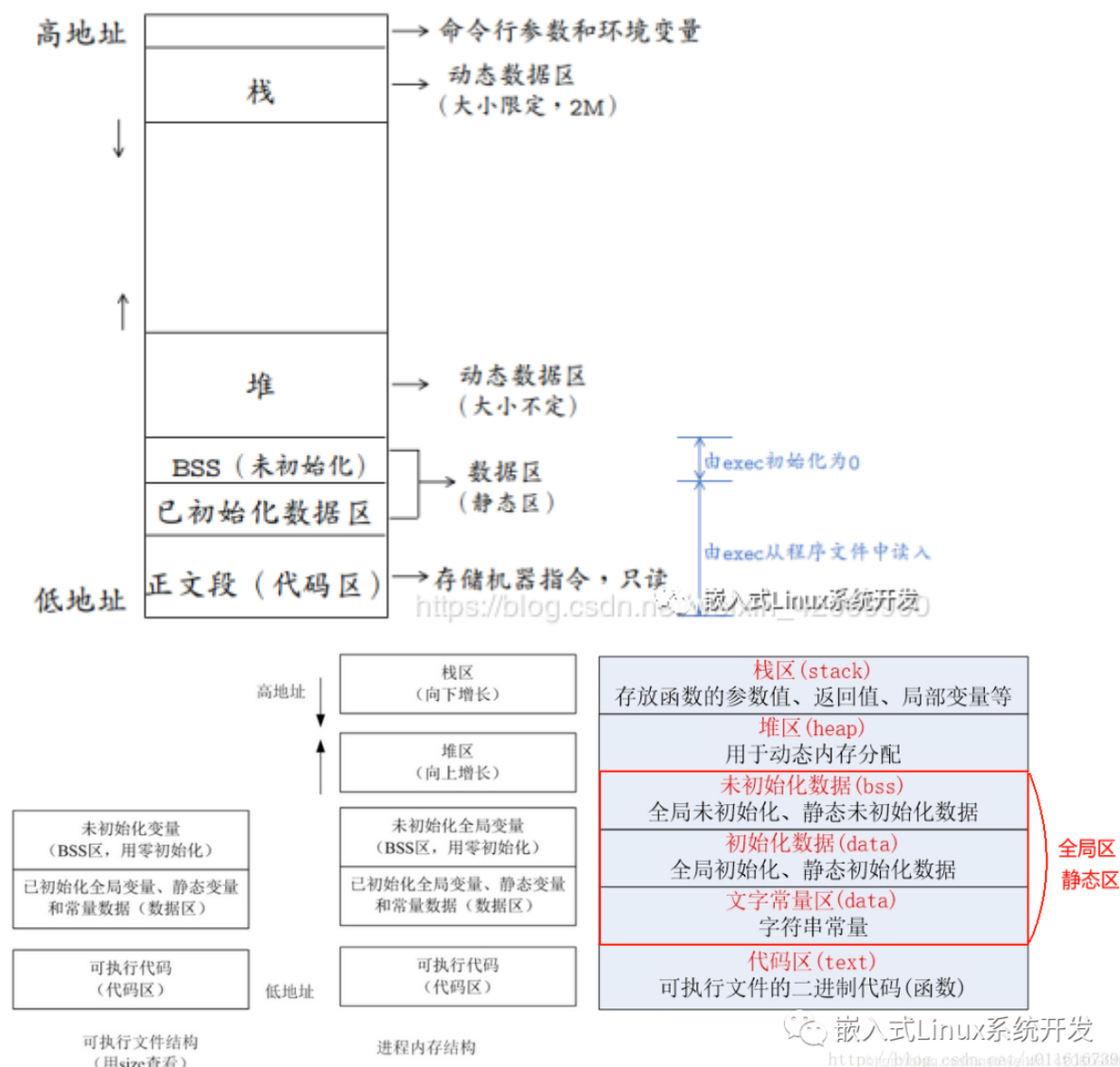
在此补充回答：Linus Torvalds在2011年3月17日的ARM Linux邮件列表宣称“this whole ARM thing is a fucking pain in the ass”，ARM Linux社区对此作出了回应，引入设备树。

ARM Device Tree起源于OpenFirmware (OF)，在过去的Linux中，arch/arm/plat-xxx和arch/arm/mach-xxx中充斥着大量的垃圾代码，相当多数的代码只是在描述板级细节，而这些板级细节对于内核来讲，不过是垃圾，如板上的platform设备、resource、i2c_board_info、spi_board_info以及各种硬件的platform_data。为了改变这种局面，Linux社区的大牛们参考了PowerPC等体系架构中使用的Flattened Device Tree (FDT)，也采用了Device Tree结构，许多硬件的细节可以直接透过它传递给Linux，而不再需要在kernel中进行大量的冗余编码。

Device Tree是一种描述硬件的数据结构，它起源于 OpenFirmware (OF)。在Linux 2.6中，ARM架构的板级硬件细节过多地被硬编码在arch/arm/plat-xxx和arch/arm/mach-xxx，采用Device Tree后，许多硬件的细节可以直接透过它传递给Linux，而不再需要在kernel中进行大量的冗余编码。

12、内存分为哪几个部分？

回答：内存分为四区：堆区、栈区、全局区、代码区。具体如下



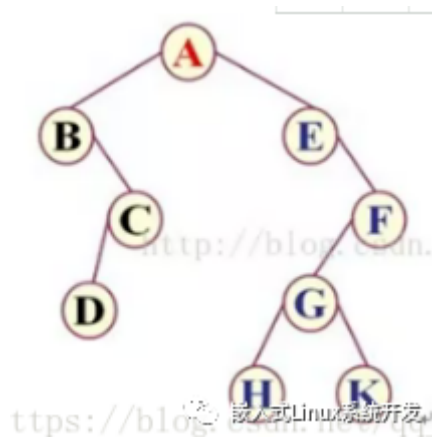
13、二分法查找的原理？

回答：类似于快速排序算法。二分法使用的前提是数组已经是有序序列，原理是折半查找，每次把表分成两半，因为已经排序的，所以只需要和中间数比较就能确定是在哪一半，然后不断分成两半，直到匹配，或者没有数字，表示查找失败。

14、二叉树了解过吗？前序，中序，后序遍历流程说一下。

回答：其实二叉树是很重要的数据结构，更深一点的知识点有平衡二叉树，红黑树，B树，B+树，B-树等，可以自行了解。这里只问了前中后序遍历，2021秋招百度笔试也考了这道题。

- 1 前序遍历：前序遍历的顺序为 根-左-右
- 2 中序遍历：中序遍历的顺序为 左-根-右
- 3 后序遍历：后序遍历的顺序为 左-右-根



- 1 前序遍历: ABCDEFGHK
- 2 中序遍历: BDCAEHGKF
- 3 后序遍历: DCBHKGF EA

15、内核裁剪说一下。

回答：个人理解，内核裁剪的原因主要是Linux内核本身很庞大，但客户有时候不需要这么多功能，想裁剪，定制内核，定制功能。

简单的内核的配置有三种方式，在命令行输入：

- 1 1、make config 文本配置方式；
- 2 2、make menuconfig 菜单配置方式；
- 3 3、make xconfig 图形界面配置方式（需安装qt）。

我一般采用第二种去配置内核，配置完以后make。

补充个例子:假如客户需要在手机固件升级的时候，有个指示灯闪烁，该怎么办？这个时候你单纯写应用代码是没用的，因为手机固件升级的时候，系统都没有跑起来，这时候代码只能写在Linux内核的bootloader里，因为bootloader起来了。做嵌入式开发还是要熟悉Linux内核的，没学习的赶紧学习，用到是早晚的事。

16、memcpy(), strcpy(), strcmp(), strcat()有什么区别，如何避免内存越界？

回答：这几个函数，做嵌入式岗位必须要会，按照学校老师的话来讲：这道题是这次考试的必考题，15分就放在那里，爱背不背。这道题，浙江大华也考了，mtk也考了，做嵌入式对内存太敏感，必考。

strcpy函数会导致内存溢出。strcpy拷贝函数不安全，他不做任何的检查措施，也不判断拷贝大小，不判断目的地址内存是否够用。

```
1 char *strcpy (char *strDest, const char *strSrc)
```

strncpy拷贝函数，虽然计算了复制的大小，但是也不安全，没有检查目标的边界。

```
1 strncpy(dest, src, sizeof(dest));
```

strncpy_s是安全的

strcmp(str1,str2)，是比较函数，若str1=str2，则返回零；若str1<str2，则返回负数；若str1>str2，则返回正数。（比较字符串）

strncat()主要功能是在字符串的结尾追加n个字符。

```
1 char * strncat(char *dest, const char *src, size_t n);
```

strcat()函数主要用来将两个char类型连接。例如：

```
1 char d[20]="Golden";
2 char s[20]="view";
3 strcat(d,s);
4 //打印d
5 printf("%s",d);
```

输出 d 为 GoldenView（中间无空格）

memcpy拷贝函数，它与strcpy的区别就是memcpy可以拷贝任意类型的数据，strcpy只能拷贝字符串类型。

memcpy 函数用于把资源内存（src所指向的内存区域）拷贝到目标内存（dest所指向的内存区域）；有一个size变量控制拷贝的字节数；

函数原型：

```
1 void *memcpy(void *dest, void *src, unsigned int count);
```

17、栈和队列的区别？

回答：这两种数据结构都很重要，栈是先入后出，队列是先入先出。

18、memcpy函数以什么结尾？

回答：与strcpy相比，memcpy遇到'\0'不结束，而且一定会复制完n个字节，函数原型在上面。

19、你最有成就的项目或者经历是什么？

20、你有什么想问的吗？

5、浙江大华面试

来源于读者投稿，作者Angel。

岗位：嵌入式软件工程师。

浙江大华嵌入式软件笔试一共37道题，35道选择填空，2道编程，时间60分钟，C语言一半，C++一半。

一面是晚上九点半打电话过来，过程15分钟，可能他们还在加班。二面是现场面，过程20分钟。现场面流程很快，二面完毕HR面，当天面试完毕。一周内发offer。

一面：

1、简单的自我介绍

回答：我主要介绍过去两年我做的项目和个人的知识框架

2、问了一下我主要用的编程语言，我说C

3、因为他们也做相机类产品，所以我把我实习的**项目**讲了一遍，包括标定sensor，标定shading、AWB这些相机相关知识。

4、如何防止编译器优化？

回答：volatile

那就顺便解释一下关键词static、const、extern的作用（经典）。

5、在C++代码中嵌入C代码，需要做什么？

回答：extern C

6、进程与线程，分配资源的最小单位是什么？

回答：进程

7、网络编程中长链接和短链接？

回答：

- ```
1 短连接
2
3 1、连接->传输数据->关闭连接
4 2、比如HTTP是无状态的短链接，浏览器和服务器每进行一次HTTP操作，就建立一次连接，但任务结束就中断连接。
5 3、因为连接后接收了数据就断开了，所以每次数据接受处理不会有联系。这也是HTTP协议无状态的原因之一。
6 长连接
7
8 1、连接->传输数据->保持连接 -> 传输数据->->直到一方关闭连接，多是客户端关闭连接。
9 2、长连接指建立SOCKET连接后不管是否使用都保持连接，但安全性较差。
```

什么时候用长连接，短连接？

长连接多用于操作频繁，点对点的通讯，而且连接数不能太多情况。每个TCP连接都需要三步握手，这需要时间，如果每个操作都是先连接，再操作的话那么处理速度会降低很多，所以每个操作完后都不断开，次处理时直接发送数据包就OK了，不用建立TCP连接。例如：数据库的连接用长连接，如果用短连接频繁的通信会造成socket错误，而且频繁的socket 创建也是对资源的浪费。

而像WEB网站的http服务一般都用短链接，因为长连接对于服务端来说会耗费一定的资源，而像WEB网站这么频繁的成千上万甚至上亿客户端的连接用短连接会更省一些资源，如果用长连接，而且同时有成千上万的客户，如果每个用户都占用一个连接的话，那可想而知吧。所以并发量大，但每个用户无需频繁操作情况下需用短连好。

## 8、多线程编程中，写线程安全的函数要注意哪些点？

回答：注意的点比较多，这是属于多线程编程的知识点。首先需要注意写的函数是可重入的：CPU运行时收到信号，于是暂停目前正在执行的函数，转到信号处理函数，而这个信号处理函数的执行过程中，又恰恰也会进入到刚刚执行的函数，这样便发生了所谓的重入。此时如果能够正确的运行，而且处理完成后，之前暂停的也能够正确运行，则说明它是可重入的，反复调用都得到正确的结果。其余的点请大家自行百度。

## 9、断言(assert)的用法

回答：assert 不仅仅是个报错函数，事实上，它是个宏，并且作用并非"报错"。assert 宏的原型定义在 assert.h 中，其作用是如果它的条件返回错误，则终止程序执行。

```
1 #include "assert.h"
2 void assert(int expression);
```

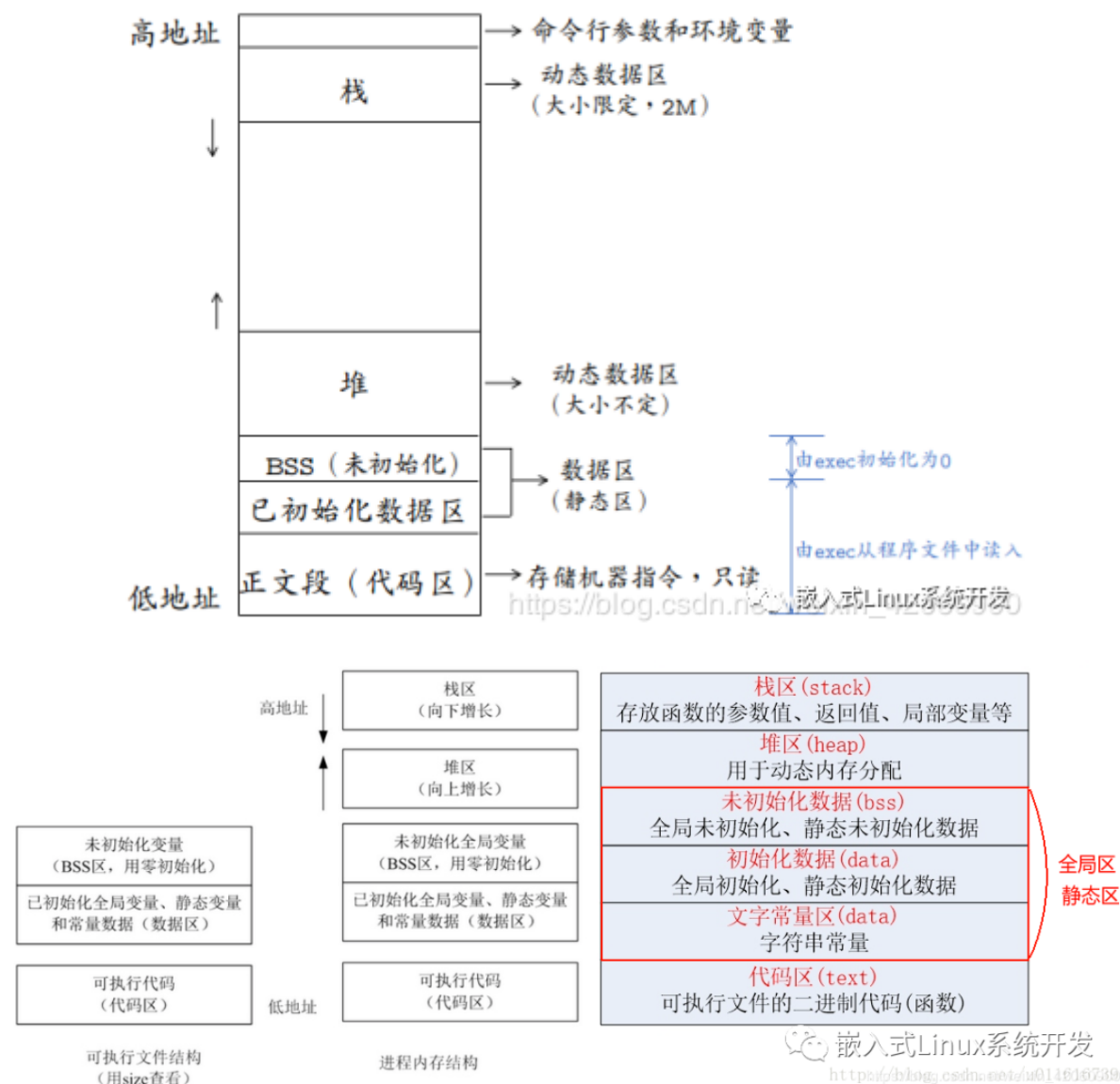
assert 的作用是现计算表达式 expression，如果其值为假（即为0），那么它先向 stderr 打印一条出错信息,然后通过调用 abort 来终止程序运行。

用法：

1)在函数开始处检验传入参数的合法性

- 2)每个assert只检验一个条件,因为同时检验多个条件时,如果断言失败,无法直观的判断是哪个条件失败
- 3)不能使用改变环境的语句,因为assert只在debug阶段生效,如果这么做,会使用程序在真正运行时遇到问题
- 4)assert和后面的语句应空一行,以形成逻辑和视觉上的一致感

## 10、内存四区（堆、栈、全局区、代码区）



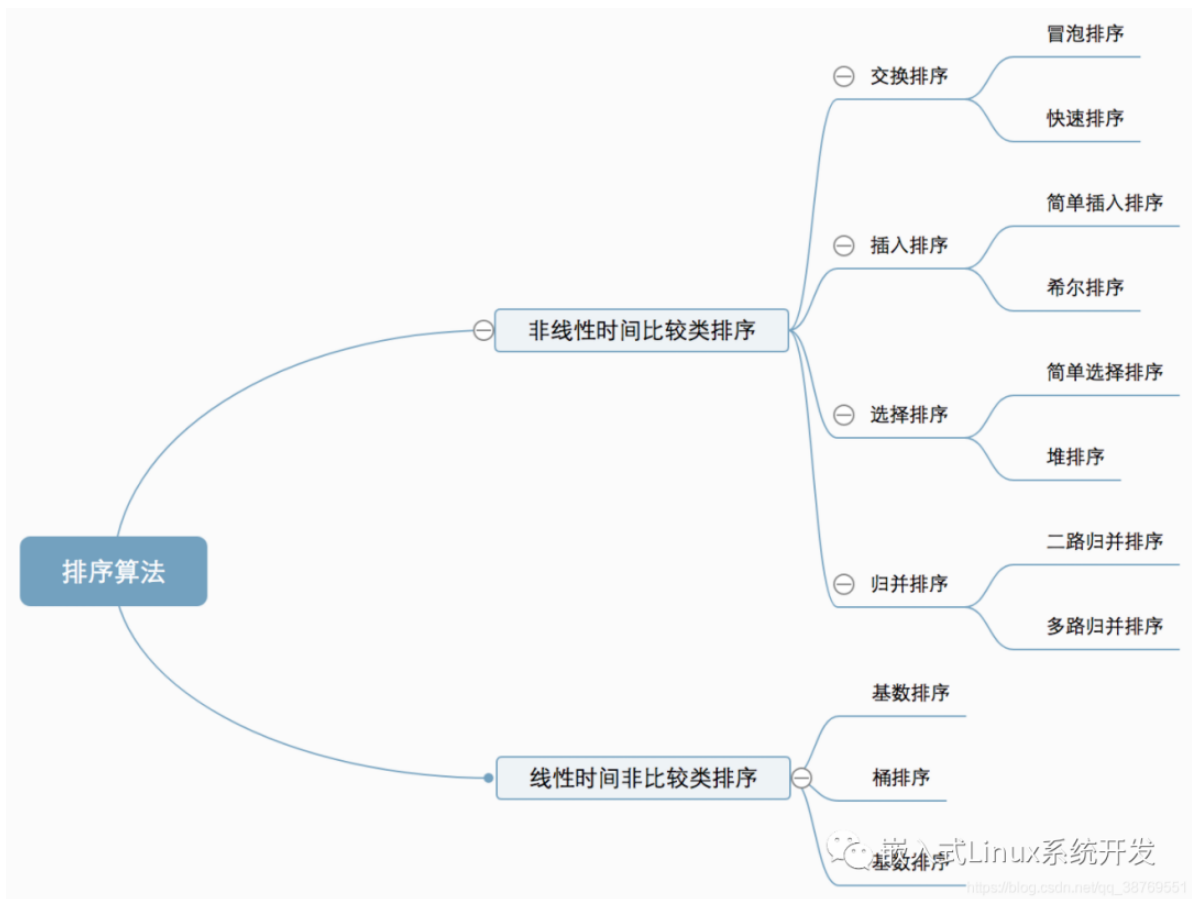
## 11、了解几种排序算法，时间复杂度是多少？

回答：

十种常见排序算法可以分为两大类：

**非线性时间比较类排序：**通过比较来决定元素间的相对次序，由于其时间复杂度不能突破 $O(n\log n)$ ，因此称为非线性时间比较类排序。

**线性时间非比较类排序：**不通过比较来决定元素间的相对次序，它可以突破基于比较排序的时间下界，以线性时间运行，因此称为线性时间非比较类排序。



算法优劣评价术语：

稳定性:

稳定：如果 a 原本在 b 前面，而  $a = b$ ，排序之后 a 仍然在 b 的前面；

不稳定：如果 a 原本在 b 的前面，而  $a = b$ ，排序之后 a 可能会出现在 b 的后面；

排序方式:

内排序：所有排序操作都在内存中完成，占用常数内存，不占用额外内存。

外排序：由于数据太大，因此把数据放在磁盘中，而排序通过磁盘和内存的数据传输才能进行，占用额外内存。

复杂度:

时间复杂度: 一个算法执行所耗费的时间。

空间复杂度: 运行完一个程序所需内存的大小。

| 排序方法 | 最好时间         | 平均时间          | 最坏时间         | 辅助空间       | 稳定性 |
|------|--------------|---------------|--------------|------------|-----|
| 直接插入 | $O(n)$       | $O(n^2)$      | $O(n^2)$     | $O(1)$     | 稳定  |
| 二分插入 | $O(n)$       | $O(n^2)$      | $O(n^2)$     | $O(1)$     | 稳定  |
| 希 尔  |              | $O(n^{1.25})$ |              | $O(1)$     | 不稳定 |
| 冒 泡  | $O(n)$       | $O(n^2)$      | $O(n^2)$     | $O(1)$     | 稳定  |
| 快 速  | $O(n \lg n)$ | $O(n \lg n)$  | $O(n^2)$     | $O(\lg n)$ | 不稳定 |
| 直接选择 | $O(n^2)$     | $O(n^2)$      | $O(n^2)$     | $O(1)$     | 不稳定 |
| 堆    | $O(n \lg n)$ | $O(n \lg n)$  | $O(n \lg n)$ |            | 不稳定 |
| 归 并  | $O(n \lg n)$ | $O(n \lg n)$  | $O(n \lg n)$ | $O(n)$     | 稳定  |
| 基 数  | $O(d(rd+n))$ | $O(d(rd+n))$  | $O(d(rd+n))$ | $O(rd+n)$  | 稳定  |

| 排序算法 | 平均时间复杂度         | 最好情况               | 最坏情况               | 空间复杂度           | 排序方式 | 稳定性 |
|------|-----------------|--------------------|--------------------|-----------------|------|-----|
| 冒泡排序 | $O(n^2)$        | $O(n)$             | $O(n^2)$           | $O(1)$          | 内排序  | 稳定  |
| 选择排序 | $O(n^2)$        | $O(n^2)$           | $O(n^2)$           | $O(1)$          | 内排序  | 不稳定 |
| 插入排序 | $O(n^2)$        | $O(n)$             | $O(n^2)$           | $O(1)$          | 内排序  | 稳定  |
| 希尔排序 | $O(n \log n)$   | $O(n(\log_2^2 n))$ | $O(n(\log_2^2 n))$ | $O(1)$          | 内排序  | 不稳定 |
| 归并排序 | $O(n \log n)$   | $O(n \log n)$      | $O(n \log n)$      | $O(n)$          | 外排序  | 稳定  |
| 快速排序 | $O(n \log n)$   | $O(n \log n)$      | $O(n^2)$           | $O(\log n)$     | 内排序  | 不稳定 |
| 堆排序  | $O(n \log n)$   | $O(n \log n)$      | $O(n \log n)$      | $O(1)$          | 内排序  | 不稳定 |
| 计数排序 | $O(n + k)$      | $O(n + k)$         | $O(n + k)$         | $O(k)$          | 外排序  | 稳定  |
| 桶排序  | $O(n + k)$      | $O(n + k)$         | $O(n^2)$           | $O(n + k)$      | 外排序  | 稳定  |
| 基数排序 | $O(n \times k)$ | $O(n \times k)$    | $O(n \times k)$    | $O(n \times k)$ | 外排序  | 稳定  |

至于各种算法的原理以及代码实现，由于太多并且比较复杂，不在本文列出。但推荐两本入门的书：《啊哈！算法》、《大话数据结构》。电子版我会发在交流群里。

排序算法很多，嵌入式要求的不会太多，你会冒泡排序、快速排序、插入排序就可以解决很多问题。难的比如动态规划问题，图的路径问题，嵌入式考的比较少，纯软才会考这些。（大公司和独角兽公司考的会相对难一些）

二面：

二面是现场面试，大概20分钟左右。

1、对着简历面试，问你简历写的东西。

2、对着你笔试的时候的题目，看你做的情况，随便选一个你做对的或者你做错的让你解释。（所以笔试完一定要搞懂）

### 3、问进程线程，经典问题。

### 4、问strcpy拷贝函数安全吗，如果不安全，用什么去替代。

不安全，strcpy函数会导致内存溢出，因为他不做任何的检查措施，也不判断拷贝大小，不判断目的地址内存是否够用。

```
1 | char *strcpy (char *strDest, const char *strSrc)
```

strncpy拷贝函数，虽然计算了复制的大小，但是也不安全，没有检查目标的边界。

```
1 | strncpy(dest, src, sizeof(dest));
```

strncpy\_s是安全的

memcpy拷贝函数，它与strcpy的区别就是memcpy可以拷贝任意类型的数据，strcpy只能拷贝字符串类型。

memcpy 函数用于把资源内存（src所指向的内存区域）拷贝到目标内存（dest所指向的内存区域）；有一个size变量控制拷贝的字节数；

```
1 | void *memcpy(void *dest, void *src, unsigned int count);
```

### 5、写一个双向链表的插入删除

### 6、解释一下动态内存和静态内存

回答：

- a) 静态内存分配在编译时完成，不占用CPU资源; 动态内存分配在运行时，分配与释放都占用CPU资源。
- b) 静态内存存在栈(stack)上分配; 动态内存存在堆(heap)上分配。
- c) 动态内存分配需要指针和引用类型支持，静态不需要。
- d) 静态内存分配是按计划分配，由编译器负责; 动态内存分配是按需分配，由程序员负责。

## 6、深信服面试

1、简历中写了做过海思项目，问海思项目最大的难点是什么?怎么解决的。（还问了编码的原理，h264和265的区别，如果sensor默认是720p的，我要怎么得到1080p，应该在哪里修改）

2、野指针产生原因和解决方法

3、指针函数和函数指针的区别？

4、strcpy是安全的吗，那strncpy又一定是安全的吗。手写memcpy实现的时候，需要考虑到什么问题（需要考虑到如果原地址+拷贝的字节数大于目标地址时，会出现覆盖的问题）

5、说一下函数宏和内联函数

6、说一下什么时候用线程什么时候用进程

7、说一下C++里面虚函数的作用

8、说一下二叉树

9、C++ STL了解过吗，说一下STL各个容器的用法

10、说一下socket的内部原理

- 11、给出两个字符串，求最长的相同子串
- 12、给一个字符串，然后把这个字符串逆序
- 13、链表有几种，每种链表有什么区别
- 14、数组和链表的区别，分情况讨论一下它们的效率问题
- 15、说一下快排的思想，它的复杂度是多少，和冒泡排序，选择排序比起来是怎么样的
- 16、怎么看待加班，能接受出差吗？
- 17、说一下你的优点和缺点
- 18、说一下你对这个岗位的理解
- 19、说一下你未来五年的工作计划
- 20、你是怎么看待以后嵌入式的发展行情的
- 21、你觉得作为一个优秀的程序员，应该怎么做
- 22、说一下linux下，和文件io相关的函数
- 23、说一下extern关键字
- 24、说一下内存四区
- 25、说一下多进程多线程的通信方式，说一下用过哪个。
- 26、说一下网络编程中，在服务器这边，需要用到哪些函数，这些函数的作用是什么
- 27、然后问到了char a [ 0 ]，也就是一个没有元素的数组，面试官把这个结合着结构体来问我，我当时完全不懂什么意思，然后面试官说应该是我没用过，所以就跳过了。
- 28、说一下栈和队列的特点，如何判断一个队列是满的或是空的，如何利用栈来实现加减乘除（很经典的一个问题）
- 29、有看过malloc源码吗？简单说一下malloc的原理
- 30、说一下你的四年大学生活

## 7、浙江大华面试（2）

一面（25分钟）：

1、自我介绍

2、简历中写了做过朱友鹏老师的海思音视频项目，所以问了海思项目。

回答：问vi-vpss-venc模块的东西，问这些模块里面的代码有没有深入去了解过，这一块问得比较深，一直问到底部用到的那些API的功能；

问我项目用了几个线程；有没有用到锁机制；VLC播放器也就是客户端这边，大概是通过什么样的方式和过程来连接服务器的；

然后详细问了我项目中用链式队列构建环状buffer的点，问我的思路；

3、问我简历上的FTP项目，问遇到过什么难点。问是按照FTP协议的写代码的，还是我自己自定义协议来写代码的。

4、你是XXX专业的，是怎么学习嵌入式的？

5、二叉树会不会？

我说之前学过，去leetcode上面刷过题，但是没有用过了。而且隔了四五个月了，基本上都忘了。平时用得比较多的还是链表、栈和队列。

6、linux创建进程和线程的函数是什么，说一下？

回答：我把函数名字和函数的各个参数都详细地说出来了，面试官就没有深问下去了。

7、linux下进程的通信方式；说一下信号量，熟悉哪些信号量，用到哪些函数；说一下共享内存，分别调用了哪些函数以及这些函数的功能？

8、在CS架构中，服务器是怎么样判断客户端什么时候断开连接（当read函数的返回值位0的时候，说明对端关闭）

**总结：**问的问题都会，而且回答得很流畅，没有拖沓，面试难度不大，主要都是细节部分，细到包括系统函数的参数分别是什么；然后他说他那边觉得我没啥问题了，明确告诉我会有一面，一面会在一个星期内给通知。

二面（20分钟）：

1、问很多简历上的海思项目，问你做的那个构建环状buffer部分，问得很深，几乎是从源码的角度来扣的。

2、问死锁，怎么生成的，举个例子？

3、数组、链表什么区别？分情况讨论一下它们的效率问题？

4、数据结构学过吗，二叉树学过吗？说一下

5、虚函数解决的是什么问题？析构函数在什么时候需要声明为虚函数？

6、进程的虚拟地址有多少，物理地址和虚拟地址的关系，这两种地址是怎么建立起关系的？

7、多线程多进程的区别？

8、手上有多少offer？

最后面试官说面试通过了，七天内会有hr跟我聊薪资的。

## 8、海康威视面试

时间：2021.03.23,15分钟

1、TCP的包头有哪些东西？

2、数组和链表的优点？

3、线程和进程的区别？

4、说一下消息队列。

5、说一下红黑树。

6、说一下内核链表的实现。

7、说一下自己的项目中，有哪些是能提现深度思考后得到的，做出来的东西。

8、简历的项目中，那个TFP项目，如果遇到网络突然中断了，中断传输了，应该怎么办？

9、说一下断点传输文件的好处。

10、简历海思项目中，有哪些情况会导致掉帧的情况，说一下。

11、用过RTMP吗？说一下（音视频领域）

回答：海康威视和浙江大华都是中国安防的龙头企业，嵌入式软件岗位偏向于音视频的技术，关于从事音视频开发岗位所需要的技能，请参看我的另一篇文章。

[音视频开发专业词汇总结及音视频处理流程](#)

[行车记录仪主要方案公司](#)

投稿读者简历中写的海思项目是跟着朱友鹏老师学的基于海思Hi3518的音视频项目，海思音视频芯片还有很多款，比如Hi3516、Hi3518、Hi3556、Hi3556a、Hi3559V100、Hi3559V200，其中Hi3559V200可以输出4K30分辨率。后来华为海思被美国制裁后，高端芯片缺货，导致很多方案公司转了安霸或者联咏芯片方案。但是音视频确实嵌入式岗位很好的一个发展领域。在这里放两张截图。

中国移动 4G

下午 10:38

14%

< C++音视频开发工程师

☆ ⚠

职位详情

Go 网络协议 音视频编解码 流媒体技术

音视频开发 架构师

岗位职责：

1、负责音视频平台架构设计；

2、根据产品需求,进行产品研发；

3、持续对系统架构进行升级和优化；

4、负责团队中初级音视频开发人员的指导培养。

岗位要求：

1、大专及以上学历，计算机相关专业，1年以上音视频开发经验；

2、熟练掌握C、C++、GOLANG等服务器开发语言；

3、能够阅读rfc文档阅读并据此实现代码；

4、熟悉 libx264/libx265/nvenc 等视频编码器、h264/h265单元结构、rtsp/rtcp/rtp/sip/hls/gb28181等协议或规定的一种或多种；

5、至少精通1种媒体封装协议或者传输协议，如WebRTC、FLV、HLS、MP4、TS、PS、RTP、RTMP、RTSP、SIP、H.323、GB28181等。

6、具备较强的学习能力，独立思考及解决问题能力，良好的表达沟通能力，有较强的团队精神。

不感兴趣

继续沟通

嵌入式Linux系统开发

工作地点：深圳，

岗位要求：

- 1、有 IPC 开发经验，熟悉海思、安霸、安凯网络摄像机平台；
- 2、熟悉 linux 内核移植和开发；
- 3、熟悉 RTSP/RTP/等流媒体协议和 TCP/IP 网络协议；
- 4、有 camera、行车记录仪、运动 DV 或可视安防产品经验优先；
- 5、熟悉音视频技术和软件开发，有 H.264 编解码经验优先；

代问一下，应届生收吗？

无名卒-嵌入式-底层劳动

深圳哪里？薪资范围？

16:33

Lee

深圳 南山区深圳湾。一般是 20-40k，14 薪。

嵌入式Linux系统开发

## 9、小米面试（2）

投递岗位：嵌入式软件工程师-BSP方向

面试时间：2021.03.29

岗位描述：

×

小米集团 - 校园招聘

...

<

职位详情

软件开发工程师-BSP方向

小米公司 | 全职 | 软件研发类

湖北 · 武汉市

职位描述

岗位描述：

将基于linux系统移植到开发板、bringup开发板，解决驱动程序问题。

任职资格：

1、本科及以上学历，计算机相关专业；

2、熟练使用c语言；

3、扎实的数据结构基础和计算机系统的基础；

4、熟练使用linux系统；

5、具有一定的硬件知识。

职位详情

投递简历

2021春季校园招聘允许每人投递2个职位。请您选择适合的职位进行投递

 嵌入式Linux系统开发

一面（30分钟）：

主要问你简历自己写的东西。

1、做一个英文自我介绍。

2、uboot启动流程？

回答：从uboot如何引导内核启动，kernel启动时怎么挂载根文件系统从头到尾说了一下。

3、面试官说通过了，进入下一轮。

二面（25分钟）：

1、驱动下半部分需要做些什么？

2、C语言和C++有什么区别？比如static，malloc和new。

3、C++的三大特征说一下。

4、你C++学的怎么样？

回答：主要用C语言，不太了解C++。

5、问简历中项目实现框架和细节。

补充：

关于嵌入式方向“驱动工程师”和“BSP工程师”的区别，做了一点总结，欢迎补充。

BSP即Board Support Package，板级支持包。它来源于嵌入式操作系统与硬件无关的设计思想，操作系统被设计为运行在虚拟的硬件平台上。对于具体的硬件平台，与硬件相关的代码都被封装在BSP中，由BSP向上提供虚拟的硬件平台，BSP与操作系统通过定义好的接口进行交互。BSP是所有与硬件相关的代码体的集合。

BSP在嵌入式系统中的角色，很相似于在PC系统中的BIOS和驱动程序的地位。

**BSP就看成BIOS+驱动或者bootloader+驱动。**

BSP和驱动的区别：bsp还包括了bootloader，或者bsp是包含了cpu、电源管理这些比驱动更深层次的硬件支持包。

1、BSP一般是芯片原厂提供的。驱动是自己写的，比如写IIC协议驱动一个EEPROM，写一些SPI、USB驱动等。

2、bsp是保证os启动前所作的必须工作，包括初始化工作，引导os启动起来，也包括一些用户的配置信息。os启动后，可以认为bsp已经不再起作用了，os会调相应的驱动来重新初始化系统。驱动可以认为是os内核中的一部分。

3、BSP是嵌入式系统中介于硬件平台和操作系统之间的中间层软件，主要目的是为了屏蔽底层硬件的多样性，根据操作系统的要求完成对硬件的直接操作，向操作系统提供底层硬件信息并最终启动操作系统。BSP具有硬件相关性和操作系统相关性的特点，其主要作用包括：

(1)初始化底层硬件，为操作系统提供底层硬件信息；

(2)初始化相关硬件设备，主要是存储设备、通信设备；

(3)检测系统硬件是否正常；

(4)加载操作系统并启动系统运行。

4、BSP也称为硬件抽象层HAL或者中间层。

它将系统上层软件和底层硬件分离开来，使系统上层软件开发人员无需关系底层硬件的具体情况，根据BSP层提供的接口开发即可。

BSP是相对于操作系统而言的，不同的操作系统有不同定义形式的BSP，要求BSP所实现的功能也有所不同。

在嵌入式Linux系统中，主要是初始化底层硬件并引导操作系统；同时，BSP又是和硬件相关的，还要考虑对硬件的初始化操作。这些初始化操作主要是对CPU、内存、中断等相关的寄存器及协处理器进行正确的配置。

在不同的开发阶段，因为核心和文件系统所处的位置不同，BSP所要完成的工作也有所不同；在开发调试阶段，BSP要能够与主机通信并从主机下载核心；在目标产品中，BSP要能够从非易失存储设备中加载核心。

## 第九章、各大公司笔试题

## 1、大疆笔试题

在深圳做嵌入式，大疆公司绝对是Top级别的，大疆的技术栈也很深。但2020受美国制裁后，有所缩招。另外提醒，研发岗对学历要求高一些。

大疆公司每年秋招是分批考试，考试时间自选，分为AB卷，本文分享是B卷。

选考时间：2020.08.16，19:00-20:30 (A卷为2020.08.10)

题型：单选(2' \* 10)、多选(3' \* 5)、填空(4' \* 4)、简答(6' \* 3)、编程(\*2 = 31')

B卷主要考察C语言，还考察一些Linux和ARM知识点

### 一、单选：

1、

```
1 | const *char 和 const char* 定义变量的区别
```

2、关于cache错误的是？

3、sizeof 结构体 (含位域的)

```
1 | unit16_t
2 | unit32_t
3 | unit8_t
```

4、FIQ中断向量入口地址？(考察ARM知识点)

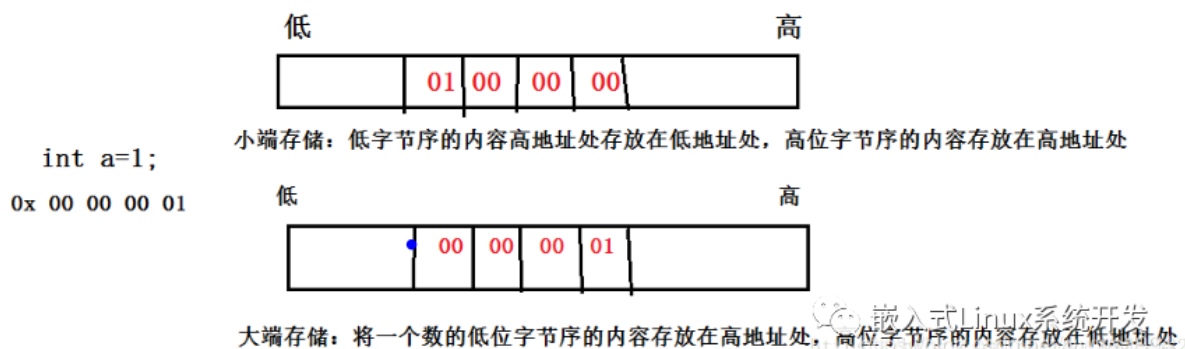
回答：FIQ的中断向量地址在0x0000001C，而IRQ的在0x00000018。

5、R15除了本身的功能还可以作为程序计数器？

回答：寄存器R13在ARM指令中常用作堆栈指针SP，寄存器R14称为子程序链接寄存器LR(Link Register)，寄存器R15用作程序计数器(PC)。

ARM微处理器共有37个32位寄存器，其中31个为通用寄存器，6个位状态寄存器。通用寄存器R0~R14、程序计数器PC(即R15)是需要熟悉其功能的。

6、如何判断机器大小端？



联合体方法判断方法：利用union结构体的从低地址开始存，且同一时间内只有一个成员占有内存的特性。大端存储符合阅读习惯。联合体占用内存是最大的那个，和结构体不一样。

a和c公用同一块内存区域，所以更改c，必然会影响a的数据

```
1 | #include<stdio.h>
```

```

2
3 int main(){
4 union w
5 {
6 int a;
7 char b;
8 }c;
9 c.a = 1;
10 if(c.b == 1)
11 printf("小端存储\n");
12 else
13 printf("大端存储\n");
14 return 0;
15 }

```

## 指针方法

通过将int强制类型转换成char单字节，p指向a的起始字节（低字节）

```

1 #include <stdio.h>
2 int main ()
3 {
4 int a = 1;
5 char *p = (char *)&a;
6 if(*p == 1)
7 {
8 printf("小端存储\n");
9 }
10 else
11 {
12 printf("大端存储\n");
13 }
14 return 0;
15 }

```

## 二、多选

- 1、哪些类型可以自加：i++
- 2、全双工总线类型有哪几个？

| 总线接口 | 串/并 | 同步/异步 | 速率         | 工作方式 | 用线                                 | 总线拓扑结构                                                                                                                                                                                  | 信距离          |
|------|-----|-------|------------|------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| UART | 串   | 异步    | 慢<br>波特率设置 | 全双工  | 2线<br>Rx、Tx                        | RS485支持总线式、<br>星形、树形                                                                                                                                                                    | 远<br>最远1200m |
| I2C  | 串   | 同步    | 慢          | 半双工  | 2线<br>SDA、SCL                      | 总线型（特殊的树形）                                                                                                                                                                              | 近            |
| SPI  | 串   | 同步    | 快          | 全双工  | 3线或4线<br>SCLK、SIMO、<br>SOMI、SS(片选) | 环形                                                                                                                                                                                      | 远            |
| USB  | 串   | 同步    | 快          | 半双工  | 4线<br>Vbus（5V）、GND、<br>D+、D-（3.3V） |  嵌入式Linux系统开发<br><a href="https://blog.csdn.net/qq_38769551">https://blog.csdn.net/qq_38769551</a> |              |

- 3、线程间同步方式？

进程间通讯：

(1) 有名管道/无名管道 (2) 信号 (3) 共享内存 (4) 消息队列 (5) 信号量 (6) socket

线程通讯：

(1) 信号量 (2) 读写锁 (3) 条件变量 (4) 互斥锁 (5) 自旋锁

### 三、填空

1、填一种编译优化选项：-o

2、在有数据cache情况下，DMA数据链路为：外设-DMA-DDR-cache-CPU,CPU需要对cache做什么操作，才可以得到数据？

3、面向对象编程三大特点？

回答：封装、继承和多态

### 四、简答

1、SPI四种模式，简述其中一种模式，画出时序图？

回答：请参考CSDN博客。

2、判断大小端的三种方式？

3、为什么TCP是稳定传输？

回答：可以从TCP和UDP的区别出发去回答。

### 五、编程

#### 1、求最大的和：

取两个不重复的字串，求他们的最大的和

输入

|   |                         |
|---|-------------------------|
| 1 | 10                      |
| 2 | 1 -1 2 2 3 -3 4 -4 5 -5 |

取 2、2、3、-3、4、5，最大输出13

输入

|   |               |
|---|---------------|
| 1 | 5             |
| 2 | -5 9 -5 11 20 |

取9、11、20，加起来40

输入

|   |                               |
|---|-------------------------------|
| 1 | 10                            |
| 2 | -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 |

答案是-2

## 2、停车，求收费最多（数据太多，可能有误，自己可以想思路）

|   |      |   |   |    |   |
|---|------|---|---|----|---|
| 1 | 用户编号 | 1 | 2 | 3  | 4 |
| 2 | 开始时间 | 1 | 2 | 3  | 7 |
| 3 | 结束时间 | 4 | 4 | 12 | 9 |

同一时间只能服务一个用户

|   |                 |        |
|---|-----------------|--------|
| 1 | $1 \leq t < 6$  | 10元一小时 |
| 2 | $6 \leq t < 10$ | 5元一小时  |
| 3 | $10 \leq t$     | 2元一小时  |

## 2、小米笔试题

岗位：嵌入式软件工程师

笔试时间：2020.09.08，18：00-19：30

选择题10（3分）=30

专项选择题10（3分）=30

编程题2（20分）=40

选择题：

- 1、如果要在根目录运行程序，需要把路径设置到下面哪个环境变量中？
- 2、inline函数的特性。
- 3、冯诺依曼体系：计算机硬件由运算器、控制器、存储器、输入设备和输出设备五大部分组成。
- 4、对900，512，613，700，810，采用选择排序，则第三轮排序后结果输出是多少？
- 5、某一段程序的时间复杂度是多少？（考察时间复杂度和空间复杂度）
- 6、简单逻辑题。
- 7、入栈ABCDE，不可能的出栈顺序是？

答案：参考我的另一篇文章

[一个栈的入栈序列为ABCDEF，则不可能的出栈序列是？](#)

- 8、死锁是怎么发生的，以下哪种情况会发生死锁？
- 9、在搜索中“主键”的意思是？
- 10、采样频率至少不小于多少才能还原波形？

专项选择题：

- 1、构造函数和析构函数的调用参数的顺序是？
- 2、进程和线程的区别？（经典）

答案：经典题目，参见下面链接。

3、java如何实现类继承？

4、java的垃圾回收机制是哪些表述？

5、结构体避免编译器优化的关键字？

比如：`attribute((packed))`

6、java中，`switch ( expression )` 语句，`expression`不可能是什么类型？

7、一段python代码运行结果是？

8、C语言中static关键字的作用？（经典）

9、C++中不能重载的符号是哪一项？

`++`、`>=`、`sizeof`、`delete`？

10、C++中生成.exe文件的过程

## 编程题

### 1、密码生成器

时间限制：1000MS

内存限制：65536KB

题目描述：

注册网站时，需要使用包含不同类型(数字、符号、大写字母、小写字母)的字符，和特定长度。检查一个密码内容同时包含以上4种类型的字符，并且长度在8-120个字符之间。符合要求，返回0;长度不符合返回1;类型不符合要求返回2.

可以一次输入多组密码，以空格符间隔，空格符不作为密码。

输入描述

1 | 需要验证的密码，多个密码以空格符间隔，空格符不作为密码的部分

输出描述

1 | 每个密码的检查结果，每个结果需要换行输出

样例输入

1 | 123 12345678 123abcABC!!!

样例输出

1 | 1  
2 | 2  
3 | 0

### 2、单词搜索

第二道题是leetcode原题第78题，难度中等。

时间限制：3000MS

内存限制：589824KB

题目描述：

给定一个二维网格和一个单词，找出该单词是否存在于网格中。

单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

输入描述：

```
1 | 需要查询的单词
```

输出描述：

```
1 | true代表单词存在，false表示不存在
```

示例

```
1 | board =
2 | [
3 | ['A','B','C','E'],
4 | ['S','F','C','S'],
5 | ['A','D','E','E']
6 |]
7 |
8 | 给定 word = "ABCCED", 返回 true
9 | 给定 word = "SEE", 返回 true
10 | 给定 word = "ABCB", 返回 false
```

### 3、锐捷网络笔试题

笔试时间：2020.09.24

时长：90分钟

题目：25道题，全是C语言代码题，问输出结果。

总结：时间紧，任务重，硬核。 答案在题目开头。

1、

```
1 | str1:
2 | str2: str2
3 | str3: null
```

第一个，函数内部定义的变量在函数结束时候释放掉了，所以返回会出错，

第二个是str2，

第三个定义是void，没有返回值，打印空，内存非法访问。

```
1 | #include<stdio.h>
2 |
3 | char *getstr1()
4 | {
```

```

5 char s[]="str1"; //static char s[]="str1";
6 return s;
7 }
8 char *getstr2()
9 {
10 char *s="str2";
11 return s;
12 }
13 void getstr3(char *s)
14 {
15 s="str3";
16 return;
17 }
18
19 int main()
20 {
21 char *s1,*s2,*s3;
22 s1=getstr1();
23 s2=getstr2();
24 getstr3(s3);
25 printf("str1:%s\r\n",s1);
26 printf("str2:%s\r\n",s2);
27 printf("str3:%s\r\n",s3);
28 }

```

2、 4 6 1

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6 char *str="Ruijie";
7
8 printf("str1:%d\r\n",sizeof(str));
9 printf("str2:%d\r\n",strlen(str));
10 printf("str3:%d\r\n",sizeof(*str));
11 }

```

3、 8 20

```

1 #include<stdio.h>
2
3 int main()
4 {
5 int x=0,y=0;`
6 while(x<15){
7 y++;
8 x+=++y;
9 }
10 printf("%d %d",y,x);
11 }

```

4、 48 ( 考察宏定义的边界效应 )

```

1 #include<stdio.h>
2
3 #define N 3
4 #define Y(n) ((N+1)*n)
5
6 int main()
7 {
8 int z;
9 z=2*(N+Y(5+1));
10 printf("%d",z);
11 }

```

5、0 1

ptr [ 0 ] 是等于&a [ 0 ] [ 0 ] , 所以\*ptr [ 0 ] =a [ 0 ] [ 0 ]

所以最后这个二维数组是 000 100 200

```

1 #include<stdio.h>
2 #include<string.h>
3
4 static int arr[3][3]={0};
5
6 void init(int (*ptr)[3])
7 {
8 int i=0;
9 for(i=0;i<3;i++){
10 *(ptr[i])=i;
11 }
12 return;
13 }
14
15 int main(int argc,char **argv)
16 {
17 init(arr);
18 printf("%d,%d\n",arr[0][1],arr[1][0]);
19 return 0;
20 }

```

6、%s 遇到\0才会截止输出，遇到\0才会截止计算

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6 char str[]={ 'o', 'u', 'l', 'd' };
7 printf("%d",strlen(str));
8 return 0;
9 }
10 #include<stdio.h>
11 #include<string.h>
12
13 int main()
14 {
15 char str[]={ 'o', 'u', 'l', 'd', '\0' };

```

```

16 | printf("%d\n",strlen(str));
17 | printf("%s\n",str);
18 |
19 | return 0;
20 | }

```

7、如果用%s输出，需要注意打印字符串的时候，他认为'\0'是结束符，所以你不加'\0'的时候，他可能会打印其他乱七八糟的东西

ABC

ABCABC

4

3

```

1 | #include<stdio.h>
2 | #include<string.h>
3 |
4 | int main()
5 | {
6 | char a[]="ABC";
7 | char b[]={ 'A', 'B', 'C' };
8 | printf("%s\n",a);
9 | printf("%s\n",b);
10 | printf("%d\n",sizeof(a));
11 | printf("%d\n",sizeof(b));
12 |
13 | return 0;
14 | }

```

8、 -1 , 4294967295

```

1 | #include<stdio.h>
2 | #include<string.h>
3 |
4 | int main()
5 | {
6 | char a=0xff;
7 | printf("%d,%u\n",a>>8,a>>8); //这里的%d 让它变成了int
8 | return 0;
9 | }

```

看下面。右移八位全部移走了，然后补了符号位，1111 1111。另外，在内存中，因为是有符号的，所以1111 1111是个负数，负数在内存中是补码的形式存在，所以11111111-1再取反就是真实的数，就是-1。

9、 0

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6 printf("%d\n",~3&(2&(3^4))); //按位取反 异或
7 return 0;
8 }

```

10、80

```

1 #include<stdio.h>
2
3 int main()
4 {
5 int a[3][4]={{10,20,30,40},{50,60,70,80},{90,91,92,93}};
6 int (*p)[4];
7 p=a+1;
8 printf("%d\n",p[1][-1]);
9 return 0;
10 }

```

11、25

```

1 #include<stdio.h>
2
3 int main()
4 {
5 int a[2][3]={{1,3,5},{2,4,6}};
6 int m,*ptr;
7
8 ptr=&a[0][0];
9 m=(*ptr)*(*(ptr+2))*(*(ptr+2));
10
11 printf("%d\n",m);
12 return 0;
13 }

```

12、静态变量全局只初始化一次 m=1,n=1 m=2,n=1

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int test()
5 {
6 static int m=0;
7 int n=0;
8
9 m++;
10 n++;
11 printf("m=%d,n=%d\n",m,n);
12
13 return 0;
14 }
15

```

```

16 int main()
17 {
18 test();
19 test();
20
21 return 0;
22 }

```

13、 4 2 2 ( 没有break的话会一直向下执行 )

```

1 #include<stdio.h>
2
3 int main()
4 {
5 int a=0,b=0,c=0;
6 int i;
7
8 for(i=0;i<5;i+=2){
9 switch(i){
10 case 1:
11 a++;
12 b++;
13 break;
14 case 2:
15 c++;
16 a++;
17 case 3:
18 c++;
19 case 4:
20 b++;
21 default:
22 a++;
23 break;
24 }
25 }
26 printf("%d,%d,%d\n",a,b,c);
27
28 return 0;
29 }

```

14、 e

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6 char a[7]="abcdef",b[4]="ABC";
7 strcpy(a,b);
8 printf("%c\n",a[4]);
9 return 0;
10 }

```

15、 4 3 ( 逗号运算符 )

```

1 #include<stdio.h>
2
3 int main()
4 {
5 int y=3,x=3,z=1;
6 printf("%d,%d\n", (++x, ++y), z+2);
7 return 0;
8 }

```

4 1

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6 int y=3,x=3,z=1;
7 //printf("%d,%d\n", (++x, ++y), z+2);
8 printf("%d,%d\n", (++x, ++y), z++);
9
10 return 0;
11 }

```

16、 b ( ASCII )

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6 char b='v';
7 printf("%C\n", b-20);
8 return 0;
9 }

```

17、 99 ( ASCII )

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6 char str[]="abcde";
7 char *p;
8 p=&str[2];
9 printf("%d\n", *p);
10 return 0;
11 }

```

18、什么情况下会编译通过，运行出错

- 1 A、gcc用-werror选项，程序组译失败。
- 2
- 3 B、gcc使用-werror选项，编译正常，程序运行时崩溃。
- 4
- 5 C、gcc使用-werror选项，编译正常)，程序正常运行，备份数据输出的为随机数，
- 6
- 7 D、程序正常运行，备份数据输出的为1.

19、以下哪些任务不可以放到中断的上半部执行？

- 1 A、时间敏感的任务
- 2
- 3 B、硬件相关的任务
- 4
- 5 C、进程通信的任务
- 6
- 7 D、不可被中断的任务

20、线程

25、如下所示程序，LINE C 和LINE P将会输出什么？

```
1 LINE C: value=
2 LINE P: value=
3
4 #include <stdio.h>
5 #include <pthread.h>
6 #include <unistd.h>
7 #include <sys/wait.h>
8
9 int value=0
10
11 void *runner(void *param) /* the thread */
12 {
13 value=5;
14 pthread_exit(0);
15 }
16
17 int main(void)
18 {
19 pid_t pid;
20 pthread_t tid;
21 pthread_attr_t attr;
22
23 pid=fork();
24 if(pid==0)
25 {
26 pthread_attr_init(&attr);
27 pthread_create(&tid,&attr,runner,NULL);
28 pthread_join(tid,NULL);
29 printf("LINE c: value=%d\n",value); /*LINE C*/
30 }
31 else if(pid> 0)
32 {
33 wait(NUL);
34 printf("LINE P: value=%d\n",value); /*LINE P*/
```

```
35 }
36 return 0;
```

## 4、360公司笔试题

投递的嵌入式Linux软件开发相关岗位，给的软件综合卷A卷（许多大公司笔试给的都是软件综合）。

因为是软件综合，所以各种知识都考查，java、C++、数据结构、算法、Linux等全部考察，所以要求你基础扎实。

笔试时间：2020.09.11，20：00——22：00

选择题40个，一共80分。

编程题2个，一共40分。

选择题：

- 1、Java中GUI菜单的设置规则
- 2、一段C++代码的运行结果
- 3、例图
- 4、E-R图的规则有哪些？
- 5、C类IP地址192.168.2.0/24，16台主机时的子网掩码是多少？
- 6、socket编程中so\_pcb 结构体的定义是哪个？
- 7、DNS定义是什么？
- 8、下列不是 NP 问题的是？
- 9、面向对象编程中：类，对象，实例的关系？
- 10、贪心算法适用于解决哪些问题？
- 11、LP问题是什么？
- 12、linux中查看路径变量的命令？

回答：echo \$PATH

- 13、linux中kill -9 什么意思？
- 14、linux中grub的配置文件哪里？

回答：在/boot/grub/grub.conf目录下

- 15、linux中mysql数据库默认的端口是？

回答：3306

mysql的默认端口是3306，可以编辑用户目录下的 .my.cnf 文件进行修改。

|   |            |             |
|---|------------|-------------|
| 1 | sqlserver  | 默认端口号为：1433 |
| 2 | oracle     | 默认端口号为：1521 |
| 3 | DB2        | 默认端口号为：5000 |
| 4 | PostgreSQL | 默认端口号为：5432 |

- 16、序列图是什么意思？

- 17、原始数据类型自己可以更改吗？
- 18、类的继承方式中，基类共有成员，保护成员，派生类有哪些不同？
- 19、使用游标的5个步骤？
- 20、下列系统调用中，不会导致进程阻塞的是？
- 21、多进程中临界区的概念如何解释？
- 22、有三个并发进程共享变量S，则与变量S相关的临界区有几个？
- 23、C++中，一个函数的返回值类型由什么决定？
- 24、由二叉树变成树的规则？
- 25、链栈？
- 26、n个顶点的有向图，所有顶点的出度之和为S，则所有顶点的入度之和为？
- 27、考察Java的语法？
- 28、举证
- 29、一段C++代码运行结果
- 30、一段C++代码运行结果
- 31、关系的三种类型？
- 32、考察分治法
- 33、如何用DFS遍历一个无环有向图
- 34、一段C++代码运行结果
- 35、一段C++代码运行结果
- 36、char数组强制转换成 int 命令是哪一个？
- 37、
- 38、一段C++代码运行结果
- 39、设串长为n，模式串长为m，则KMP算法所需的附加空间为？

- A、 $O(m)$
- B、 $O(n)$
- C、 $O(m*n)$
- D、 $O(n \log_2 m)$

KMP算法时间复杂度为 $O(m+n)$ ，空间复杂度为 $O(m)$ 。因为KMP算法涉及到next数组的存储，且next数组是基于模式串长度计算的。

- 40、关于标准形线性规划表述正确的是？

编程题：

- 1、企业管理：根据打开记录推算工号（具体忘了）
- 2、编写密码复杂性验证程序：输入几组字符串，要求有大写字母，小写字母，数字，符号，长度大于8，则输出pass，否则提示错误。

## 5、百度笔试题

笔试时间：2020.09.03，19：00-21：00

题型：

单选15道×3=45

不定项选择题5道×3=15

编程题3道×20=60



### 百度2021校招嵌入式Linux软件研发工程师笔试卷（9月3日）

试卷开放时间(北京时间)：2020年09月03日 19:00 - 2020年09月03日 21:00（其余时段均不可作答）

考试时长：120分钟

试卷总分：120

考试题型：单选 15 道，不定项 5 道，编程 3 道

 嵌入式Linux系统开发

单选：

- 1、反转链表的代码补全（嵌入式笔试链表类题目必考，比如双向链表的插入删除。字符串翻转考察的也不少）
- 2、二叉平衡树（AVL）树，插入30，重新平衡后，某个数的左右孩子节点是多少。
- 3、数据查询指令是哪一个？
- 4、如果IP是212.212.77.55，子网掩码是255.255.252.0，那么它在自己组的子网广播可以达到的地址是哪个？
- 5、一段php脚本运行结果是哪一个？
- 6、一段代码的时间复杂度和空间复杂度是多少？
- 7、给出一个二叉树的遍历方式（前、中、后）。
- 8、一段C++代码，判断一段程序是内存泄漏、编译出错还是运行出错？
- 9、用户线程和内核线程的区别和联系。
- 10、简单逻辑题。
- 11、linux中让磁盘大小从小到大排列的命令是？
- 12、某算法的时间复杂度是多少？
- 13、一段java代码，让你判断while循环的时间复杂度？
- 14、一段伪代码的同步和互斥，以及PV操作补全代码
- 15、排序算法会一个即可完成（冒泡，选择，快排，二分）

不定项选择题：

- 1、while循环的时间复杂度判断。
- 2、ping、tcpdump、strace、wireshark工具分别是用来干啥的？
- 3、C++语言中static变量的作用域是什么？
- 4、C++中：类重写、纯虚函数、虚析构函数的编译错误和内存泄漏问题该如何？

## 5、一段C++代码的运行结果。

编程题：

最后的编程题，三道都没有看懂，好像全是动态规划，没思路。

总结：

百度考察的范围比较广，数据结构考察的比较多，C++考察的也比较多，C语言反而少一些，所以大家要好好准备。

以上题目没有列出选项，大家主要关注百度考察的知识点，自己总结一下，把知识点归类。不同企业的嵌入式岗考察是有技术偏向性的，大家要针对性准备。

## 6、汇顶笔试题

笔试时间：2020.08.24，20：00-21：00

提醒：很多大公司秋招在七八月份就要投递简历，九月已经晚了。

通用部分

- |   |          |
|---|----------|
| 1 | 单选10道    |
| 2 | 不定项选择题2道 |
| 3 | 填空题2道    |
| 4 | 开放题目1道   |

专业A类

- |   |          |
|---|----------|
| 1 | 单选10道    |
| 2 | 不定项选择题3道 |
| 3 | 填空题3道    |
| 4 | 程序设计题1道  |

专业B类

- |   |          |
|---|----------|
| 1 | 单选10道    |
| 2 | 不定项选择题3道 |
| 3 | 填空题3道    |
| 4 | 问答题2道    |



### 汇顶科技2021届校园招聘现场应用工程师A卷

试卷开放时间(北京时间)：2020年08月24日 20:00 - 2020年08月24日 21:00 (其余时段均不可作答)

考试时长：60分钟

试卷总分：158

考试题型：通用部分-单选题 10 道，通用部分-不定项选择题 2 道，通用部分-填空题 2 道，通用部分-开放题 1 道，专业A类-单选题 10 道，专业A类-不定项选择题 3 道，专业A类-填空题 3 道，专业A类-程序设计题 1 道，专业B类-单选题 10 道，专业B类-不定项选择题 3 道，专业B类-填空题 3 道，专业B类-问答题 2 道

嵌入式Linux系统开发

投的嵌入式软件岗位，给的题目是现场应用工程师（音频）考卷。

主要考察硬件，单片机，C语言三方面。

1、单片机程序存在哪里（有flash的情况下）：D

- 1 A ROM
- 2 B RAM
- 3 C 寄存器
- 4 D flash

2、波特率的单位是什么？

答案：bps

3、读写寄存器快还是flash快？

答案：寄存器

4、中断返回地址是谁的地址？

5、简单逻辑题。

6、典型IO口电路有哪三个寄存器？

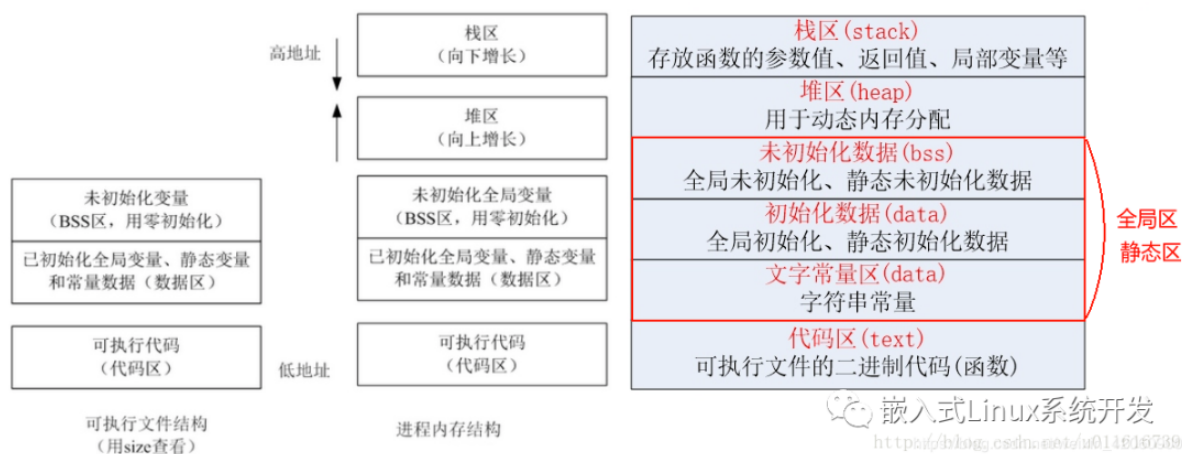
答案：状态寄存器、控制寄存器、数据寄存器。

7、三态门是什么？

答案：高电平、低电平、高阻态。

8、内存四区中各种变量存在哪里？

答案：



9、数组初始化方式

- 1 1. 直接给每个元素赋值
- 2 `int array[4] = {1,2,3,4};`
- 3 2. 给一部分赋值, 后面的全部为0
- 4 `int array[4] = {1,2};`
- 5 3. 由赋值参数个数决定数组的个数
- 6 `int array[] = {1,2};`
- 7 4. 数组元素的值是随机分配的
- 8 `int array[4] = {};`

10、一段程序考察i++与++i的区别。

11、一段代码考察运算符优先级

| 优先级 | 运算符                                        | 结合性  |
|-----|--------------------------------------------|------|
| 1   | () [] .                                    | 从左到右 |
| 2   | ! + (正) - (负) ~ ++ --                      | 从右向左 |
| 3   | * / %                                      | 从左到右 |
| 4   | + (加) - (减)                                | 从左到右 |
| 5   | << >> >>>                                  | 从左到右 |
| 6   | < <= > >= instanceof                       | 从左到右 |
| 7   | == !=                                      | 从左到右 |
| 8   | & (按位与)                                    | 从左到右 |
| 9   | ^                                          | 从左到右 |
| 10  |                                            | 从左到右 |
| 11  | &&                                         | 从左到右 |
| 12  |                                            | 从左到右 |
| 13  | ?:                                         | 从右向左 |
| 14  | = += -= *= /= %<br>= &=  = ^= ~< <= >> >>= | 从右向左 |

## 12、指针变量

答案：存放地址的变量称为指针变量。指针变量是一种特殊的变量，它不同于一般的变量，一般变量存放的是数据本身，而指针变量存放的是数据的地址。

## 13、队列和栈的区别（经典问题）

## 14、考察const作用，以下两种表达方式有什么区别？

```
1 int const * a
2 const int * a
```

## 15、sizeof 数组的计算。

答案：这种主要考察内存。比如sizeof或者strlen。

## 16、编程题

判断是否是回字符串（首尾相同）

```
1 输入abcba 返回true
2 输入abc 返回false
```

## 7、远景能源笔试题

笔试时间：2020.09.09

岗位：嵌入式软件工程师

题型：

单选20道，40分

不定项选择题10道，20分

编程题2道，40分

单选：

1、长度最大为150的队列满二叉树，采用广度优先遍历时，能够容纳的二叉树最大的深度为？用公式算节点

2、n个元素的有序单链表，插入一个新节点，仍然保持有序，时间复杂度为？

3、进程与线程的区别。

4、存储器速度排名：U盘、硬盘、CD-ROM、RAM？

5、内部异常分为故障（fault）、陷阱（trap）、终止（abort），他们的区别是什么？

6、HTTP1.0中，状态码302表示？答案：D

- |   |         |
|---|---------|
| 1 | A.成功    |
| 2 | B.内部错误  |
| 3 | C.网页未找到 |
| 4 | D.文件被转移 |

7、C++创建对象时，对象的内存和指向对象的指针放在哪里？答案：B

- |   |          |
|---|----------|
| 1 | A.栈区，堆区  |
| 2 | B.堆区，栈区  |
| 3 | C.全局区，栈区 |
| 4 | D.常量区，堆区 |

解析：自己创建的，在堆。指针是自动的，在栈。

8、ABCDE类地址，B类IP地址的范围是多少？

答案：128.0.0.0到191.255.255.255

9、cache的作用是什么？

答案：高速缓冲存储器，一种特殊的存储器子系统，其中复制了频繁使用的数据以利于快速访问。

10、顺序存储的线性表访问节点和增加节点的时间复杂度为？答案：C

- |   |                  |
|---|------------------|
| 1 | A. $O(n)$ $O(n)$ |
| 2 | B. $O(n)$ $O(1)$ |
| 3 | C. $O(1)$ $O(n)$ |
| 4 | D. $O(1)$ $O(1)$ |

11、考察数据库的查找命令。

12、数据库事务的四个特性是？

答案：原子性 Atomicity、一致性 Consistency、隔离型 Isolation、持久性 Durability

13、冯诺依曼工作方式的基本特点？答案：B

- |   |                |
|---|----------------|
| 1 | A.多指令流单数据流     |
| 2 | B.按地址访问并顺序执行指令 |
| 3 | C.堆栈操作         |
| 4 | D.存储器按内部选择地址   |

冯诺依曼体系结构的特点是：

(1)计算机处理的数据和指令一律用二进制数表示

(2)顺序执行程序：计算机运行过程中，把要执行的程序和处理的数据首先存入主存储器（内存），计算机执行程序时，将自动地并按顺序从主存储器中取出指令一条一条地执行，这一概念称作顺序执行程序。

(3)计算机硬件由运算器、控制器、存储器、输入设备和输出设备五大部分组成。

14、假设某一虚拟存储系统采用先进先出（FIFO）页面淘汰算法，有一个进程在内存中占3页（开始时内存为空），当访问如下页面序号（1,2,3,1,2,4,2,3,5,3,4,5）后，会产生几次缺页？

- 1 A.4
- 2 B.5
- 3 C.6
- 4 D.7

解析：

```
1 1 1 （一次）
2 2 1 2 （一次）
3 3 1 2 3 （一次）
4 1 1 2 3
5 2 1 2 3
6 4 2 3 4 （一次）
7 2 2 3 4
8 3 2 3 4
9 5 3 4 5 （一次）
10 3 3 4 5
11 4 3 4 5
12 5 3 4 5
13 B 5次
14
15 1,2,3 miss 3次 *** 为 1,2,3
16 1, 2 hit
17 4 miss 1次，此时1出4进 此时***为2,3,4
18 2,3 hit
19 5 miss, 2出5进，***为3, 4,5
20 3,4,5 hit
21
22 总计5次
```

15、下面哪个不是线性表？ 答案：D

- 1 A.循环链表
- 2 B.队列
- 3 C.栈
- 4 D.关联数组
- 5 E.空字符串数组
- 6 F.双向链表

关联数组没有顺序，不是线性表

16、若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是？ 答案：C

- 1 A.257
- 2 B.258
- 3 C.384
- 4 D.385

解析：首先理解什么是完全二叉树，完全二叉树从根结点到倒数第二层满足完美二叉树，最后一层可以不完全填充，其叶子结点都靠左对齐。

有768个结点，则树的高度为10。前9层结点数为511，则第10层有 $768-511=257$ 个叶子结点。257个叶子结点在第9层有129个根结点。则第九层剩下 $256-129=127$ 个叶子结点。所以共有 $127+257=384$

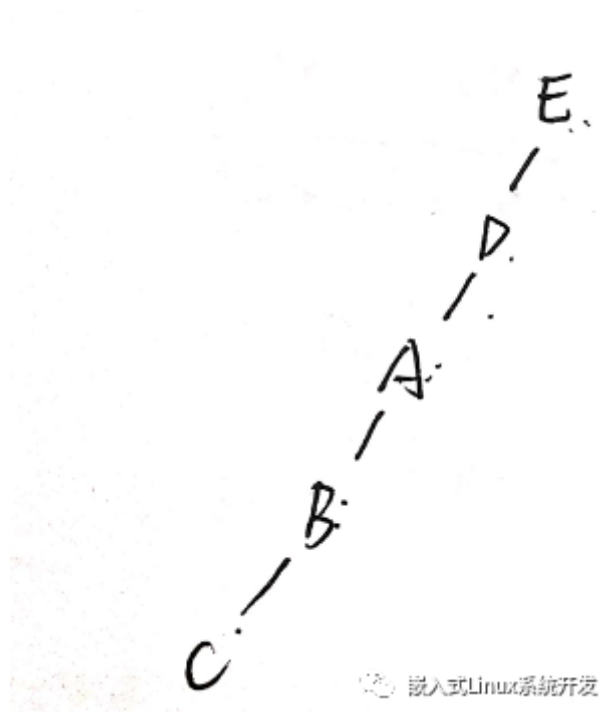
17、交换机工作在OSI七层的哪一层？ 答案：B

- 1 A. 一层
- 2 B. 二层
- 3 C. 三层
- 4 D. 三层以上

18、某二叉树的中序遍历序列为CBADE，后序遍历序列为CBADE，则前序遍历序列为？

- 1 A. EDABC
- 2 B. CBEDA
- 3 C. CBADE
- 4 D. EDCBA

解析：



19、一段C++代码的运行结果

20、面向链接的传输协议是？

答案：TCP（传输控制协议）

不定项选择题：

1、ArrayList和LinkedList的区别是什么？

2、查找哈希表，解决冲突的方法包括？ 答案：AD

- 1 A. 链地址法
- 2 B. 除留余数法
- 3 C. 直接地址法
- 4 D. 线性探测再散列法

3、构造散列 ( hash ) 的方法？

4、堆的形状是？

答案：完全二叉树

5、linux中硬链接的作用是为了什么？

答案：硬链接是指通过索引节点来进行链接。在Linux的文件系统中，保存在磁盘分区中的文件不管是什么类型都会给它分配一个编号，这个编号被称为索引节点编号(Inode Index)或者Inode，它是文件或目录在一个文件系统中的唯一标识，文件的实际数据放置在数据区域 ( data block )，它存储着文件重要参数信息，也就是元数据 ( metadata )，比如创建时间、修改时间、文件大小、属主、归属的用户组、读写权限、数据所在block号等。

6、linux下创建文件的命令：touch,vi,vim

linux下创建文件夹的命令:mkdir

7、ISO七层网络中每一层的功能是什么？

8、死锁出现的条件是什么？

答案：有四个必要条件。1、互斥，2、占有且等待，3、不可抢占，4、循环等待。具体解释请百度。

9、java反射机制是什么？

答案：JAVA反射机制是在运行状态中，对于任意一个实体类，都能够知道这个类的所有属性和方法;对于任意一个对象，都能够调用它的任意方法和属性;这种动态获取信息以及动态调用对象方法的功能称为java语言的反射机制。

10、AVL和hash的考察。

编程题：

1、闭合符号检测。就是检测 ( ) 【】 {} "" 是不是一对。

case通过率66.7%，不知道为什么。

```
1 #include<stdio.h>
2 #include<map>
3 #include<iostream>
4 #include<stack>
5 #include <algorithm>
6 using namespace std;
7 int main()
8 {
9 string s;
10 getline(cin,s);
11 int len=s.size();
12 map<char,char> m;
13 m['(']=')';
14 m['{']='}';
15 m['[']=']';
16 stack <char> a;
17 int flag=1;
18 for(int i=0;i<len;i++)
19 {
20 if(s[i]=='(' || s[i]=='{' || s[i]=='[')
21 a.push(s[i]);
```

```

22 else if(s[i]=='(' || s[i]=='{' || s[i]==']')
23 {
24 if(a.size() && m[a.top()]==s[i])
25 a.pop();
26 else
27 {
28 flag=0;
29 break;
30 }
31 }
32 }
33 if(flag && a.size() == 0)
34 printf("yes\n");
35 else
36 printf("no\n");
37
38
39 return 0;
40 }

```

2、输入一个数，判断它的阶乘后面有几个0。

输入3，即3！输出0

输入5，即5！输出1

输入1000，即1000！输出249

```

1 #include<stdio.h>
2
3 int countFive(int n)
4 {
5 int i, ans = 0;
6 for(i = 5; i <= n; i++)
7 {
8 int j = i;
9 while(j%5==0)
10 {
11 ans++;
12 j /= 5;
13 }
14 }
15 return ans;
16 }
17
18 int main()
19 {
20 int n;
21 scanf("%d",&n);
22 printf("%d\n",countFive(n));
23 return 0;
24 }

```

## 8、科大讯飞笔试题

笔试时间：2020.09.12,19:00—21:00

岗位：嵌入式软件开发

题型：四道编程题，4\*10=40分

- 1、定义一个n\*m矩阵，找到两个不在同一行同一列的数字，使得成绩最大。
- 2、输入一个数N，求数的二进制中1的个数（已AC）

```
1 //写一个函数返回参数的二进制中1的个数
2 #include<stdio.h>
3 #include<stdlib.h>
4 int count_one_bits(unsigned int value)
5 {
6 int count = 0;
7 while (value != 0)
8 {
9 if (value % 2 == 1)
10 {
11 count++;
12 }
13 value = value >>1;
14 }
15 return count;
16 }
17 int main()
18 {
19 int num;
20 int ret;
21 printf("请输入一个大于0的数\n");
22 scanf("%d", &num);
23 ret=count_one_bits(num);
24 printf("%d", ret);
25 system("pause");
26 return 0;
27 }
```

- 3、数组的排序（时间复杂度最小），应该是让你用冒泡或者选择排序。

题型是给了一种排序算法的中间过程，让你认出是哪一种排序，然后写代码。

```
1 #include<stdlib.h>
2 #include<stdio.h>
3 #include<string.h>
4
5 void sortA1(int a[], int length){
6 int i, j, temp;
7 for(i = 0; i < length; ++i){
8 for(j = i + 1; j < length; ++j){
9 if(a[j] < a[i]){ //如果后一个元素小于前一个元素则交换
10 temp = a[i];
11 a[i] = a[j];
12 a[j] = temp;
13 }
14 }
15 }
16 }
```

```

13 }
14 }
15 }
16 }
17
18 void printA1(int a[], int length){
19
20 int i;
21 for(i = 0; i < length; ++i){
22 printf("%d,", a[i]);
23 }
24 printf("\n");
25 }
26
27 void sortA2(int a[], int length){
28 int i, j, temp;
29 for(i = 0; i < length; ++i){
30 for(j = length - 1; j > i; --j){
31 if(a[j] > a[j - 1]){
32 temp = a[j];
33 a[j] = a[j - 1];
34 a[j - 1] = temp;
35 }
36 }
37 }
38 }
39
40 int main(){
41
42 int length = 0;
43 int a[] = {12, 43, 8, 50, 100, 52, 0};
44 length = sizeof(a) / sizeof(a[0]);
45 printf("排序前\n");
46 printA1(a, length);
47 sortA1(a, length);
48 printf("选择排序后\n");
49 printA1(a, length);
50 sortA2(a, length);
51 printf("冒泡排序后\n");
52 printA1(a, length);
53 system("pause");
54 }

```

4、字符串左旋 ( case 20% , 不知道是哪里没考虑到 , 读者可以尝试 )

输入 1234abcd , 左旋3

输出 4abcd123

```

1 void swap(char *start, char *end)
2 {
3 while (start < end)
4 {
5 *start ^= *end;
6 *end ^= *start;
7 *start ^= *end;
8 start++, end--;
9 }

```

```

10 }
11
12 void reverse_left_2(char *str, int n, int len)
13 {
14 char *mid = NULL; // 定义一个指针，将指向左旋分段点
15
16 n %= len; // 判断左旋的有效次数
17 mid = str + n - 1; // 指向分段点的最后一个字符
18 swap(str, mid); // 逆置前一段字符串
19 swap(mid + 1, str + len - 1); // 逆置后一段字符串
20 swap(str, str + len - 1); // 整个字符串逆置
21 }
22
23 int main()
24 {
25 char str[] = "abcd1234";
26 int n = 0, len = strlen(str);
27
28 printf("please enter->");
29 scanf("%d", &n); // 输入左旋的次数
30 printf("before reverse_left string is :%s\n", str);
31 reverse_left_2(str, n, len);
32 printf("reverse_left string is :%s\n", str);
33
34 system("pause");
35 return 0;
36 }

```

## 9、小马智行笔试题

笔试时间：2020.09.20，15：00——16：30

岗位：嵌入式软件工程师

题型：

单选7道

不定项选择题2道

简答1道（10分）

编程2道（60分）

单选：

1、可以作为中断使用的安全同步方式是：

- |   |   |           |     |
|---|---|-----------|-----|
| 1 | A | spinlock  | 自旋锁 |
| 2 | B | mutex     | 互斥量 |
| 3 | C | semaphore | 信号量 |
| 4 | D | 全局变量      |     |

2、linux中改变文件属性的命令：chmod

3、linux中查找文件中匹配字符串的命令：grep

4、下列哪一种协议是靠设备地址的不同区分的：IIC

5、全局未初始化变量存在哪里：BSS段

6、代码题。

```
1 #include<stdio.h>
2 void change(int *,int);
3
4 int main(){
5 int a=25,b=50;
6 change(&a,b);
7 printf("a:%d,b %d",a,b);
8 return 0;
9 }
10
11 void change(int *x,int y){
12 *x=100;
13 y=200;
14 }
```

答案：100 50

提示：如果把函数写到main之前，那么就不需要声明。而且一般我们都是把main写在文件最下面。

解析：这里只有传的指针才会改变，b没有返回值。

7、代码题。

```
1 #include<stdio.h>
2
3 struct ponyai{
4 char a;
5 int b;
6 short c;
7 void *d;
8 };
9
10 int main(){
11 struct ponyai hw;
12 printf("%d\n",sizeof(hw));
13 return 0;
14 }
15 64位linux下:24, 内存对齐8*3=24, (void *) =8
16
17 32位linux下:16, 8*2, (void *) =4
```

不同平台数据类型占用字节

| 数据类型        | 32位   | 64位   | 备注                                |
|-------------|-------|-------|-----------------------------------|
| char        | 1     | 1     |                                   |
| short       | 2     | 2     |                                   |
| int         | 4     | 4     |                                   |
| long        | 4     | 8     | 32位与64位不同                         |
| float       | 4     | 4     |                                   |
| char*       | 4     | 8     | 其他指针类型如long *, int * 也是如此         |
| long long   | 8     | 8     |                                   |
| double      | 8     | 8     |                                   |
| long double | 10/12 | 10/16 | 有效位10字节。32位为了对齐实际分配12字节；64位分配16字节 |

8、答案：7

```

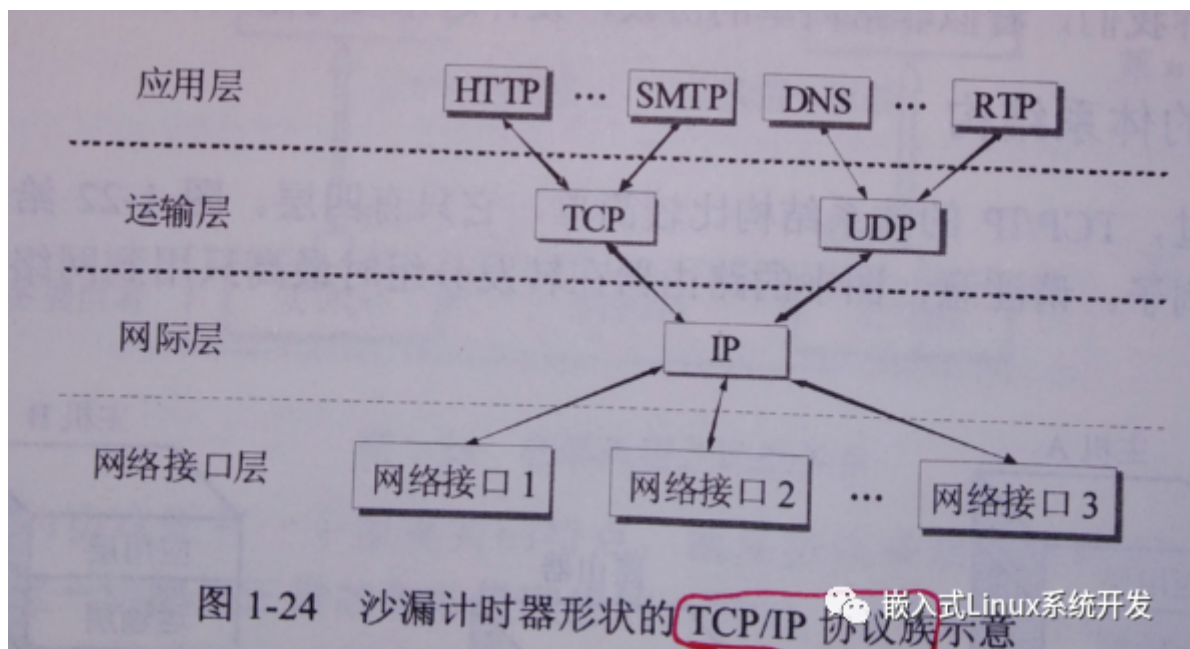
1 #include<stdio.h>
2
3 int main(){
4 unsigned char a=2,b=5;
5 printf("%d", (int)a^b);
6 return 0;
7 }
```

不定项选择题：

1、OSI网络模型中，下列不属于应用层协议的是？

| ISO/OSI 模型 |                              | TCP/IP 协议              |                           |                       |                      | TCP/IP 模型 |
|------------|------------------------------|------------------------|---------------------------|-----------------------|----------------------|-----------|
| 应用层        | 文件传输<br>协议<br>(FTP)          | 远程登录<br>协议<br>(Telnet) | 电子邮件<br>协议<br>(SMTP)      | 网络文件服<br>务协议<br>(NFS) | 网络管理<br>协议<br>(SNMP) | 应用层       |
| 表示层        |                              |                        |                           |                       |                      |           |
| 会话层        |                              |                        |                           |                       |                      |           |
| 传输层        | TCP                      UDP |                        |                           |                       |                      | 传输层       |
| 网络层        | IP                           | ICMP                   | ARP      RARP             |                       |                      | 网际层       |
| 数据链路层      | Ethernet<br>IEEE 802.3       | FDDI                   | Token-Ring/<br>IEEE 802.5 | ARCnet                | PPP/SLIP             | 网络接口层     |
| 物理层        |                              |                        |                           |                       |                      | 硬件层       |

TCP/IP 模型与 OSI 模型的对比



2、进程间通讯：

(1) 管道/无名管道 (2) 信号 (3) 共享内存 (4) 消息队列 (5) 信号量 (6) socket

注意：临界区则是一种概念，指的是访问公共资源的程序片段，并不是一种通信方式。

线程通讯（锁）：

(1) 信号量 (2) 读写锁 (3) 条件变量 (4) 互斥锁 (5) 自旋锁

简答题：

linux内存碎片化的定义，可能的原因以及可以避免或者缓解的方式？

定义：内部碎片与外部碎片

在页式虚拟存储系统中，用户作业的地址空间被划分成若干大小相等的页面，存储空间也分成等页大小相等的物理块，但一般情况下，作业的大小不可能都是物理块大小的整数倍，因此作业的最后一页中仍有部分空间被浪费掉了。由此可知，页式虚拟存储系统中存在内碎片。

在段式虚拟存储系统中，作业的地址空间由若干个逻辑分段组成，每段分配一个连续的内存区，但各段之间不要求连续，其内存的分配方式类似于动态分区分配。由此可知，段式虚拟存储系统中存在外碎片。

换一种问法也是一样的：

在内存管理中，“内零头”和“外零头”个指的是什么？在固定式分区分配，可变式分区分配，页式虚拟存储系统，段式虚拟存储系统中，各会存在何种碎片？为什么？

解答：

在存储管理中内碎片是指分配给作业的存储空间中未被利用的部分。

在固定式分区分配中，为将一个用户作业装入内存，内存分配程序从系统分区表中找出一个能满足作业要求的空闲分区分配给作业，由于一个作业的大小并不一定与分区大小相等，因此，分区中有一部分存储空间浪费掉了。由此可知，固定式分区分配中存在内碎片。

外碎片是指系统中无法利用的小存储块。

在可变式分区分配中，为把一个作业装入内存，应按照一定的分配算法从系统中找出一个能满足作业需求的空闲分区分配给作业，如果这个空闲分区的容量比作业申请的空间容量要大，则将该分区一分为二，一部分分配给作业，剩下的部分仍然留作系统的空闲分区。由此可知，可变式分区分配中存在外碎片。

简言之：

在固定分区管理算法中, 分给程序的内存空间往往大于程序所需的空间, 这剩余部分的空间不能被其他程序所用, 这就是“内部碎片”

随着存储区的分配和释放过程的进行, 在各个被分配出去的分区之间会存在很多的小空闲区, 暂时不能被利用, 这就是“外部碎片”.

**避免方式：**

- 1、依据可移动性组织页是防止物理内存碎片的一种可能方法.
- 2、虚拟可移动内存域避免内存碎片

编程题：

- 1、用C写一个function , 可以获取一个文件名的宏定义。

输入 filename.h

输出 FILE\_NAME\_H

```
1 #include <stdbool.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 char *GetFilenameDefine(char *filename, char *target_str, int
target_str_size) {
6 //获取头文件的宏定义字符串
7 }
8
9 int main() {
10 char target_string[1000] = {0};
11 char filename[1000] = {0};
12 scanf("%s", filename);
13 printf("%s\n", GetFilenameDefine(filename, target_string,
sizeof(target_string)));
14 }
```

大家可以自己实现。

- 2、比较字符串是否相等。

case 90% , 有什么我没考虑到的吗？

```
1 #include <stdbool.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 bool CompareString(char *str1, char *str2) {
6 // 请判断两个字符串是否相等
7 int ret;
8 ret = strcmp(str1, str2);
9 if(ret == 0){
10 return 0;
11 }
12 else{
13 return 1;
14 }
```

```

14 }
15
16 }
17
18 char str1[1000];
19 char str2[1000];
20 int main()
21 {
22 scanf("%s", str1);
23 scanf("%s", str2);
24 printf("%d\n", CompareString(str1, str2));
25 return 0;
26 }

```

## 10、思科笔试题

笔试时间：2020.09.07，19：00——21：00

岗位：嵌入式软件工程师

题型：数据结构4道，网络3道，操作系统3道，C++4道，Java4道，python4道，数据库4道，大数据4道，编程题1道。



### 思科2021校园招聘笔试（软件岗）

试卷开放时间(北京时间)：2020年09月07日 19:00 - 2020年09月07日 21:00（其余时段均不可作答）

考试时长：120分钟

试卷总分：110

考试题型：数据结构 4 道，网络 3 道，操作系统 3 道，C++ 4 道，Java 4 道，Python 4 道，数据库 4 道，大数据 4 道，编程题 1 道

嵌入式Linux系统开发

题目全是英文题目，所以小题目没记下来，只把编程题记了下来，大家一起做一做。

思科给的是软件岗位综合卷，考察范围很广泛，大家投递简历要注意。

题目：

## [编程题|20分] Jumping

时间限制：C/C++ 1秒，其他语言 2秒

空间限制：C/C++ 32768K，其他语言 65536K

64bit IO Format: %lld

本题可使用本地IDE编码，不能使用本地已有代码，无跳出限制，编码后请点击“保存并调试”按钮进行代码提交。

### ■ 题目描述

Given an array of non-negative integers, you are initially positioned at the first index of the array.

Each element in the array represents your maximum jump length at that position.

Determine if you are able to reach the last index. [来源：嵌入式Linux系统开发](#)

#### 输入描述:

```
The array of non-negative integers.
```

#### 输出描述:

```
Result of whether could reach the last index.
```

**示例1** 输入输出示例仅供调试，后台判题数据一般不包含示例

输入

复制

```
[2,3,1,1,4]
```

输出

复制

```
true
```

 嵌入式Linux系统开发

**示例2** 输入输出示例仅供调试，后台判题数据一般不包含示例

输入

复制

```
[3,2,1,0,4]
```

输出

复制

```
false
```

说明

```
You will always arrive at index 3 no matter
what. Its maximum
jump length is 0, which makes it impossible
to reach the last index.
```

嵌入式Linux系统开发

## 11、商汤科技笔试题

笔试时间：2020.09.18，19：00——21：00

岗位：嵌入式软件工程师

题型：单选4道，不定项选择题2道，填空2道，编程2道。

单选：

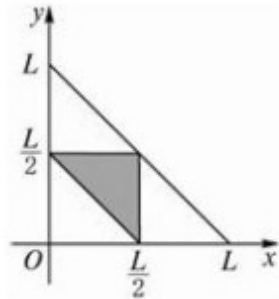
1、在一棵二叉树上第5层的结点数最多是：16

- 1 第1层1个  $2^0$
- 2 第2层2个  $2^1$
- 3 第3层4个  $2^2$
- 4 第n层  $2^{(n-1)}$
- 5 //就是2的层数-1次方.

2、一根长度为a的木棒折成三段,求能构成三角形的概率：1/4

解：设A=“3段构成三角形”，x、y分别表示其中两段的长度，则第3段的长度为L-x-y，则 $\Omega = \{(x, y) | 0 < x < L, 0 < y < L, 0 < x+y < L\}$ . 要使3段构成三角形，当且仅当任意2段之和大于第3段，即 $x+y > L-x-y \Rightarrow x+y > \frac{L}{2}$ ； $x+L-x-y > y \Rightarrow y < \frac{L}{2}$ ； $y+L-x-y > x \Rightarrow x < \frac{L}{2}$ .

故 $A = \{(x, y) | x+y > \frac{L}{2}, y < \frac{L}{2}, x < \frac{L}{2}\}$ .



由上图可知所求概率为

$$P(A) = \frac{A \text{ 的面积}}{\Omega \text{ 的面积}} = \frac{\frac{1}{2} \cdot (\frac{L}{2})^2}{\frac{L^2}{2}} = \frac{1}{4}.$$

嵌入式Linux系统开发

3、两堆硬币，分别为100和200枚，两人可从一堆取任意枚或从两堆取相同枚，不可以不取，取到最后一枚赢。怎么赢？（先手还是后手，后手从哪一堆取？）

4、权值为8.6.4.11的四个叶子节点构造一棵哈弗曼树，该树的带权路径长度为？

不定项选择题：

1、十进制小数55.125对应的二进制数是？

2、以下关于static说法正确的是？

回答：经典考察，另外还有const、volatile等关键字。

填空题：

1、此程序崩溃，为什么？

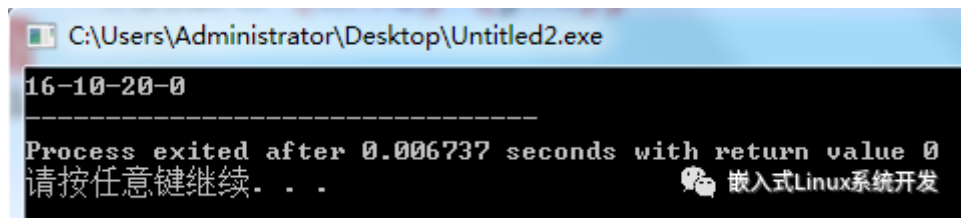
```
1 #include<stdio.h>
2
3 char *f()
4 {
5 char str[512];
6 sprintf(str,"AI helps world!");
7 return str;
8 }
9 int main(){
10 printf("%s",f()+3);
11 return 0;
12 }
```

2、求输出

```

1 #include<stdio.h>
2
3 int fun(int x,int y){
4 return (x>>2)*(y+4);
5 }
6 int main(){
7 int a[4];
8 int i = 0,value;
9 a[0]=1;
10 a[1]=18;
11 for(i=0;i<2;i++){
12 value = fun(a[i++],a[i++]);
13 a[i-1]=value>>1;
14 a[i-2]=value-4;
15 a[i]=value;
16 }
17 printf("%d-%d-%d-%d",a[0],a[1],a[2],a[3]);
18 return 0;
19 }

```



编程题：

1、给定一个多边形P，是一个 $(n,2)$ 的数组，表示按顺序每个顶点的坐标，和一个点A $(x,y)$ ，请计算点A到多边形P上的所有线段中，最短的举例的数值（点到多边形的最短距离理论）

2、有一个人在迷宫里，要从入口s抵达出口g，迷宫里的“x”表示墙壁，“.”表示道路，大写字母比如A表示门，小写字母比如“a”表示钥匙。求能不能走出来？（应该是考的广度优先搜索）

## 12、盛弘电气笔试题

笔试时间：2020.09.16，60分钟

宣讲会后直接笔试，若通过会有两轮面试，7-15天出结果。

题型：简答题8道，每题5分，共40分。编程题4道，每题15分，共60分。

公司介绍：

公司现阶段主要产品为充电桩，专注于电力电子技术控制电能，交直流变换。

薪资待遇：本科8-15K，研究生10-20K。

简答题：

1、如何使用GDB进行调试？

答案：大家可自行梳理调试流程。

## 2、Linux程序如何定位内存泄漏？

答案：如果在实际的调试过程中，怀疑某处发生了内存泄露，可以查看该进程的maps表，看进程的堆段或者mmap段的虚拟地址空间是否持续增加，如果是，说明很可能发生了内存泄露，如果mmap段虚拟地址空间持续增加，还可以看到各个段的虚拟地址空间的大小，从而可以确定是申请了多大的内存，对调试内存泄露类问题可以起到很好的定位作用。

## 3、简述一个Linux驱动程序的主要流程与功能。

```
1 1、 insmod module
2 2、 驱动module的初始化（初始化结束后即进入“潜伏”状态，直到有系统调用）
3 3、 当操作设备时，即有系统调用时，调用驱动module提供的各个服务函数
4 4、 rmod module
```

## 4、简述一个完整的中断响应过程。

答案：中断请求、中断排队、中断响应、中断处理和中断返回。

## 5、C语言编译过程中，volatile关键字和extern关键字分别在哪个阶段起作用？

答案：volatile是在编译阶段，extern在链接阶段。

## 6、关键字static的作用是什么？

答案：经典问题。

## 7、下面的代码输出是什么？如果要得到想要的结果，需要怎么修改？

```
1 char *p_str = "hello,world";
2 printf("len(p_str)=%d",sizeof(p_str));
```

答案：输出是4，应该要把sizeof改成strlen。

## 8、下面的代码输出是什么？为什么？

```
1 int main(){
2 int a[5]={1,2,3,4,5};
3 int *ptr=(int *)(&a+1);
4 printf("%d,%d",*(a+1),*(ptr-1));
5 }
```

答案：2 5

《程序员面试宝典》经典题目

编程题：

## 1、请写一个简单的代码，实现数学中的阶乘（n）计算。

```

1 #include <stdio.h>
2 int main()
3 {
4 int i,n;
5 double sum=1;
6 scanf("%d",&n);
7 for(i=1;i<=n;i++){
8 sum=sum*i;
9 }
10 printf("%d!=%lf",n,sum);
11 printf("\n");
12 return 0;
13 }

```

2、请写一个单例模式 ( C++ )。

3、是否对Linux下的编译有了解，如果有请写一个编译命令（编译文件为：Hello.cpp），是否对makefile有了解，如果有请写一个简单的Makefile脚本。

4、写一个代码，产生一个线程。

答案：主要是pthread\_create(&tid, NULL, thread, NULL);

### 13、乐鑫科技笔试题

笔试时间：2020.09.09，10：00-11：30

岗位：嵌入式软件工程师

题型：单选题20道，40分。编程题2道，60分。

乐鑫科技最出名的产品就是ESP8266 WIFI模块和ESP32。

单选题：

- 1、算术右移指令执行的操作是？符号位会变化吗？
- 2、以太网交换机的一个端口可以看做一个？
- 3、长度为n的单链表，链接在长度为m的单链表之后的算法时间复杂度为？
- 4、定长为16位字长的字，采用2的补码形式表示时，一个字能表示的整数范围是？
- 5、定长32位的字能表示的整数范围是？
- 6、临界资源的访问规则有哪些？
- 7、一段C语言程序中do while循环执行结果
- 8、VLAN是什么意思？
- 9、以太网MAC地址的基础知识。
- 10、CPU组成部分有哪几个？
- 11、系统“抖动”的原因是？
- 12、可变式分区方案分别是什么？
- 13、n个元素的单链表，查询一个值为x的元素，时间复杂度为多少？
- 14、进程有哪三种状态，如何切换？

- 15、考察C语言数组的下标。
- 16、内存管理方法有哪些？
- 17、考察线性表的定义。
- 18、考察邻接表的定义。
- 19、在七层网络中，中继器在哪一层发挥作用？

编程题：

- 1、第一题没看懂
- 2、假设有n个核酸样本，其中m个病毒成阳性。一开始进行分组，就是武汉采用的分组检测。一组是阴性那么这一组就不再检测。一组是阳性，则再把这一组再次分组。

求病毒检测最小组合数

## 14、深信服笔试题

笔试时间：2020.08.25，19：00---21：00。

岗位：嵌入式软件工程师。

题型：

5个不定项选择题，16分

5个填空题，19分

2道编程题，65分

不定项选择题：

- 1、下面哪个可以判断是2的倍数？

- |   |   |               |
|---|---|---------------|
| 1 | A | $x \& (x+1)$  |
| 2 | B | $x \& (-x+1)$ |
| 3 | C | $x \& (-x)$   |
| 4 | D | $x \& (x-1)$  |

- 2、双向链表删除P节点该如何操作？

- 3、考察栈和队列。

答案：栈先进后出，队列先进先出。

- 4、判断一个程序时间复杂度是多少？

- 5、简单逻辑题

填空题：

- 1、

```
1 前序: 1 2 3 4 5 6 7 8
2 中序: 3 2 5 4 1 7 6 8
3 后序: ?
```

2、在X86\_64环境下，请问printf输出的结果是？

```
1 typedef union{
2 long i;
3 char j[10];
4 int k;
5 }DATE;
6 struct data{
7 int m;
8 DATE n;
9 double l;
10 }test;
11 DATE max;
12 printf("%d",sizeof(struct data)+sizeof(max));
```

3、简单逻辑题。

4、求输出。

```
1 void init(int a,int *c)
2 {
3 *c=3;
4 a=2;
5 *c+=a;
6 a+=*c;
7 }
8 void main(void)
9 {
10 int a=1;
11 int(a,&a);
12 printf("%d \n",a);
13 }
```

5、请问printf的输出结果是？

```
1 int array[5]={7,8,3,2,6};
2 int *p=(int*)(array+1);
3 int *m=(int*)&array+1;
4 printf("%d %d %d\n",*(array+1),*(p+1),*(m+1));
```

编程题：

第一题

## ■ 题目描述

森林中有N棵树，它们都有灵性会相互攀比谁比较高，你是一个魔法师，  
，当你每次用魔法棒指定一棵树，这棵树的高度就不会变化，但其它树的高度会加1，  
只有当所有树都一样高的时候，它们就不会再生长了。  
你不希望他们无休止得长下去。问你至少操作多少次可以让所有的树都相等呢？  
（比如你第一次指定第一棵树，下次指定了第二棵树，此时第一棵树还是会高度加一，每次指定只影响本轮的树是否生长）

🔗 嵌入式Linux系统开发

例如：

3棵树高度是

1 2 3

第一次指定 第三棵不变，之后高度变成

2 3 3

第二次指定第二棵不变，之后高度变成

3 3 4

第三次指定第三棵不变，之后高度变成

4 4 4

然后高度就永远是4 4 4了。

答案就是3。即你至少需要使用3次魔法棒才能让所有树都相等

🔗 嵌入式Linux系统开发

### 输入描述:

输入第一行给出一个整数n，  
第二行给出n个整数，空格隔开  
1 <= n <= 1000000  
1 <= 每棵树高 <=10000000

### 输出描述:

输出为一个整数表示最少使用魔法棒的次数

**示例1** 输入输出示例仅供调试，后台判题数据一般不包含示例

输入

复制

3  
1 2 3

🔗 嵌入式Linux系统开发

输出

复制

3

嵌入式Linux系统开发

## 第二题

### ■ 题目描述

给出一个数字组成的字符串S及n个数字变换(a,b)，  
每个变换(a,b)是将字符串里面的数字a变换成数字b，  
如果字符串里面不存在数字a，则不做变换。  
输出全部操作完后的字符串  
注：请注意算法执行效率。

#### 输入描述:

第一行一串0-9组成的字符串 s  
第二行一个整数 n  
第三行开始的n行每行是两个数a b代表数字a变成b

数据规模：

1 <= 字符串s长度 <= 1000000

1 <= n <= 1000000

嵌入式Linux系统开发

#### 输出描述:

变换后的字符串

**示例1** 输入输出示例仅供调试，后台判题数据一般不包含示例

输入

复制

0285289430826  
2  
0 2  
2 3

输出

复制

3385389433836

嵌入式Linux系统开发

## 15、美团笔试题

笔试时间：2020.09.06，10:00-12:00。

岗位：嵌入式软件（技术综合-硬件开发方向）

题型：四道编程题（80分），一道选择题（20分）。

### 编程题

#### 第一题

## 美团 2021校招 技术综合-硬件开发方向 在线考试

编程题 | 20.0分

1/4

### 争夺地盘

时间限制：1000MS

内存限制：65536KB

题目描述：

A国和B国正在打仗，他们的目的是 $n$ 块土地。现在，A国和B国暂时休战，为了能合理分配这一些土地，AB国开始协商。

A国希望得到这 $n$ 块土地中的 $p$ 块土地，B国希望得到这 $n$ 块土地中的 $q$ 块土地。每个国家都将自己希望得到的土地编号告诉了小团和小美——两位战争调和人。

你需要帮小团和小美计算，有多少块土地是只有A国想要的，有多少块土地是只有B国想要的，有多少块土地是两个国家都想要的。

 嵌入式Linux系统开发

#### 输入描述

输入第一行包含三个整数 $n, p, q$ ，含义如题意所示

接下来一行 $p$ 个整数，空格隔开，第 $i$ 个整数代表A国需要的土地编号 $a_i$

接下来一行 $q$ 个整数，空格隔开，第 $i$ 个整数代表B国需要的土地编号 $b_i$

土地编号从1开始， $n$ 结束。

$p, q \leq n \leq 100000$ ， $1 \leq a_i, b_i \leq n$ ，数据保证所有的 $a_i$ 互不相同， $b_i$ 互不相同。

#### 输出描述

输出包含三个数，只有A国想要的土地数，只有B国想要的土地数，两个国家都想要的土地数。

 嵌入式Linux系统开发

#### 样例输入

```
5 3 3
1 2 3
3 4 5
```

#### 样例输出

```
2 2 1
```

#### 提示

1,2号土地只有A想要，4,5号土地只有B想要，3号土地都想要

#### 规则

请尽量在全场考试结束10分钟前调试程序，否则由于密集排队提交，可能查询不到编译结果

点击“调试”亦可保存代码

编程题可以使用本地编译器，此页面不记录跳出次数

 嵌入式Linux系统开发

## 第二题

编程题 | 20.0分

2/4

### 修改大小写

时间限制：3000MS

内存限制：589824KB

题目描述：

在小美的国家，任何一篇由英文字母组成的文章中，如果大小写字母的数量不相同会被认为文章不优雅。

现在，小美写了一篇文章，并且交给小团来修改。小美希望文章中的大小写字母数量相同，所以她想让小团帮她把某些小写字母改成对应的大写字母，或者把某些大写字母改成对应的小写字母，使得文章变得优雅。

小美给出的文章一定是由偶数长度组成的，她想知道最少修改多少个字母，才能让文章优雅。

 嵌入式Linux系统开发

#### 输入描述

输入包含一个字符串S，仅包含大写字母和小写字母  
 $|S| \leq 100000, |S| \bmod 2 = 0$

#### 输出描述

输出包含一个整数，即最少需要修改的字符数。

#### 样例输入

AAAb

#### 样例输出

1

#### 提示

修改为AaAb即可，也有其他两种修改方法：aAAb，AAab

嵌入式Linux系统开发

### 第三题

编程题 | 20.0分

3/4

## 式子求值

时间限制：3000MS

内存限制：655360KB

题目描述：

小团有一个序列a，下标从1开始直到n，分别为 $a_1, a_2, \dots, a_n$ 。现在，小团定义了以下式子：

$$b_i = a_i \oplus \bigoplus_{j=1}^n (i \bmod j)$$

现在小团想让小美回答  $\bigoplus_{i=1}^n (b_i)$  的值

其中， $\oplus$ 代表异或运算，请你帮助小美。

小提示：

$$b_i = a_i \oplus \bigoplus_{j=1}^n (i \bmod j) = a_i \oplus (i \bmod 1) \oplus (i \bmod 2) \oplus (i \bmod 3) \dots \oplus$$

样例解释

$$b_1 = a_1 \oplus (1 \bmod 1) \oplus (1 \bmod 2) = 2$$

$$b_2 = a_2 \oplus (2 \bmod 1) \oplus (2 \bmod 2) = 2$$

$$\bigoplus_{i=1}^n (b_i) = b_1 \oplus b_2 = 0$$

嵌入式Linux系统开发

#### 输入描述

输入第一行包含一个整数 $n$ ，表示序列 $a$ 的长度。  
接下来一行 $n$ 个数，空格隔开，第 $i$ 个数表示 $a_i$   
 $n \leq 100000, 0 \leq a_i \leq n$

#### 输出描述

输出包含一个数，即题干中所求的值

#### 样例输入

```
2
3 2
```

#### 样例输出

```
0
```

 嵌入式Linux系统开发

#### 第四题

编程题 | 20.0分

4/4

### 公司管理

时间限制：5000MS

内存限制：655360KB

题目描述：

在小团的公司中，有 $n$ 位员工。除了最高领导——小团外，每位员工有且仅有一位直接领导。所以，公司内从属关系可以看成一棵树。

现在，公司接到一个项目，需要重新划分这 $n$ 位员工的从属关系。新的划分描述如下：

1. 每个人要么没有下属，要么有至少两个直接下属（即至少有两人的直接领导为这个人）
2. 第 $i$ 个人的下属（包括自己）有恰好 $a_i$ 个。

请注意，直接下属和下属（包括自己）可分别看做树上点的“儿子”和“子树”。

请问是否存在这么一种关系？注意，输入不会给出最高领导的编号。

 嵌入式Linux系统开发

### 输入描述

输入包含多组数据。

对于每组数据，第一行一个整数 $n$ ，表示公司有 $n$ 个人。

接下来一行 $n$ 个数，第 $i$ 个数为 $a_i$ ，含义如题面所示。

( $n \leq 24$ ，数据组数在10以内)

### 输出描述

对每组数据，输出一行"YES"或"NO"，代表是否存在这一种从属关系。

嵌入式Linux系统开发

### 样例输入

```
3
1 1 3
2
1 2
```

### 样例输出

```
YES
NO
```

### 提示

对于第一组样例，1和2的直接领导均为3即可

对于第二组样例，无法构造出符合题目要求的关系。注意每个有下属的人至少有2个直接下属。

嵌入式Linux系统开发

## 多选题

多选题 | 20.0分

特别说明：多选题错选不得分，少选得部分分。

请为横线处选择合适的程序，使得C++程序段的输出结果是02()

```
class T
{
public:
 T(int i=5) {x=i; _____;}
 static void Count() {cout<<c;}
private:
 int x;
 static int c;
};
int T::c= 0;
int main()
{
 T::Count();
 T t1(0);
 T t2;
 T::Count();
 return 0;
}
```

嵌入式Linux系统开发

## 16、康冠医疗笔试题

笔试时间：2020.09.24。

岗位：嵌入式软件工程师。

题型：13道题，40分钟。

6道填空，2道简答，5道编程，时间紧任务重。

### 1、填空

#### 一、填空题（20分）

第1题：

1、（6分）在一个32位操作系统中，下列类型分别占用多少字节？char:、short:、int:、long:、float:、double:

空1: 1

空2: 2

空3: 4

空4: 4

空5: 4

空6: 8

 嵌入式Linux系统开发

第2题：

5、（4分）在printf("output intval=%d", i)中，使用%d来输出10进制int类型数据，那么输出下列类型时分别使用 long:、float:、char\*:、16进制int:

空1: %ld

空2: %f

空3: %d

空4: %0x

 嵌入式Linux系统开发

第3题：

3、（2分）在32位操作系统且不考虑边界对齐的情况下，请问 sizeof(A)=  
struct A { char t:4; char k:4; unsigned short i:8; unsigned long m; }

空1: 8

嵌入式Linux系统开发

4、考察extern关键字。

第5题：

2、（2分）已有定义 char a[]="12345"和char\* b="12345"，请问：sizeof(a)= 和sizeof(b)=

空1: 6

空2: 4

嵌入式Linux系统开发

6、const可以用来代替define，define 只是简单的代替，但是const还会进行类型检查。

怎么避免头文件重复包含：

```
#ifndef H_HEADFILENAME
```

```
#define H_HEADFILENAME
```

文件内容

```
.... #endif
```

2、简答

第7题：

1、malloc/free和new/delete有什么主要区别？

- 1、malloc/free是库函数，以字节为单位申请内存
- 2、new/delete是关键字，以类型为单位申请内存
- 3、malloc和free单纯的堆内存进行申请和释放
- 4、对于基本类型new关键字会对内存进行初始化
- 5、对于类型new和delete还负责构造函数和析构函数的调用

第8题：

2、静态内存、栈内存、堆内存各如何申请？

- 1、静态内存用static关键字去分配
- 2、栈内存是系统自动存储局部变量
- 3、堆内存用malloc分配，用free释放

3、编程题

三、附件题综合论述及编程题（60分）

第9题：

3、（5分）下面代码有什么问题？并改正。

```
void test2() { char string[10], str1[10]; for(i=0; i<10; i++) { str1[i] = 'a'; } strcpy(string, str1); }
```

上传文件 未选择文件

 嵌入式Linux系统开发

此题用的拷贝函数不安全，容易越界造成内存溢出，用strncpy

```
1 #include<stdio.h>
2 #include<string.h>
3
4
5 int main(){
6
7 int i;
8 char string[10]={0},str1[10]={0};
9 for(i=0;i<10;i++){
10 str1[i]='a';
11 }
12 strncpy(string,str1,sizeof(string));
13 //printf("%s\n",string);
14 //printf("%s\n",str1);
15 return 0;
16 }
```

第10题：

1、（10分）请编写一个my\_strcpy(..)函数用于替代标准库中的strcpy(..)函数，要求不得使用C/C++标准库中的函数。

上传文件 未选择文件

 嵌入式Linux系统开发

```

1 #include <stdio.h>
2 char* strcpy(char *strDest, const char* strSrc)
3 {
4 char *p=NULL;
5 if(strDest == NULL || strSrc == NULL)
6 {
7 return NULL;
8 }
9 p = strDest;
10 while((*strDest++ = *strSrc++) != '\0');
11 return p;
12 }

```

第11题：

2、（10分）请编写完成下面函数，在32位操作系统下，要求输入的参数d是2的幂次方时返回true；否则返回false。

例如：d=32时返回true(32=2的5次方)；d=33时返回false。

bool check(unsigned int d)

```

{
}

```

上传文件 未选择文件

 嵌入式Linux系统开发

```

1 bool check(unsigned int d)
2 {
3 if(d%2==0){
4 return 1;
5 }
6 else
7 {
8 return 0;
9 }
10 }

```

第12题：

4、（5分）请写出如下代码的输出：

main()

```
{
```

```
int a[5]={1,2,3,4,5};
```

```
int *ptr=(int *)&a+1;
```

```
printf("%d,%d\n",*(a+1),*(ptr-1));
```

```
}
```

上传文件 未选择文件

 嵌入式Linux系统开发

答案：2，5

第13题：

5、（30分）写一个程序，用来存储学生的学号和成绩，并按分数从低到高排序，请用链表实现。

例如用户输入：1,90<回车>、2,100<回车>、3,80<回车>、OK<回车>，输出：3=80<换行>、1=90<换行>、2=100<换行>

说明：

<回车>指输入完成后按下ENTER按键

OK表示录入学生学号和成绩完成

<换行>指输出一个换行符

上传文件 未选择文件

 嵌入式Linux系统开发

此题代码较长，就不贴了，大家可以百度到。

## 17、景嘉微电子笔试题

笔试时间：2020.10.11。

岗位：嵌入式软件开发工程师。

题型：60分钟,45道题，时间紧任务重。

选择题25道，判断题12道，填空题5道，编程题3道。

长沙景嘉微电子，在长沙找嵌入式工作，景嘉微的薪资是top级别的。并且公司有很多开发平台，都可以去应聘试试。

选择题：

1、求输出

```
1 #include<stdio.h>
2 #include<string.h>
3 int main()
4 {
5 int a,b;
6 a=0x12345678;
7 char c=0x0988264301;
8 char str1[]="TEST";
9 char str2[]="\r\n\0";
10 printf("%d %d %d %d",sizeof(a),sizeof(c),sizeof(str1),sizeof(str2));
11 return 0;
12 }
```

2、

第2题：

现已定义整形变量int i=1;执行循环语句while(i++<=4);后i的值为（ ）。

☐ 1

☐ 5

☐ 6

☐ 以上三个答案都不正确

 嵌入式Linux系统开发

3、

第3题：

某实时操作系统，系统运行节拍为100Hz，系统提供基于系统运行节拍的延时函数taskDelay()。理论上，taskDelay(10)延时时长为（ ）。

- ☐ 10ms
- ☐ 200ms
- ☐ 90ms~100ms
- ☐ 180ms~200ms

 嵌入式Linux系统开发

4、

第4题：

D/A变换器中，分辨率表示它的一个LSB（最低有效位）输入输出变化的程度。分辨率通常用D/A变换器输入的二进制位数来描述，如8位，10位，12位等。对于一个10位D/A变换器来说，当它的输出电压范围为0~+5V时，则其分辨率为（ ）。

- ☐ 19.5mv
- ☐ 4.88mv
- ☐ 9.7mv
- ☐ 2.44mv

 嵌入式Linux系统开发

5、

第5题：

```
int x = 1;
if (10 && (x = 2));
 x = 3;
执行后，x 的值为（ ）
```

- ☐ 1
- ☐ 3
- ☐ 4
- ☐ 10

 嵌入式Linux系统开发

6、

第6题：

以下表示函数指针的是（ ）

- ☒ int token;
- ☐ int (\*token)();
- ☐ int token[];
- ☐ int \*token();

 嵌入式Linux系统开发

7、

第7题：

如果算数运算符中的一个操作数是有符号整数，另一个是无符号整数，那么（ ）。

- ☐ 如果不发生“溢出”，则什么都不会发生
- ☐ 无符号整数会被转换为有符号整数
- ☐ 有符号整数会被转换为无符号整数
- ☐ 转换为浮点数进行计算

 嵌入式Linux系统开发

8、

第8题：

若有定义`#define F 3+4`，则表达式`F*F`的值为（ ）

- ☐ 11
- ☐ 19
- ☐ 25
- ☐ 49

 嵌入式Linux系统开发

9、

第9题：

宏定义`#define f(x) ((x) - 1)`，`f`后面有空格，实际的宏定义代表什么（ ）

- ☐ `((x) - 1)`
- ☐ `(x) ((x) - 1)`
- ☐ `x - 1`
- ☐ `xx - 1`

 嵌入式Linux系统开发

10、

第10题：

`unsigned int *p = (unsigned int *)0x1000;`

`p = p + 1;`

`p = (unsigned int *)((unsigned int)p+1);` `p`所指向的地址为（ ）

- ☐ 0x1002
- ☐ 0x1005
- ☐ 0x1004
- ☐ 0x1008

 嵌入式Linux系统开发

第六行那里p+1其实不是加一个字节，是加了四个字节，所以就执行完第六行代码后p就是1004了，然后第七行代码，是先强制类型转换，再去+1，所以这个时候的+1是真的只是+1，就变成了1005。

11、

**第11题：**

全局变量int global = 1；在生成可执行程序后，该变量被存放于下列哪个段（ ）

- ☐ .bss
- ☐ .comment
- ☐ .text
- ☐ .data

 嵌入式Linux系统开发

12、

**第12题：**

以下代码片的运行结果是（ ）。

```
char str1[20] = "abcd"; strcat(str1, '!'); printf("%s\n", str1);
```

☒ abcd

☐ abcd!

☐ !

☐ 运行错误

 嵌入式Linux系统开发

13、

**第13题：**

判断a和b 按位与的值是否不等于0，语句正确的是（ ）

☒ if((a&b) != 0)

☐ if(a & b != 0)

☐ if((a&&b)!= 0)

☐ if(a&&b != 0)

 嵌入式Linux系统开发

14、

第14题：

下面说法错误的是（ ）。

- A. 预处理使用连接运算符“##”可以把两个独立的符号连接成一个符号
- B. 下述语句将在屏幕上打印为“x is equal to 20.”  
`#define DEBUG_VALUE(v) printf(#v" is equal to %d.\n", v)`  
`int x=20; DEBUG_VALUE(x);`
- C. 分号可以定义在预处理宏内
- D. 以下代码片段能正常编译运行  
`#define HTTYPE unsigned int *`  
`HTTYPE var1, var2; *var2 = 4;`

☐ A

☐ B

☐ C

☐ D

 嵌入式Linux系统开发

15、

第15题：

以下代码片的运行结果是（ ）

```
int a = 1, b = 2, c = 3, d=4;
int r;
r = a>b?c:c>b?a:d;
printf("%d\n", r);
```

☒ 1

☐ 2

☐ 3

☐ 4

 嵌入式Linux系统开发

16、

第16题：

C 标准库函数中,常用的比较字符串大小的函数是（ ）

☒ strcmp

☐ strstr

☐ strtok

☐ strcpy

 嵌入式Linux系统开发

17、

第17题：

以下代码的输出结果是（ ）

```
void func(char**p)
{
 char*t;
 t =(p+= sizeof(int)) [-2];
 printf("%s", t);
}
main() {
 char* argv[]={"ab", "cd", "ef", "gh", "ij", "kl"};
 func(argv);
}
```

- ☐ ab
- ☐ cd
- ☐ ef
- ☐ gh

 嵌入式Linux系统开发

18、

第18题：

printf("%%d prints test")执行结果为（ ）

☐ 编译报错;

☒ %d prints test;

如果想输出一个百分号，输出%，就要%%这样

☐ d prints test;

☐ %%d prints test;

 嵌入式Linux系统开发

来输出

19、

第19题：

实时操作系统中对任务阻塞影响最小的操作是（ ）

- ☐ 信号量
- ☐ free()释放内存
- ☐ read()读取操作
- ☐ 消息队列

 嵌入式Linux系统开发

20、

第20题：

下列对C语言编译过程描述正确的是（ ）

- ☒ 编译过程主要是词法分析、词义分析、语法分析和语义分析；
- ☐ 编译过程最终产生二进制可执行文件；
- ☐ 编译过程是完全可逆的；
- ☐ 如果源码丢失，只有编译生成的可执行文件，则不可进行反汇编操作。

21、

第21题：

下面中断处理函数说法错误的是（ ）

- ☐ 中断是实时系统对异步事件的处理方式
- ☒ Linux系统下一个函数被定义为中断服务程序的时候，编译器会自动为该函数增加中断服务程序所需要的中断现场入栈和出栈代码
- ☐ 中断服务程序内可以包含运行时间在50~100ms以内的浮点运算
- ☐ 中断服务程序必须短小精悍

嵌入式Linux系统开发

22、

第22题：

hi 与 low 为0~15间的整数，r 低4位与 low 各位上的数一致，r 高4位与 hi 各位上的数一致，求 r 的表达式正确的有()

- ☐  $r = hi \ll 4 + low;$
- ☐  $r = hi \ll (4 + low);$
- ☒  $r = (hi \ll 4) + low;$
- ☐  $r = hi \ll 4 | low;$

嵌入式Linux系统开发

23、

第23题：

float \*fp; 说法正确的有()

- ☐ \*fp 是一个浮点数
- ☐ fp 是一个指向浮点数的指针
- ☐ fp 可以初始化为 NULL
- ☐ \*fp 是一个整数

嵌入式Linux系统开发

24、

第24题：

比+ - 优先级高的运算符有()

☒ ( )

☐ !

☐ %

☐ |

 嵌入式Linux系统开发

25、

第25题：

static关键词可以修饰下面哪些内容()

☒ 变量

☐ 文件

☐ 函数

☐ 函数返回值

 嵌入式Linux系统开发

判断题：

26、

第26题：

C语言中，每一个if语句都必须要写一个else与之匹配。()

☐ 对

☐ 错

 嵌入式Linux系统开发

27、

第27题：

switch(c) 语句中c可以是int,long,char,unsigned int 类型。()

☐ 对

☐ 错

 嵌入式Linux系统开发

28、

第28题：

有符号整数右移可能最高位补符号位。()

☐ 对

☐ 错

 嵌入式Linux系统开发

29、

第29题：

C语言中，switch语句中，每一条case下都必须写一个break；跳出()

☐ 对

☐ 错

 嵌入式Linux系统开发

30、

第30题：

`h_score = v_score = 0;`与下面两个语句表达的意思一样

`v_score=0;`

`h_score=v_score;()`

☐ 对

☐ 错

 嵌入式Linux系统开发

31、

第31题：

`int timer[24][60]`是一个数组，它拥有24个数组类型的元素，其中每个元素都是一个拥有60个整型元素。

☐ 对

☐ 错

 嵌入式Linux系统开发

32、

第32题：  
以下代码片段的打印结果是：4

```
void ht(void)
{
 int n = 1;
 {
 int n = 4;
 }
 printf("%d\n", n);
}
```

☐ 对

☐ 错

 嵌入式Linux系统开发

33、

第33题：  
当p是指针时, if(p)是合法的表达式 ( )

☐ 对

☒ 错

 嵌入式Linux系统开发

34、

第34题：  
(i & j) 和 (i && j) , 其结果始终是一样的 ( )

☐ 对

☒ 错

 嵌入式Linux系统开发

35、

第35题：  
(i & ++j)和(i && j++) , 其j的值都一定会自增。( )

☐ 对

☐ 错

 嵌入式Linux系统开发

36、

第36题：  
(i | ++j)和(i || j++)执行效果是一样的。( )

☒ 对

☐ 错

 嵌入式Linux系统开发

37、

第37题：

`int buffer[N]; int *p; 则if(p == &(buffer[N-1]))的写法是非法的。( )`

☒ 对

☐ 错

嵌入式Linux系统开发

填空题：

38、

第38题：

`sizeof("HELLO""JINGJIA!\n")`计算出的字符串长度为\_\_字节。(2分)

15

嵌入式Linux系统开发

39、

第39题：

鱼分三段，鱼头鱼身鱼尾，鱼头长5，鱼尾等于鱼头加半个鱼身，鱼身等于鱼头加鱼尾，鱼全长为\_\_。(2分)

40

嵌入式Linux系统开发

40、

第40题：

`0x01<<2+4`的值是\_\_。(2分)

64

嵌入式Linux系统开发

41、

第41题：

C语言代码 (2分)

```
switch (color){
 case 1: printf("read"); break;
 case 2: printf("blue"); break;
 case 3: printf("yellow"); break;
}
```

color 值为 2 时,上述语句会打印\_\_\_\_\_

blue

嵌入式Linux系统开发

42、

(char)((~((int)(0x06EA6F09) & (0xE1A39F6B))) >> 16) = 0x\_\_\_\_\_ (16进制)。(2分)

嵌入式Linux系统开发

43、

以下是某排序函数中用于比较并获取最大者的语句，请改正代码中的错误：  
(请将答案作为附件上传，支持Word、TXT、Excel、图片上传)

嵌入式Linux系统开发

(请将答案作为附件上传, 支持Word、TXT、Excel、图片上传)

◎ 嵌入式Linux系统开发

(请将答案作为附件上传, 支持Word、TXT、Excel、图片上传)

嵌入式Linux系统开发

题型：选择题8道，填空题10道，编程题4道。

杰发科技主要做汽车电子，由北京四维图新控股，对汽车电子感兴趣的有机会可以应聘试试。

选择题：

1、128, 4

```
1 #include<stdio.h>
2 unsigned int getstrsize(char *str)
3 {
4 return sizeof(str);
5 }
6 int main()
7 {
8 char str[128]={0};
9 printf("%d,%d\n",sizeof(str),getstrsize(str));
10 return 0;
11 }
```

sizeof(数组名)等于数组的大小。str是指针，所以sizeof指针就是等于4。

2、autochips

```
1 #include<stdio.h>
2
3 void print(char **str)
4 {
5 ++str;
6 printf("%s\n",*str);
7 }
8
9 int main()
10 {
11 static char *arr[]{"navinfo","autochips","c++"};
12 char **ptr;
13 ptr=arr;
14 print(ptr);
15 return 0;
16 }
```

3、代码编译错误。

```
1 #include<stdio.h>
2
3 int main()
4 {
5 char *str="Autochips";
6 str[0]='\0';
7 printf("%s\n",str);
8 return 0;
9 }
```

字符串，在文字常量区，叫.rodata，不可以改变，改变会导致段错误。

4、8, 11

```

1 #include<stdio.h>
2
3 int main()
4 {
5 int a[]={8,2,9,4,1},*p;
6 p=a;
7 *(p+2)+=2;
8 printf("%d,%d",*p,*(p+2));
9 return 0;
10 }

```

## 5、求输出

```

1 #include<stdio.h>
2
3 int main()
4 {
5 int x=0xfedcb98;
6 int y=0x76543210;
7 //printf("%d %d\n",x,y);
8 int c;
9 c=(y>>8&0xffff)|(x<<4&~0xff);
10 printf("%d\n",c);
11 return 0;
12 }

```

## 6、12

```

1 #include<stdio.h>
2
3 union package
4 {
5 char head;
6 int body;
7 };
8 struct message
9 {
10 char id;
11 int crc;
12 union package pack;
13 };
14 int main()
15 {
16 int c;
17 c=sizeof(struct message);
18 printf("%d\n",c);
19 return 0;
20 }

```

## 7、一个栈的入栈顺序是a,u,t,o,c,h,i,p,s，则出栈顺序不可能是？

- 1 A、ohpisc tua
- 2 B、uoipshcta
- 3 C、otuaspihc
- 4 D、tuaihocsp

8、有如下定义enum Autochips{SD1,SD2,SD3,QA=7,SD4,SD5};则常量名SD3和SD5代表的值分别是？

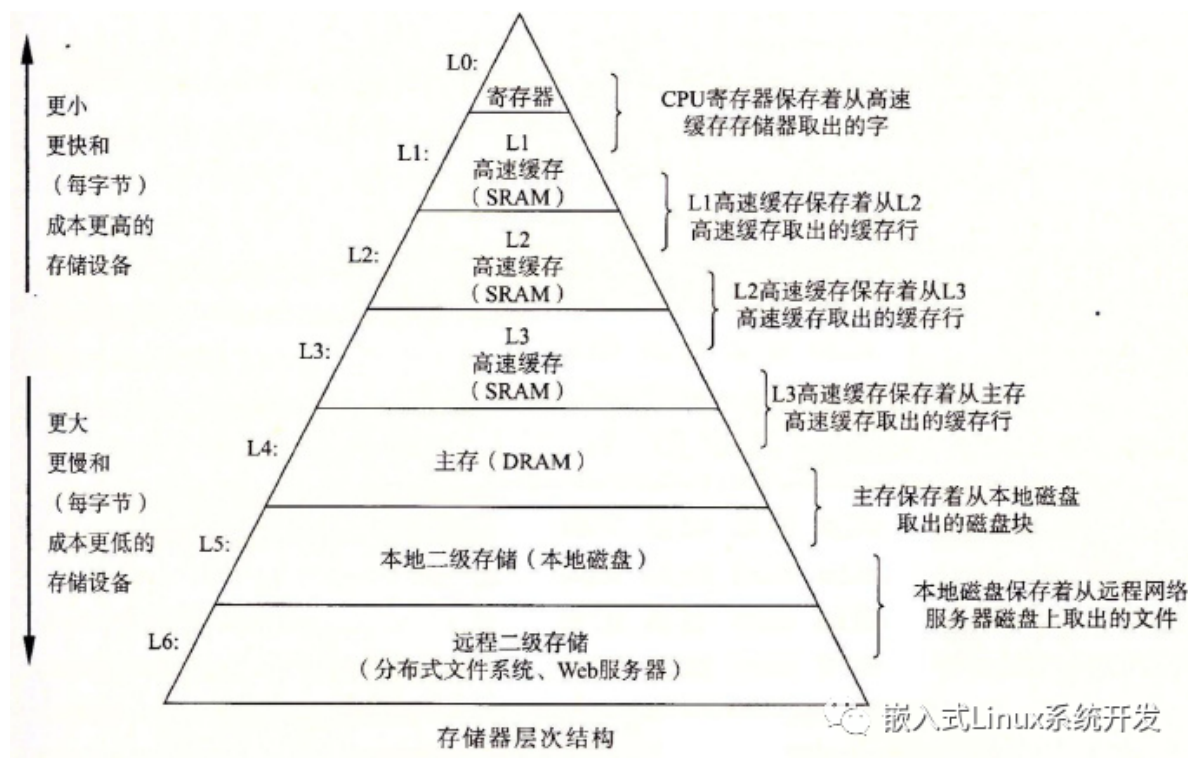
答案：2,9。

考察枚举（从0开始，第一个枚举成员的默认值为整型的0，后续枚举成员的值在前一个成员上加1。如有定义值，后面按定义值增加）

填空题：

1、有一个有序表，{1,4,6,10,18,35,42,53,67,71,78,84,92,99}，当用二分查找法查找键值为84的节点时，所需要的比较次数为？

2、考察存储速度排序



由快到慢：寄存器、cache、DRAM、本地磁盘。

3、以下C程序的运行结果是？

```
1 int main(){
2 int a[]={8,2,9,4,1},*p;
3 p=a;
4 *(p+2)+=2;
5 printf("%d, %d",*p,*(p+2));
6
7 return 0;
8 }
```

4、若栈S和队列Q的初始化状态均为空，元素abcdefghijklmn依次入栈A且每个元素出栈后会立即进入队列Q。若元素出队顺序是bacdeihgflkjmn，则栈S的容量至少是？

5、已知x=0xfedc698和y=0x76543210，则(y>>8&0xffff)|(x<<4&~0xff)值为？

6、

```

1 union package
2 {
3 char head;
4 int body;
5 }
6 struct message
7 {
8 char id;
9 int crc;
10 union package pack;
11 }

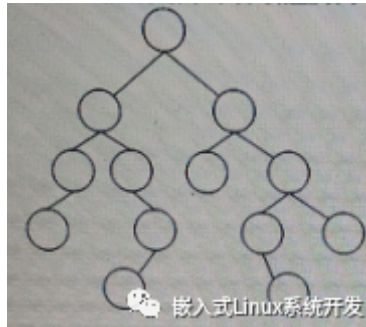
```

则sizeof ( struct message ) 为 ?

7、一台计算机有10个资源，被4个进程竞争，每个进程至少需要\_\_个资源，系统才会有发生死锁的危险。

8、一个16进制数0x45678910存储在起始地址为0x3000的存储空间中，若地址0x3003里的内容是0x10，则此种存储方式被称为？

9、已知二叉树的后序遍历为naviautochips，则前序遍历为？



10、16

```

1 #include<stdio.h>
2 #define f(x) x*(x)*x
3
4 int main()
5 {
6 int a=3;
7 int s=f(a+1);
8 printf("%d\n",s);
9 return 0;
10 }

```

编程题：

1、射击训练中，假设有n (n为整数且n<=40)发子弹，小明可以选择次连续打出两发子弹，也可以选择一次只打出一发子弹，请问小明要打完这n发子弹共有多少种方法。

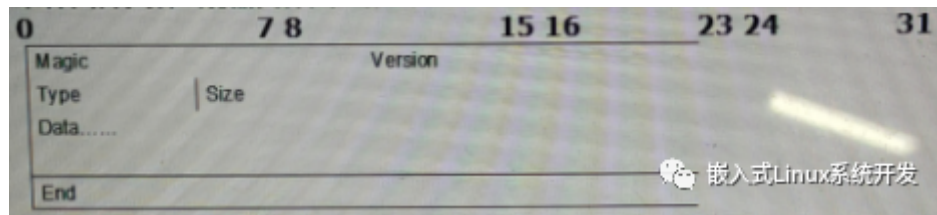
函数原型:

unsigned int calcMethodNum(int n);其中n为子弹数目。

(1)请用文字或流程图简要描述您的思路。

(2)请用C语言编程实现，请考虑执行效率及资源消耗情况。

2、某种图片文件的数据结构布局如下:



此图片文件具有以下特征:

1. Magic字段16 bit,固定为0xFEFE
2. Version字段16 bit,高6 bit是主版本号,低10 bit是次版本号,最大有效版本号是"6.10"
3. Type字段为8 bit,固定是0x1
4. Size字段为24 bit,表示Data字段的有效长度(单位: Byte)
5. 为保证End字段起始地址是在4 Byte对齐的位置,Data字段尾部可能会填充几个无效的0x00
6. End字段是0xABABABAB

在32位大端系统中加载一个文件到内存中, 请根据以上信息编写函数来判断该文件是否为这种图片文件。

函数原型为:

```
bool verifyImage(const char* const image, unsigned int length);
```

// image参数为文件加载到内存中的地址, 在4 Byte对齐位置; length代表文件的大小。验证通过返回true,反之返回false.

要求:

- (1)请用文字或流程图简要描述您的思路。
- (2)请用C语言编程实现, 注意时间和空间效率。

3、已知单向链表 L 的节点的结构体Node如下所示, 节点个数不小于2, 请编写函数实现将链表L平均截为2段L1和L2,它们的段内节点顺序保持不变, 节点个数相差不超过1.

```
1 typedef struct Node {
2 int data;
3 struct Node* pNext;
4 } Node_ T;
5
6 typedef Node* PL inkHead;
```

函数原型为:

```
void SplitLink(Node *PL inkHead, Node *pLink1head, Node *pLink2head)
```

- (1)请用文字或流程图简要描述您的思路(5分)。
- (2)请用C语言编程实现, 请考虑执行效率及资源消耗( 10分)。

4、IPv6 地址由8组16进制的数字来表示, 每组表示16比特。这些组数字通过(":")分割。比如, 2001:0db8:85a3:0000:0000:8a2e:0370:7334是一个有效的地址。而且, 我们可以加入一些以0开头的数字, 字母可以使用大写, 也可以是小写。所以, 2001:db8:85a3:0:0:8A2E:0370:7334也是一个有效的IPv6 address地址(即, 忽略0开头, 忽略大小写)。

然而，我们不能因为某个组的值为0,而使用一个空的组，以至于出现(::)的情况。比如，2001:0db8:85a3::8A2E:0370:7334是无效的IPv6地址。

同时，在IPv6地址中，多余的0也是不被允许的。比如，

02001:0db8:85a3:0000:0000:8a2e:0370:7334是无效的。

要求:

编写一个函数来验证输入的字符串是否是有效的IPv6地址。函数原型为:

```
bool validIPv6Address(char * IP)
```

1).简单描述你的思路(5分)

2).请实现函数来验证是否为有效IPv6地址，如果是，返回true,否则返回false,请注意代码格式(10分)

## 19、360未来安全研究院笔试题

笔试时间：2020.04.16，15:00-17:30。

岗位：Linux 安全开发工程师（实习生）

题型：

能力测试——逻辑题（20个×5分=100分）

专业测试——客观题（40个×2分=80分）

专业测试——在线编程题（2个×25分=50分）

逻辑题：

一共40道题目，很多逻辑推断题，包含数字找规律和图片找规律，还有一些高中概率计算（C42那种，红球白球），二元一次方程和一元二次方程的计算。

客观题：

1、Linux系统中vi编程，挂起进程用什么操作？

2、Squid服务acl对象的类型有？

3、Shell程序中，在test.sh文件第六行加入 `i='expr$i+1'`，则标准输入输出为？

4、考察银行家算法。

5、关于死锁，错误的是？

6、传输对象模式？

7、创作模式？

8、C++语言的程序题，有问结果的，有让你补全程序的，包含9、11、12、13、14、16、17、18、19、20、21、22、23、24、25、26、27这些题目。必须熟悉C++语言，占了一半的分数。

15、二维数组中，如何访问第二行第四列数据（考察二维数组如何+1变到下一行）

28、TCP中三次握手和四次挥手的解释。

29、队列中不可能的出队顺序，《啊哈！算法》原题：

新学期开始了，小哈是小哼的新同桌，小哼向小哈询问 QQ 号，小哈当然不会直接告诉小哼啦，原因嘛你懂的。所以小哈给了小哼一串加密过的数字，同时小哈也告诉了小哼解密规则。规则是这样的：首先将第 1 个数删除，紧接着将第 2 个数放到这串数的末尾，再将第 3 个数删除并将第 4 个数放到这串数的末尾，再将第 5 个数删除.....直到剩下最后一个数，将最后一个数也删除。按照刚才删除的顺序，把这些删除的数连在一起就是小哈的 QQ 啦。现在你来帮帮小哼吧。小哈给小哼加密过的一串数是“6 3 1 7 5 8 9 2 4”。

30、考察二维数组。

31、判断有向图是否存在回路，需要什么算法？

32、动态规划和分治法。

33、拓扑排序。

34、线性规划问题的约束不等式为小于等于不等式时，可以通过什么转化为标准型？

35、NPC问题需要满足的条件是？

36、不是NP问题的是？

37、KMP算法模式中（ abcdefghijklmn的next数组为 ）？

38、考察最小生成树。

39、Abcdefghijklmn子字符串的个数是？如'a'b'等。

40、考察数据模型

两道编程题：

1、最后赢家

时间限制：C/C++语言 1000MS；其他语言 3000MS

内存限制：C/C++语言 65536KB；其他语言 589824KB

题目描述：

最强的不一定是最后的赢家。

某赛事有n名选手参加，但是不同于其他的比赛，本比赛采取的是擂台赛的形式，n名选手排成一排，每次队伍的第一位和第二位选手进行比赛，输的一方会排到队尾。

当某位选手取得m连胜时，他将成为最后的赢家，且游戏结束，请问截止到游戏结束，共会进行多少次比赛。

两位选手的比赛结果由他们的战斗力决定，n位选手的战斗力是一个1~n的排列，也就是说他们的战斗力两两不同，不会有平局的情况。

输入

输入第一行包含两个正整数n，m，分别代表参赛选手数量和取得连胜的要求。(1<=n<=100000，1<=m<=10^9)

输入第二行包含n个正整数，中间用空格隔开，第i个数表示队伍的第i位选手的战斗力，整体是一个1~n的排列。

输出

输出仅包含一个正整数，表示截止到游戏终止，共进行多少场比赛。

样例输入

```
1 | 4 2
2 | 1 3 2 4
```

样例输出

```
1 | 2
```

提示:样例解释

显然第一局应该是战斗力为3的选手获胜，第二局同样是战斗力为3的选手获胜，2连胜终止游戏，所以答案是2。此时若修改m为3，则结果是5。

## 2、随机减法

时间限制：C/C++语言 2000MS；其他语言 4000MS

内存限制：C/C++语言 131072KB；其他语言 655360KB题目描述：

给定一个数 $a_0$ ，定义如下随机序列 $a_1, a_2, \dots, a_n$ 。

1、从闭区间 $[0, a_0]$ 中等概率随机选出一个整数 $k_0$ ，令 $a_1 = a_0 - k_0$ 。

2、得到随机数 $a_1$ 之后，再从闭区间 $[0, a_1]$ 中等概率随机选出一个整数 $k_1$ ，再令 $a_2 = a_1 - k_1$ 。

3、一般地，得到随机数 $a_i$ 之后，再从闭区间 $[0, a_i]$ 中等概率随机选出一个整数 $k_i$ ，令 $a_{i+1} = a_i - k_i$ 。

问 $a_n = 0$ 的概率是多少？

输入

输入两个整数 $n, a_0 (1 \leq n, a_0 \leq 100)$ 。

输出

输出概率，小数点后四舍五入保留5位小数。

样例输入

```
1 | 3 3
```

样例输出

```
1 | 0.72049
```

输入样例2

```
1 | 1 3
```

输出样例2

```
1 | 0.25000
```

输入样例3

```
1 | 100 3
```

1 | 1.00000

## 20、腾讯微民保险笔试题

笔试时间：2020.04.14，20:00-21:30。

岗位：系统测试实习生

题型：单选题（3分×10），判断题（2分×5），问答题（10分×2），编程题（20分×2）。

控股关系如下：深圳腾讯 → 北京驿码神通信息技术有限公司 → 深圳腾富博投资有限公司 → 微民保险代理有限公司。

微保WeSure是腾讯首家控股的保险平台。

单选题（3分×10）：

- 1、递归函数最后递归出来，代表？
- 2、同步机制遵循的规则
- 3、内存四区中，静态变量存在哪个区域？
- 4、考察#define宏定义的边际效应。
- 5、如果 $154=112$ ，是采用的几进制？
- 6、1.2.3.4...15中，甲取5的倍数，甲乙不相同，那么甲比乙大的概率是多少？
- 7、考察struct结构体和指针。
- 8、HTTP中返回值403代表什么？
- 9、HTTP基础知识。
- 10、引用&和指针的区别？

判断题（2分×5）：

- 1、网络字节序是大端还是小端？
- 2、考察TCP三次握手？
- 3、#include中<>和" "的区别？
- 4、C++中异常处理，如果没有抛出异常的代码段，是不是不会占用程序的时间和空间？
- 5、C++中纯虚函数的目的就是为了实现？

问答题（10分×2）：

- 1、手写strcpy拷贝函数，自己写main函数调用。
- 2、数据库查询，用SQL查询语句，查询大于80分的同学，该如何实现？

编程题（20分×2）：

1、输入三个数scanf ( "" )，判断三个数能不能组成三角形，如果能组成三角形，请判断是什么三角形。

2、为上一个题目设计测试用例。

## 第十章、谢谢

原创不易，如果对你有帮助，谢谢支持。

